Course: COMP 333 — Final Project Phase 1, 2 and 3

Group: O
Names:

- Carson Johnston - **Student ID:** 40312846
- Charlotte Lauzon - **Student ID:** 40285642
- Ava Samimi - **Student ID:** 40048117

Description of the task:

The objective of Phase 1 of this project is to select a large, real-world dataset (>= 1GB). We then do the data retrieval, the wrangling, an EDA with 2 research questions, a baseline model and a report on Jupyter Notebook.

Phase 2 then implements advanced machine learning algorithms, engineers informative features, and applies unsupervised learning techniques. In this Phase, we do comparative analysis and model interpretation. This is a big step in answering our Research Questions.

Phase 3 integrates all phases; the work is refactored into a cohesive and reproducible pipeline. We also conduct comprehensive evaluations regarding our results from Phase 2. It is presented in a live demonstration, where we will explain and defend our choices.

Source of the datasets:

The **US Used Cars** dataset is a public dataset available on Kaggle:
https://www.kaggle.com/datasets/ananaymital/us-used-cars-dataset

Description of the datasets:

The **US Used Cars** dataset exceeds the minimum size requirements, has a large number of features, includes real-world inconsistencies and supports both supervised and unsupervised learning tasks.

It contains detailed information about 3 million used cars details across the United States.
Each row represents a single vehicle listing, wth each column being an attribute.

The *US Used Cars* dataset contains:

- Vehicle Identification (Vehicle identification number, listing ID)
- Vehicle Specifications (Engines, horsepower, fuel, size, legroom, bed, body...)
- Market/Dealer Information (city, ZIP, position, days on market, date of listing)
- Fuel efficiency
- Condition and History (accidents, damages, if it was a cab, if it is new...)
- Categorical and Descriptive Features (colours, description, picture...)

The dataset has a mix of numerical, boolean, high-cardinality categorical and string variables

**The notebook will include: **

1. Data Retrieval
2. Wrangling/Cleaning
3. EDA
4. Baseline Model
5. Advanced Supervised Learning
6. Feature Engineering
7. Unsupervised Learning
8. Interpretation
9. Comprehensive Evaluation

Division-of-labour

The work was split and shared by all team members. More precisely:

Carson:

- **Phase 1**: Step 2 (Wrangling/Cleaning), with contributions to Step 3 (Research Question) and Step 4 (Evaluation, Discussion).
- **Phase 2**: Step 2 (Feature Engineering) with contributions to Step 4 (Interpretation, Feature Importance, Partial Dependence Plots, Insights)
- **Phase 3**: Pipeline Integration of 1.1, 1.2, 2.2, 2.4; Executive Summary

Charlotte:

- **Phase 1**: Step 1 (Data Retrieval), with contributions to Step 2 (Cleaning Pipeline), Step 3 (Research Question), Step 4 (Setup of the model, train/val/test, Linear Regression, metrics and evaluation setup) and the overall introduction and README.md.
- **Phase 2**: Step 1 (Advanced Supervised Learning), with contributions to Step 4 (Interpretation, Feature Importance, Relation to Research Question)
- **Phase 3**: Pipeline integration of 1.3, 1.4, 2.1, 2.3; Step 3.0 Bonus implementation: Stacking model, Anomaly detection. Contributions to Step 3.1 Comprehensive Evaluation, Step 3.3 Limitations and Future Work; Overall documentation and Table of Content

Ava:

- **Phase 1**: Step 3 (EDA), with contributions to Step 4 (discussion) and README.md.
- **Phase 2**: Step 3 (Unsupervised learning).
- **Phase 3**: Comprehensive Evaluation, Ethical Considerations

```
import base64
from IPython.display import display, HTML

def display_pdf(file_path):
    with open(file_path, "rb") as f:
        base64_pdf = base64.b64encode(f.read()).decode('utf-8')
    pdf_display = f'<embed src="data:application/pdf;base64,{base64_pdf}" width="800"
height="500" type="application/pdf">'
    display(HTML(pdf_display))

display_pdf("executive_summary.pdf")
```

Table of Contents

Phase 1

Phase 2

Phase 3

Phase 1

```
# --------------------------------
#
# PHASE 1 PIPELINE
#
# --------------------------------

#import required libraries
import pandas as pd
import seaborn as sns
from scipy import stats
```

```
import matplotlib.gridspec as gridspec
from pandas.api.types import is_numeric_dtype, is_bool_dtype
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    mean_absolute_percentage_error,
    median_absolute_error
)
import numpy as np
import matplotlib.pyplot as plt
import re
import warnings
import os
from IPython.display import display, Markdown

# Avoid unnecessary warnings in output
#warnings.filterwarnings("ignore")

# Doesn't cut tables
pd.set_option('display.width', 1000)
pd.set_option('display.max_colwidth', 50)
pd.set_option('display.max_columns', 30)

# Apply predefined visual theme for consistence in the export
plt.style.use('seaborn-v0_8')
sns.set_theme()

# ------------------------
# ----------------
# 1.1 DATA RETRIEVAL FUNCTIONS
# ----------------

def data_retrieval():
    """
    Retrieves the raw dataset for Phase 1.

    Loads the trimmed raw dataset if it already exists. Otherwise, attempts to
    create it from the full dataset by iteratively reading the CSV in chunks and
    sampling rows from each chunk.

    Returns:
    - testing_cars: raw dataset loaded into a pandas DataFrame

    Raises:
    - FileNotFoundError: if neither the trimmed dataset nor the full dataset is available
    """

    # STEP 1: Load dataset into testing_cars
    trimmed_path = "used_cars_trimmed_raw.csv"
    if os.path.exists(trimmed_path):
```

```
        print("Loading trimmed raw dataset...")
        testing_cars = pd.read_csv(trimmed_path, low_memory=False)
    else:
        # if trimmed dataset is not found, attempt to create trimmed dataset from the
full dataset
        input_file = "used_cars_data.csv"
        output_file = "used_cars_trimmed_raw.csv"
        chunksize = 200_000
        sample_frac = 0.12

        if os.path.exists(input_file):
            # Remove old trimmed file if it already exists
            if os.path.exists(output_file):
                os.remove(output_file)

            first_write = True

            print("Creating trimmed raw dataset iteratively...")

            for chunk_idx, chunk in enumerate(pd.read_csv(input_file,
chunksize=chunksize, low_memory=False)):
                # Sample rows from each chunk without cleaning or changing columns
                sampled_chunk = chunk.sample(frac=sample_frac, random_state=42)

                # Write first chunk with header, append remaining chunks without header
                sampled_chunk.to_csv(
                    output_file,
                    mode="w" if first_write else "a",
                    header=first_write,
                    index=False
                )

                first_write = False
                print(f"Processed chunk {chunk_idx + 1}")

            print("Trimmed raw dataset saved as:", output_file)

        else:
            print("Full dataset not found. Skipping trimmed dataset creation.")
            raise FileNotFoundError(
                "Trimmed dataset not found. Please create used_cars_trimmed_raw.csv from
used_cars_data.csv (see README)."
            )

    display(print("Shape:", testing_cars.shape))
    display(testing_cars.head())
    return testing_cars


# ----------------
# ----------------
# 1.2 WRANGLING/CLEANING FUNCTIONS
# ----------------
```

```python
def quantDDA(df):
    """
    Computes a quantitative data quality audit for a DataFrame.

    Generates a summary table for each feature, including counts of observations,
    unique values, missing values, mode, and, for numeric columns, descriptive
    statistics, outlier counts, skewness, and kurtosis.

    Parameters:
    - df: input pandas DataFrame

    Returns:
    - result: pandas DataFrame containing the audit summary for all columns
    """

# create a list to put all column's summary in
    summary_list = []

# Loop through all columns of the argument dataset
    for column in df.columns:
        # One column is a series, we create a dictionary to store the statistics
        series = df[column]
        summary = {}

        # Store the column name as the feature
        summary["Feature"] = column
        # Store the number of rows of the database
        summary["Number of Observations"] = len(series)
        # Store the number of only valid, non-missing values
        summary["Number of Entries"] = series.count()
        # Store the number of distinct values only
        summary["Number of Unique Entries"] = series.nunique()
        # Store the number of missing values (where its n/a is "true")
        summary["Number of Missing Entries"] = series.isna().sum()

        # Store the mode values as a list
        modes = series.mode()
        summary["Mode(s)"] = list(modes)

        # The categorical columns do not have all the numeric fields;
        # To make sure the table has no issues with columns, we set as NaN
        summary["Number of Outliers (IQR)"] = np.nan
        summary["Number of Extreme Values (top/bottom 1%)"] = np.nan
        summary["Mean"] = np.nan
        summary["Standard Deviation"] = np.nan
        summary["Maximum"] = np.nan
        summary["Minimum"] = np.nan
        summary["Q1"] = np.nan
        summary["Q2 (Median)"] = np.nan
        summary["Q3"] = np.nan
        summary["Skewness"] = np.nan
        summary["Kurtosis"] = np.nan
```

```
        # If its numerical column, compute numeric statistics
        # Otherwise, skip the calculations
        if pd.api.types.is_numeric_dtype(series) and not pd.api.types.is_bool_dtype
(series):
            # We remove missing values to not compute with invalid numbers
            # pandas ignore NaN, but not SciPy, so we drop beforehand
            x = series.dropna()

            # Store all the numerical statistics
            if len(x) > 0:
                summary["Mean"] = x.mean()
                summary["Standard Deviation"] = x.std()
                summary["Maximum"] = x.max()
                summary["Minimum"] = x.min()
                summary["Q1"] = x.quantile(0.25)
                summary["Q2 (Median)"] = x.median()
                summary["Q3"] = x.quantile(0.75)

                # Number of outliers with IQR Method
                IQR = summary["Q3"] - summary["Q1"]
                lower = summary["Q1"] - 1.5 * IQR
                upper = summary["Q3"] + 1.5 * IQR
                # Count values outside the lower and upper bounds
                outliers = x[(x < lower) | (x > upper)]
                summary["Number of Outliers (IQR)"] = outliers.count()

                # Extreme values (Top and Bottom 1%)
                lower_ext = x.quantile(0.01)
                upper_ext = x.quantile(0.99)
                # Counts the number of extreme values of the top and bottom 1%
                extreme = x[(x < lower_ext) | (x > upper_ext)]
                summary["Number of Extreme Values (top/bottom 1%)"] = extreme.count()

                # Use SciPy for skewness and kurtosis
                summary["Skewness"] = stats.skew(x, bias=False)
                summary["Kurtosis"] = stats.kurtosis(x, fisher=True, bias=False)

        # Add the dictionary to list
        summary_list.append(summary)

    result = pd.DataFrame(summary_list)

    # Make sure the columns are in the same order as in assignment
    ordered_columns = [
        "Feature",
        "Number of Observations",
        "Number of Entries",
        "Number of Unique Entries",
        "Number of Missing Entries",
        "Number of Outliers (IQR)",
        "Number of Extreme Values (top/bottom 1%)",
        "Mode(s)",
```

```
        "Mean",
        "Standard Deviation",
        "Maximum",
        "Minimum",
        "Q1",
        "Q2 (Median)",
        "Q3",
        "Skewness",
        "Kurtosis"
    ]

    return result[ordered_columns]

# ------------------

def vizDDA(df):
    """
    Visualizes the structure and quality of a DataFrame.

    Produces a square visualization grid with univariate plots on the diagonal
    and bivariate plots off-diagonal, along with a heatmap of missing values.

    Parameters:
    - df: input pandas DataFrame

    Returns:
    - None
    """

    df = df.copy()

# Sample if too large to avoid slow crosstabs and memory issues
    max_rows = 5000
    if len(df) > max_rows:
        df = df.sample(n=max_rows, random_state=42)

    # Remove high-cardinality categorical columns for visualization grid
    high_cardinality_cols = []

    for col in df.columns:
        if not pd.api.types.is_numeric_dtype(df[col]):
            # put an arbitrary 20 threshold
            if df[col].nunique() > 20:
                high_cardinality_cols.append(col)
    # df for the plots
    df_plot = df.drop(columns=high_cardinality_cols)

    # Limit grid size to avoid memory/speed issues (max 6x6)
    max_features = 6
    if len(df_plot.columns) > max_features:
        df_plot = df_plot.iloc[:, :max_features]
    num_features = len(df_plot.columns)
```

```
    # Detect datetime columns
    is_datetime = {
        column: pd.api.types.is_datetime64_any_dtype(df_plot[column])
        for column in df_plot.columns
    }

    # Create square grid
    # put 4x4 inch space for each subplot for readability
    fig = plt.figure(figsize=(4 * num_features, 4 * num_features))
    grid = gridspec.GridSpec(num_features, num_features)

    # Consider i as index for rows, and j as index for columns
    for i, col_i in enumerate(df_plot.columns):
        for j, col_j in enumerate(df_plot.columns):

            ax = fig.add_subplot(grid[i, j])

            # Detect if its numeric or categorical
            is_num_i = pd.api.types.is_numeric_dtype(df_plot[col_i])
            is_num_j = pd.api.types.is_numeric_dtype(df_plot[col_j])

            # Univariate on diagonal
            # if we compare a feature with itself, we are doing a univariate analysis
            if i == j:

                # If it's a datetime univariate, we count how many times each unique
datetime appear
                if is_datetime[col_i]:
                    series = df_plot[col_i].dropna()
                    if len(series) > 0:
                        series.value_counts().sort_index().plot(ax=ax)
                    ax.set_ylabel("count")

                # if numeric, we do a histogram to visualize the distribution
                elif is_num_i:
                    sns.histplot(df_plot[col_i], kde=True, ax=ax)

                # For categorical univariate, we do a bar chart
                else:
                    df_plot[col_i].value_counts().plot(kind="bar", ax=ax)

            # Bivariate off diagonal
            # We study two different variables
            else:

                # If one variable is datetime, and the other is numeric:
                # We do a line plot
                # Datetime vs Numeric when datetime is the row variable, and numeric is
column variable
                if is_datetime[col_i] and is_num_j:
                    # Select the relevant columns, drop the missing values, and sort by
the datetime column
                    tmp = df_plot[[col_i, col_j]].dropna().sort_values(col_i)
```

```
                    # We draw datetime on x-axis, numeric on y-axis
                    ax.plot(tmp[col_i], tmp[col_j])

                # Same but for the symmetric situation;
                # Datetime vs Numeric when datetime is the column variable, and numeric
is row variable
                elif is_datetime[col_j] and is_num_i:
                    tmp = df_plot[[col_j, col_i]].dropna().sort_values(col_j)
                    ax.plot(tmp[col_j], tmp[col_i])

                # If both variables are numeric (Numeric vs numeric), we do a scatter plot
                elif is_num_i and is_num_j:
                    sns.scatterplot(x=df_plot[col_j], y=df_plot[col_i], ax=ax)

                # If there are mixed types (categorical vs numeric) we do a boxplot
                # For when numerical is the row feature and categorical is the column
feature
                elif is_num_i and not is_num_j and not is_datetime[col_j]:
                    tmp = df_plot[[col_j, col_i]].dropna()
                    tmp[col_j] = tmp[col_j].astype("category")
                    sns.boxplot(data=tmp, x=col_j, y=col_i, ax=ax)

                # For when numerical is the column feature and categorical is the row
feature
                elif not is_num_i and is_num_j and not is_datetime[col_i]:
                    tmp = df_plot[[col_i, col_j]].dropna()
                    tmp[col_i] = tmp[col_i].astype("category")
                    sns.boxplot(data=tmp, x=col_i, y=col_j, ax=ax)

                # If both variable are categorical (Categorical vs categorical), we do a
grouped bar chart
                # Since all cases with numerical variables have been checked,
                # only categorical on both row and column left
                else:
                    categorical = pd.crosstab(df_plot[col_i], df_plot[col_j])
                    categorical.plot(kind="bar", ax=ax)

            # Axis labeling
            # Only label bottom row with x labels
            if i == num_features - 1:
                ax.set_xlabel(col_j, fontsize=15, fontweight="bold")
            else:
                ax.set_xlabel("")

            # Only label left column with y labels
            if j == 0:
                ax.set_ylabel(col_i, fontsize=15, fontweight="bold")
            else:
                ax.set_ylabel("")

    # Add title
    fig.suptitle("Visualization Grid (Univariate on Diagonal, Bivariate Off-Diagonal)",
fontsize=25, fontweight="bold")
```

```
    # Improve spacing so subplots don't overlap
    plt.tight_layout(rect=[0, 0, 1, 0.96])
    plt.show()

    # Heatmap of missing values
    plt.figure(figsize=(12, 6))
    sns.heatmap(df.isna(), cbar=True, vmin=0, vmax=1)
    plt.title("Missing Values Heatmap")
    plt.show()


# --------------


def data_cleaning(df, drop_missing_threshold):
    """
    Cleans and preprocesses the used cars dataset.

    Performs manual column removal, drops columns with high missingness, removes
    duplicate rows, imputes missing numeric values with indicator flags, fills
    missing categorical values, converts measurement strings to numeric values,
    and removes outliers based on dataset-specific rules.

    Parameters:
    - df: input pandas DataFrame
    - drop_missing_threshold: maximum allowed proportion of missing values before
dropping a column

    Returns:
    - df: cleaned pandas DataFrame
    - report: dictionary summarizing the cleaning steps performed
    """

    df = df.copy()

    # manual removal of columns
    copy_columns = ['wheel_system_display', 'engine_cylinders', 'latitude', 'longitude',
'exterior_color', 'listing_id', 'power']
    descriptive_columns = ['description', 'trim_name', 'transmission_display', 'sp_name',
'major_options']
    unecessary_columns = ['trimId', 'dealer_zip', 'savings_amount', 'sp_id',
'seller_rating']


    remove_columns = []
    remove_columns.extend(copy_columns)
    remove_columns.extend(descriptive_columns)
    remove_columns.extend(unecessary_columns)

    df = df.drop(columns=[c for c in remove_columns if c in df.columns], errors='ignore')
```

```
    report = {}

    # Drop columns with extremely high missingness
    missing_rate = df.isna().mean()
    drop_cols = missing_rate[missing_rate > drop_missing_threshold].index.tolist()
    df = df.drop(columns=drop_cols)
    report["dropped_cols_high_missing"] = drop_cols

    # Remove row duplicates
    before = len(df)
    df = df.drop_duplicates()
    report["rows_removed_duplicates"] = before - len(df)

    # Missing values
    # Numeric columns: add missing flag
    imputed_numeric = []
    for col in df.columns:
        if is_numeric_dtype(df[col]) and not is_bool_dtype(df[col]):
            if df[col].isna().any():
                df[f"{col}_missing"] = df[col].isna()
                df[col] = df[col].fillna(df[col].median())
                imputed_numeric.append(col)
    report["numeric_imputed_with_flags"] = imputed_numeric

    # For categorical columns: fill missing with label
    filled_categorical = []
    for col in df.columns:
        if df[col].dtype == "object" and df[col].isna().any():
            df[col] = df[col].fillna("Missing")
            filled_categorical.append(col)
    report["categorical_filled"] = filled_categorical

    # String unification & converting inches measurements into numeric values
    for col in df.select_dtypes(include=["object", "string"]).columns:
        # Standardize strings first
        df[col] = df[col].astype(str).str.strip()

        # Check if the column mostly contains "number in" patterns
        # We'll check the first non-null value as a heuristic
        sample_value = df[col].dropna().iloc[0] if not df[col].empty else ""

        # Regex pattern: look for digits (and optional decimal) followed by 'in'
        if re.search(r'\d+\.?\d*\s*in', str(sample_value).lower()):
            # 1. Extract the number
            df[col] = df[col].str.extract(r'(\d+\.?\d*)').astype(float)

            # 2. Rename the column (e.g., 'wheel_size' -> 'wheel_size_in')
            df = df.rename(columns={col: f"{col}_in"})

    # column "fuel_tank_volume" will be converted to numerical data
    # Ensure the column is treated as a string
    column_as_string = df['fuel_tank_volume'].astype(str)
```

```
    # Extract the numbers (including decimals) using Regex
    # (\d+\.?\d*) looks for digits, an optional dot, and more digits
    df['fuel_tank_volume'] = column_as_string.str.extract(r'(\d+\.?\d*)').astype(float)

    # Rename the column to keep track of the units
    df = df.rename(columns={'fuel_tank_volume': 'fuel_tank_gal'})

    # column "maximum_seating" will be converted to numerical data
    # ensure teh column is treated as a string
    column_as_string = df['maximum_seating'].astype(str)

    # extract the numbers using Regex
    # (\d+\.?\d*) looks for digits, an optional dot, and more digits. I have no idea if
vehicles can have half seats
    df['maximum_seating'] = column_as_string.str.extract(r'(\d+\.?\d*)').astype(float)



    # remove outliers specific to used cars dataset
    # list of columns to completely ignore, just add column names to ignore for testing
    columns_ignore = ['maximum_seating']

    for column in df.columns:
        series = df[column]
        # only compute numeric types
        if pd.api.types.is_numeric_dtype(series) and not pd.api.types.is_bool_dtype
(series):
            # only act on columns that need outliers removed
            if column not in columns_ignore:

                summary = {}

                Q1 = series.quantile(0.25)
                Q3 = series.quantile(0.75)
                IQR = Q3 - Q1

                lower = Q1 - 1.5 * IQR
                upper = Q3 + 1.5 * IQR

                # only remove lower outliers for year
                if column == 'year':
                    df = df[df[column] >= lower]
                elif column == 'mileage':
                    df = df[df[column] <= upper]
                else:
                    # Remove both upper and lower outliers for any other included numeric
columns
                    df = df[(df[column] >= lower) & (df[column] <= upper)]

    return df, report

# ----------------
```

```python
# ----------------
# 1.3 EDA FUNCTIONS
# ----------------

def eda_summary_statistics(df):
    """
    Section 3.1 - Summary Statistics
    Displays top body types, top makes, and returns numeric summary statistics.
    """
    display(Markdown("## 1.3.1 Summary Statistics"))

    numeric_cols = [
        c for c in df.select_dtypes(include=[np.number]).columns
        if not str(c).endswith("_missing")
    ]

    print("\nTop body types:")
    if "body_type" in df.columns:
        display(df["body_type"].value_counts().head(10).to_frame("Count"))
    else:
        print("body_type column not found.")

    print("\nTop makes:")
    if "make_name" in df.columns:
        display(df["make_name"].value_counts().head(10).to_frame("Count"))
    else:
        print("make_name column not found.")


# ----------------

def eda_univariate_visualizations(df):
    """
    Section 3.2 - Univariate Visualizations
    Plots distributions of price, body type, year, and mileage.
    """
    display(Markdown("## 1.3.2 Univariate Visualizations"))

    fig, axes = plt.subplots(2, 2, figsize=(12, 10))

    # Price distribution
    if "price" in df.columns:
        axes[0, 0].hist(df["price"].dropna(), bins=50, edgecolor="black", alpha=0.7)
    axes[0, 0].set_xlabel("Price ($)")
    axes[0, 0].set_ylabel("Count")
    axes[0, 0].set_title("Distribution of Price")

    # Body type distribution
    if "body_type" in df.columns:
        df["body_type"].value_counts().plot(kind="bar", ax=axes[0, 1])
    axes[0, 1].set_xlabel("Body Type")
    axes[0, 1].set_ylabel("Count")
    axes[0, 1].set_title("Distribution of Body Type")
```

```
    axes[0, 1].tick_params(axis="x", rotation=45)

    # Year distribution
    if "year" in df.columns:
        y_min, y_max = int(df["year"].min()), int(df["year"].max())
        axes[1, 0].hist(
            df["year"].dropna(),
            bins=range(y_min, y_max + 2),
            edgecolor="black",
            alpha=0.7
        )
    axes[1, 0].set_xlabel("Year")
    axes[1, 0].set_ylabel("Count")
    axes[1, 0].set_title("Distribution of Vehicle Year")

    # Mileage distribution
    if "mileage" in df.columns:
        axes[1, 1].hist(df["mileage"].dropna(), bins=50, edgecolor="black", alpha=0.7)
    axes[1, 1].set_xlabel("Mileage")
    axes[1, 1].set_ylabel("Count")
    axes[1, 1].set_title("Distribution of Mileage")

    plt.tight_layout()
    plt.show()

# ----------------

def eda_bivariate_visualizations(df):
    """
    Section 3.3 - Bivariate Visualizations
    Plots price vs mileage, price by body type, price by year, and price vs horsepower.
    """
    display(Markdown("## 1.3.3 Bivariate Visualizations"))

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # Price vs Mileage
    if "mileage" in df.columns and "price" in df.columns:
        axes[0, 0].scatter(df["mileage"], df["price"], alpha=0.5)
    axes[0, 0].set_xlabel("Mileage")
    axes[0, 0].set_ylabel("Price ($)")
    axes[0, 0].set_title("Price vs Mileage")

    # Price by Body Type
    if "body_type" in df.columns and "price" in df.columns:
        body_order = df.groupby("body_type")["price"].median().sort_values().index
        sns.boxplot(data=df, x="body_type", y="price", order=body_order, ax=axes[0, 1])
    axes[0, 1].set_xlabel("Body Type")
    axes[0, 1].set_ylabel("Price ($)")
    axes[0, 1].set_title("Price by Body Type")
    axes[0, 1].tick_params(axis="x", rotation=45)

    # Price by Year
```

```
    if "year" in df.columns and "price" in df.columns:
        sns.boxplot(data=df, x="year", y="price", ax=axes[1, 0])
    axes[1, 0].set_xlabel("Year")
    axes[1, 0].set_ylabel("Price ($)")
    axes[1, 0].set_title("Price by Year")
    axes[1, 0].tick_params(axis="x", rotation=45)

    # Price vs Horsepower
    if "horsepower" in df.columns and "price" in df.columns:
        axes[1, 1].scatter(df["horsepower"], df["price"], alpha=0.5)
    axes[1, 1].set_xlabel("Horsepower")
    axes[1, 1].set_ylabel("Price ($)")
    axes[1, 1].set_title("Price vs Horsepower")

    plt.tight_layout()
    plt.show()

# ----------------

def eda_correlation_analysis(df):
    """
    Section 3.4 - Correlation Analysis
    Displays correlation heatmap and correlations with price.
    """
    display(Markdown("## 1.3.4 Correlation Analysis"))

    corr_cols = [
        c for c in df.select_dtypes(include=[np.number]).columns
        if not str(c).endswith("_missing")
    ]

    corr_matrix = df[corr_cols].corr()

    plt.figure(figsize=(12, 10))
    sns.heatmap(
        corr_matrix,
        annot=True,
        fmt=".2f",
        cmap="RdBu_r",
        center=0,
        square=True,
        linewidths=0.5,
        annot_kws={"size": 8}
    )
    plt.title("Correlation Matrix of Numerical Features")
    plt.tight_layout()
    plt.show()

    if "price" in corr_matrix.columns:
        print("\nCorrelation with price (sorted by absolute value):")
        sorted_idx = corr_matrix["price"].drop("price").abs().sort_values(ascending=False
).index
        price_corr = corr_matrix["price"].drop("price").loc[sorted_idx]
```

```
        display(price_corr.to_frame("Correlation with Price"))
        return corr_matrix, price_corr

    return corr_matrix, None

# ----------------

def run_eda(df):
    """
    Runs the full Phase 1 EDA pipeline on the cleaned dataset.
    """
    display(Markdown("# 1.3 Exploratory Data Analysis (EDA)"))

    summary_stats = eda_summary_statistics(df)
    eda_univariate_visualizations(df)
    eda_bivariate_visualizations(df)
    corr_matrix, price_corr = eda_correlation_analysis(df)

    return {
        "summary_stats": summary_stats,
        "corr_matrix": corr_matrix,
        "price_corr": price_corr
    }

# ----------------

# ----------------
# 1.4 BASELINE MODEL FUNCTIONS
# INCLUDES FUNCTIONS FOR REGRESSION AND VISUALIZATIONS USED IN PHASE 2
# ----------------

def base_model_setup(

    df,
    target_col,
    columns_to_drop,
    train_size,
    val_size,
    test_size,
    random_state,
    high_unique_threshold
    ):

    """
    Prepares data for baseline regression.

    This function:
    - Removes the target column and specified columns
    - Drops high-cardinality categorical features (likely identifiers)
    - Keeps only numeric features and handles missing values
    - Splits the data into training, validation, and test sets

    Returns:
```

```
    - X_train, X_val, X_test: feature datasets
    - y_train, y_val, y_test: target datasets
    """

    # Copy of the df
    df_copy = df.copy(deep=True)

    # Set X (features)
    X = df_copy.drop(columns=[target_col] + columns_to_drop, errors="ignore")

    # Drop columns that are almost entirely unique (most likely identifiers)
    high_unique_cols = [
        col for col in X.select_dtypes(include=["object", "string"]).columns
        if X[col].nunique() / len(X) > high_unique_threshold
    ]
    X = X.drop(columns=high_unique_cols, errors="ignore")

    # Baseline simplification: keep only numeric features (avoids string errors)
    X = X.select_dtypes(include=["number"]).fillna(0)

    # Set y (target)
    y = df_copy[target_col]

    # Split 1: train vs temp
    X_train, X_temp, y_train, y_temp = train_test_split(
        X, y,
        test_size=(1 - train_size),
        random_state=random_state
    )

    # Split 2: validation vs test
    relative_test_size = test_size / (val_size + test_size)

    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp,
        test_size=relative_test_size,
        random_state=random_state
    )

    return X_train, X_val, X_test, y_train, y_val, y_test

# ----------------

def linear_regression_training(X_train, y_train, **kwargs):
    """
    Trains a Linear Regression model using the provided training dataset.

    Parameters:
    - X_train: feature matrix for training
    - y_train: target vector for training
    - **kwargs: optional parameters passed to the LinearRegression model

    Returns:
```

```
    - model: fitted Linear Regression model
    """

    # Setting the Linear Regression
    model = LinearRegression(**kwargs)
    model.fit(X_train, y_train)
    return model

# ----------------
# ----------------

# VISUALIZATION FUNCTIONS
# THESE FUNCTIONS CAN BE USED IN BOTH PHASE 1 AND PHASE 2
# THEY PRODUCE VISUALIZATIONS FOR ANY TRAINED MODEL

# ----------------

def plot_actual_vs_predicted(trained_model, X_data, y_data, model_name, dataset_name):
    """
    Plots actual vs predicted values for a regression model.

    Generates a scatter plot on the selected dataset comparing actual target values to
model predictions,
    including a diagonal reference line representing perfect predictions.

    Parameters:
    - trained_model: fitted regression model
    - X_data: feature matrix used for prediction
    - y_data: true target values
    - model_name: name of the model (for labeling)
    - dataset_name: name of the dataset (e.g., Train, Validation, Test)

    Returns:
    - y_pred: predicted values from the model
    """

    #Prediction
    y_pred = trained_model.predict(X_data)

    #Plot
    plt.figure(figsize=(6, 6))
    plt.scatter(y_data, y_pred, alpha=0.3)

    #Perfect prediction line
    plt.plot(
        [y_data.min(), y_data.max()],
        [y_data.min(), y_data.max()],
        color="red",
        linestyle="--",
        label="Perfect Prediction"
    )

    display(Markdown(f"### Price Prediction using {model_name}"))
```

```
    plt.xlabel("Actual Price ($)")
    plt.ylabel("Predicted Price ($)")
    plt.title(f"{model_name}: Actual vs Predicted Prices ({dataset_name})")
    plt.legend()
    plt.grid(alpha=0.2)
    plt.tight_layout()
    plt.show()

    return y_pred

# ------------------------

def plot_residuals(trained_model, X_data, y_data, model_name, dataset_name):
    """
    Plots residuals versus predicted values for a regression model.

    Helps assess model bias, spread of errors, and possible heteroscedasticity.

    Parameters:
    - trained_model: fitted regression model
    - X_data: feature matrix used for prediction
    - y_data: true target values
    - model_name: name of the model (for labeling)
    - dataset_name: name of the dataset (e.g., Train, Validation, Test)

    Returns:
    - residuals: difference between actual and predicted values
    """

    y_pred = trained_model.predict(X_data)
    residuals = y_data - y_pred
    display(Markdown(f"### Residual Analysis for {model_name}"))
    plt.figure(figsize=(6, 5))
    plt.scatter(y_pred, residuals, alpha=0.3)
    plt.axhline(y=0, color="red", linestyle="--")

    plt.xlabel("Predicted Price ($)")
    plt.ylabel("Residual (Actual - Predicted)")
    plt.title(f"{model_name}: Residuals vs Predicted Prices ({dataset_name})")
    plt.grid(alpha=0.2)
    plt.tight_layout()
    plt.show()

    return residuals

# ------------------------

def plot_error_distribution(trained_model, X_data, y_data, model_name, dataset_name):
    """
    Plots the distribution of prediction errors for a regression model.

    Helps assess whether prediction errors are centered around zero and whether
    the error distribution is symmetric or skewed.
```

```
    Parameters:
    - trained_model: fitted regression model
    - X_data: feature matrix used for prediction
    - y_data: true target values
    - model_name: name of the model (for labeling)
    - dataset_name: name of the dataset (e.g., Train, Validation, Test)

    Returns:
    - residuals: difference between actual and predicted values
    """

    y_pred = trained_model.predict(X_data)
    residuals = y_data - y_pred

    display(Markdown(f"### Error Distribution for {model_name}"))
    plt.figure(figsize=(6, 4))
    plt.hist(residuals, bins=50, edgecolor="black", alpha=0.7)
    plt.axvline(x=0, color="red", linestyle="--")

    plt.xlabel("Prediction Error (Actual - Predicted)")
    plt.ylabel("Frequency")
    plt.title(f"{model_name}: Distribution of Prediction Errors ({dataset_name})")
    plt.grid(alpha=0.2)
    plt.tight_layout()
    plt.show()

    return residuals

# ----------------
# ----------------

def create_metrics_table(y_train, y_pred_train, y_val, y_pred_val, y_test, y_pred_test):
    """
    Creates a metrics table for regression model evaluation.

    Computes R2, MAE, RMSE, MAPE, and MedAE for the train, validation, and test sets.

    Parameters:
    - y_train: true target values for the training set
    - y_pred_train: predicted target values for the training set
    - y_val: true target values for the validation set
    - y_pred_val: predicted target values for the validation set
    - y_test: true target values for the test set
    - y_pred_test: predicted target values for the test set

    Returns:
    - metrics_table: pandas DataFrame containing the evaluation metrics
    """

    metrics_table = pd.DataFrame({
        "Set": ["Train", "Validation", "Test"],
        "R2": [
```

```
            r2_score(y_train, y_pred_train),
            r2_score(y_val, y_pred_val),
            r2_score(y_test, y_pred_test)
        ],
        "MAE ($)": [
            mean_absolute_error(y_train, y_pred_train),
            mean_absolute_error(y_val, y_pred_val),
            mean_absolute_error(y_test, y_pred_test)
        ],
        "RMSE ($)": [
            np.sqrt(mean_squared_error(y_train, y_pred_train)),
            np.sqrt(mean_squared_error(y_val, y_pred_val)),
            np.sqrt(mean_squared_error(y_test, y_pred_test))
        ],
        "MAPE (%)": [
            mean_absolute_percentage_error(y_train, y_pred_train) * 100,
            mean_absolute_percentage_error(y_val, y_pred_val) * 100,
            mean_absolute_percentage_error(y_test, y_pred_test) * 100
        ],
        "MedAE": [
            median_absolute_error(y_train, y_pred_train),
            median_absolute_error(y_val, y_pred_val),
            median_absolute_error(y_test, y_pred_test)
        ]
    })

    return metrics_table

# ----------------

def evaluate_linear_regression(model, X_train, y_train, X_val, y_val, X_test, y_test):
    """
    Evaluates a trained Linear Regression model.

    Generates predictions for the train, validation, and test sets, computes
    evaluation metrics, displays a formatted metrics table, creates a coefficient
    table, and produces residual and error distribution plots.

    Parameters:
    - model: fitted Linear Regression model
    - X_train: feature matrix for the training set
    - y_train: target vector for the training set
    - X_val: feature matrix for the validation set
    - y_val: target vector for the validation set
    - X_test: feature matrix for the test set
    - y_test: target vector for the test set

    Returns:
    - dictionary containing predictions, metrics table, coefficients, and residuals
    """

    # Predictions
    y_pred_train = model.predict(X_train)
```

```
    y_pred_val = model.predict(X_val)
    y_pred_test = model.predict(X_test)

    # Metrics table
    metrics_table = create_metrics_table(
        y_train, y_pred_train,
        y_val, y_pred_val,
        y_test, y_pred_test
    )

    display(Markdown("### Model Performance Metrics"))

    display(
        metrics_table.style
        .format({
            "R2": "{:.4f}",
            "MAE ($)": "{:,.2f}",
            "RMSE ($)": "{:,.2f}",
            "MAPE (%)": "{:.2f}",
            "MedAE": "{:,.2f}"
        })
        .set_caption("Baseline Linear Regression Performance (Train / Validation / Test)")
    )

    # Coefficients table
    coefficients = pd.DataFrame({
        "Feature": X_train.columns,
        "Coefficient": model.coef_
    }).sort_values(by="Coefficient", ascending=False)

    residuals = y_test.values - y_pred_test

    # Residual analysis plot
    plot_residuals(model, X_test, y_test, "Baseline Linear Regression", "Test Set")
    # Error distribution histogram
    plot_error_distribution(model, X_test, y_test, "Baseline Linear Regression", "Test
Set")

    return {
        "y_pred_train": y_pred_train,
        "y_pred_val": y_pred_val,
        "y_pred_test": y_pred_test,
        "metrics_table": metrics_table,
        "coefficients": coefficients,
        "residuals": residuals
    }

# ----------------

def run_baseline_linear_regression_pipeline(
    df,
    target_col,
    columns_to_drop,
```

```
    train_size,
    val_size,
    test_size,
    random_state,
    high_unique_threshold,
    **model_kwargs
):
    """
    Runs the full baseline Linear Regression pipeline.

    Combines data preparation, model training, prediction visualization, and
    evaluation into a single reproducible workflow.

    Parameters:
    - df: input pandas DataFrame
    - target_col: name of the target column
    - columns_to_drop: list of columns to exclude from modeling
    - train_size: proportion of data used for training
    - val_size: proportion of data used for validation
    - test_size: proportion of data used for testing
    - random_state: random seed for reproducibility
    - high_unique_threshold: threshold for dropping high-cardinality categorical columns
    - **model_kwargs: optional parameters passed to the LinearRegression model

    Returns:
    - dictionary containing the trained model, datasets, predictions, and evaluation
results
    """

    # Setup
    X_train, X_val, X_test, y_train, y_val, y_test = base_model_setup(
        df=df,
        target_col=target_col,
        columns_to_drop=columns_to_drop,
        train_size=train_size,
        val_size=val_size,
        test_size=test_size,
        random_state=random_state,
        high_unique_threshold=high_unique_threshold
    )

    # Training
    linear_regression_model = linear_regression_training(X_train, y_train, **model_kwargs)

    # Actual vs Predicted plot
    y_pred_test = plot_actual_vs_predicted(linear_regression_model, X_test, y_test,
"Baseline Linear Regression", "Test Set")

    # Evaluation
    evaluation_results = evaluate_linear_regression(
        model=linear_regression_model,
        X_train=X_train,
        y_train=y_train,
```

```
        X_val=X_val,
        y_val=y_val,
        X_test=X_test,
        y_test=y_test
    )

    return {
        "model": linear_regression_model,
        "X_train": X_train,
        "X_val": X_val,
        "X_test": X_test,
        "y_train": y_train,
        "y_val": y_val,
        "y_test": y_test,
        "y_pred_test": y_pred_test,
        "evaluation_results": evaluation_results
    }

# ----------------
```

1.1 Data Retrieval

Phase 1 – Step 1

1.1.1 Source

We are using the **US Used Cars** public dataset from Kaggle: https://www.kaggle.com/datasets/ananaymital/us-used-cars-dataset

This dataset is downloaded as a zipped csv file, and, extracted, has a size of approximately 9.98 GB.
It contains tabular vehicle listing data.

1.1.2 Retrieval

The dataset is retrieved as a **file-based CSV source** and is loaded into Python using the pandas function `read_csv()`.

We selected file-based retrieval, because the data is structured and tabular, and Kaggle provides it as a flat CSV file.
Also, by retrieving the data as a CSV file, it avoids API authentification dependency. Since the dataset is static and doesn't change in real time, the most reproducible and efficient approach is to store the raw file locally.

1.1.3 Challenge Handling

- **Large file size (~10GB):** Because the Data file is nearly 10 GB, direct loading can be slow and memory-heavy.
    - Mitigation:
        - We made a sample of the data of ~1GB so the full trimmed data would be used
            - We read the original file by chunks
            - Sampling was performed uniformly across all chunks of the dataset to ensure that all portions of the data were represented.
        - we use `low_memory=False`. This makes sure the entire file is read before deciding the columns' data type. This prevents dtype warnings.
- **Data type Inconsistencies:** Some numeric variables are stored as strings; During the Data Wrangling Phase, these will be cleaned and converted.
- **Authentication / Rate Limits:** Not applicable in this phase because there are no API calls used (download is done once via Kaggle).
    - If Kaggle API were used, authentication would require a private Kaggle API token. However, for this phase, one static file download was more efficient and safe. To ensure grading reproducibility, we avoid credential-based retrieval,.

1.1.4 Raw Data Storage

The dataset is stored in the project's root directory and is treated as read-only to preserve the data integrity. All transformations and cleaning steps are performed on the copied dataset. Only the trimmed dataset will be used for the purpose of this assignment. Trimmed data is created by reading the full input file by chunks of 200k rows, then sample each chunk to create a trimmed dataset of approximately 1.2GB.

```
testing_cars = data_retrieval()
```

```
Loading trimmed raw dataset...
Shape: (360005, 66)
None
```

	vin	back_legroom	bed	bed_height	bed_length	body_type	cabin	cit
0	JTMDFREV2HJ173436	37.2 in	NaN	NaN	NaN	SUV / Crossover	NaN	North Attlebor
1	5TDGZRBH1LS518759	41 in	NaN	NaN	NaN	SUV / Crossover	NaN	Wobur
2	3FA6P0HD3HR375170	38.3 in	NaN	NaN	NaN	Sedan	NaN	Lapee
3	2HGFC3B37HH355203	35.9 in	NaN	NaN	NaN	Coupe	NaN	Forest Hill
4	19XFC2F57HE215572	37.4 in	NaN	NaN	NaN	Sedan	NaN	Windsor Lock

5 rows × 66 columns

1.2 Wrangling/Cleaning

Phase 1 – Step 2

1.2.1 Initial audit

We perform an initial data audit:

- Missing value analysis
- Duplicate detection
- Outlier detection
- Data type validation
- Reproducible cleaning pipeline

We use the `quantDDA()` and `vizDDA()` functions from Lab 2 to standardize our audit process.

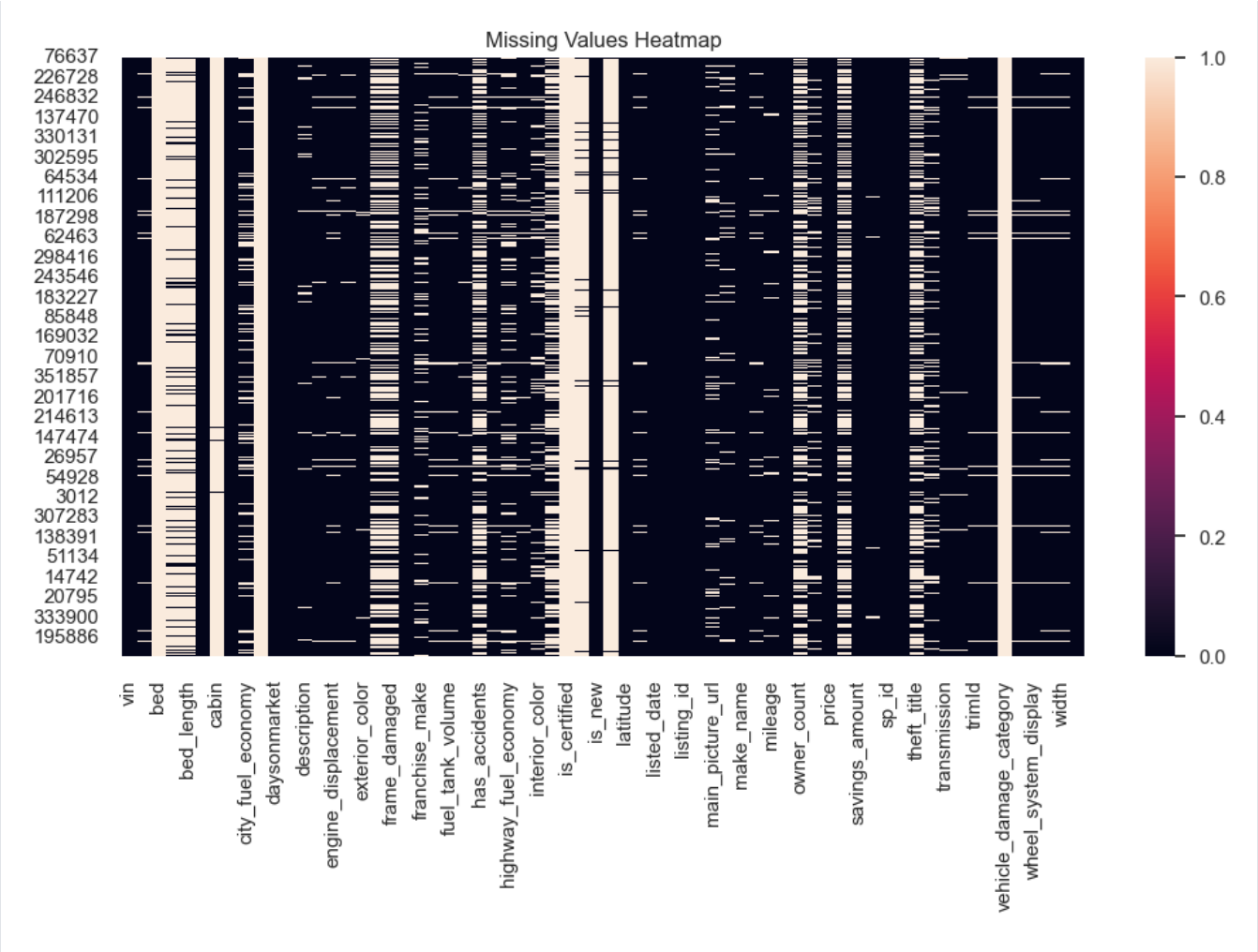
General Findings of the First Audit

```
display(quantDDA(testing_cars))
vizDDA(testing_cars)
```

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top /bottom 1%)	M
0	vin	360005	360005	360004	0	NaN	NaN	[1FT7W2BT8GEC
1	back_legroom	360005	340854	200	19151	NaN	NaN	[3
2	bed	360005	2354	3	357651	NaN	NaN	[
3	bed_height	360005	51657	1	308348	NaN	NaN	
4	bed_length	360005	51657	78	308348	NaN	NaN	[6
...	...	...	...	...	...	...	...	
61	wheel_system	360005	342438	5	17567	NaN	NaN	[
62	wheel_system_display	360005	342438	5	17567	NaN	NaN	[Front-Wheel
63	wheelbase	360005	340854	452	19151	NaN	NaN	[10
64	width	360005	340854	274	19151	NaN	NaN	[7
65	year	360005	360005	87	0	32469.0	2816.0	

66 rows × 17 columns

Visualization Grid (Univariate on Diagonal, Bivariate Off-Diagonal)



1.2.2: Reproducible Pipeline

For the cleaning pipeline, we first manually remove columns. Then, we apply a function to clean the dataset

A cleaning Report is provided for an easier view of what was performed.

Cleaning of the Data Set

Reproducible Data Cleaning Pipeline

The clean_cars() function implements a structured and reproducible data cleaning pipeline. Compared to the manual removal process described earlier, this function performs automated transformations based on measurable data properties (e.g., missingness rate, data type detection, regex pattern matching). This function ensures that cleaning steps are systematic, transparent, and reproducible.

Defensive Copy of the Dataset

A copy of the dataframe is created to prevent accidental modification of the original dataset. This preserves the raw data as read-only.

Dropping Columns with Extremely High Missingness

This step computes the proportion of missing values per column, removes columns where the missing rate exceeds a user-defined threshold, and stores removed columns in a cleaning report. We do this because columns with excessive missing values add noise, increase computational cost, and provide unreliable signal for modeling.

Removing Duplicate Rows

This step removes fully duplicated rows, because duplicates can bias statistical results, artificially inflate dataset size, and distort frequency distributions.

The number of removed rows is tracked in the report for transparency.

Handling Missing Values (Numeric)

For numeric columns, missing values are imputed using a missing indicator column and the median. This preserves information about missingness via a flag, and median imputation is robust to outliers.

Handling Missing Values (Categorical)

For categorical columns, missing values are replaced with a label "Missing". This preserves row count, avoids dropping potentially useful data, and allows models to treat the missingness as a category.

String Unification and Measurement Normalization

We standardize string formatting by converting to string, removing the whitespace and detecting measurements columns. For example, we checked if a column contained a pattern such as "20 in", "15.5 in"...

If the pattern was detected, we extracted only the numeric portion, and converted to float. We also renamed to column to reflect the units.

Removal of outliers

The function to remove outliers will only be implemented at the end of the EDA and data cleaning. This is to prevent functions like quantDDA from doing analysis on data that has outliers removed.

The process of removing outliers relies on simple algorithms like the one found in quantDDA, but also requires a human understanding of the data columns. For example, the quantDDA function might identify 0 as an outlier for the "mileage" column, but it is very reasonable for a new car to have not driven.

For the outlier removal function a list will be created for columns that require different observations. Just below is the list in markdown format with reasoning for special treatment.

- maximum_seating: Between 2 and 15 seats is reasonable for small cars and vans
- mileage: Cars with 0 mileage are possible if they are brand new. Only upper outliers will be removed
- year: Only exclude the lower end of outliers. Cars made the year the data was collected are not outliers.

Cleaning Report and Summary of Cleaned Dataset

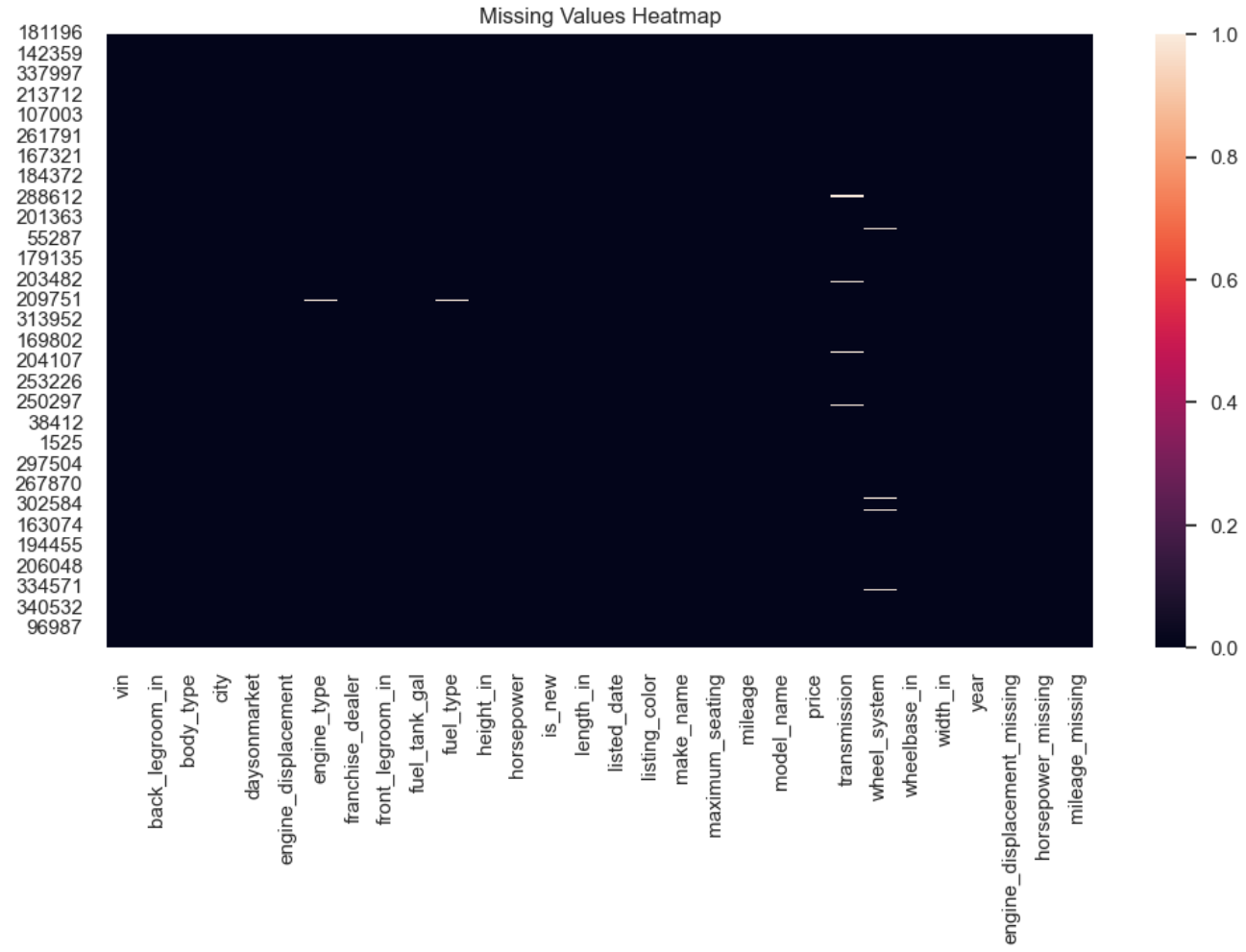
```
cars_clean, cleaning_report = data_cleaning(testing_cars, 0.1)
cars_clean.to_csv("clean_cars.csv", index=False)
print(cleaning_report)
display(cars_clean.head(5))
display(quantDDA(cars_clean))
display(vizDDA(cars_clean))
```

```
{'dropped_cols_high_missing': ['bed', 'bed_height', 'bed_length', 'cabin',
'city_fuel_economy', 'combine_fuel_economy', 'fleet', 'frame_damaged', 'franchise_make',
'has_accidents', 'highway_fuel_economy', 'interior_color', 'isCab', 'is_certified',
'is_cpo', 'is_oemcpo', 'main_picture_url', 'owner_count', 'salvage', 'theft_title',
'torque', 'vehicle_damage_category'], 'rows_removed_duplicates': 1,
'numeric_imputed_with_flags': ['engine_displacement', 'horsepower', 'mileage'],
'categorical_filled': []}
```

	vin	back_legroom_in	body_type	city	daysonmarket	engine_displacemen
0	JTMDFREV2HJ173436	37.2	SUV / Crossover	North Attleboro	28	2500.0
1	5TDGZRBH1LS518759	41.0	SUV / Crossover	Woburn	46	3500.0
2	3FA6P0HD3HR375170	38.3	Sedan	Lapeer	78	2000.0
4	19XFC2F57HE215572	37.4	Sedan	Windsor Locks	7	2000.0
6	5TDJZRFH9JS529988	38.4	SUV / Crossover	Schenectady	7	3500.0

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top /bottom 1%)	
0	vin	197762	197762	197762	0	NaN	NaN	[0N0RDER
1	back_legroom_in	197762	197762	114	0	1933.0	2765.0	
2	body_type	197762	197761	9	1	NaN	NaN	
3	city	197762	197762	3856	0	NaN	NaN	
4	daysonmarket	197762	197762	183	0	11610.0	1781.0	
5	engine_displacement	197762	197762	33	0	3259.0	2833.0	
6	engine_type	197762	196619	19	1143	NaN	NaN	
7	franchise_dealer	197762	197762	2	0	NaN	NaN	
8	front_legroom_in	197762	197762	59	0	4705.0	2571.0	
9	fuel_tank_gal	197762	197762	109	0	2359.0	2513.0	
10	fuel_type	197762	196619	4	1143	NaN	NaN	
11	height_in	197762	197762	200	0	0.0	3931.0	
12	horsepower	197762	197762	202	0	232.0	2992.0	
13	is_new	197762	197762	2	0	NaN	NaN	
14	length_in	197762	197762	336	0	2887.0	3552.0	
15	listed_date	197762	197762	187	0	NaN	NaN	
16	listing_color	197762	197762	15	0	NaN	NaN	
17	make_name	197762	197762	34	0	NaN	NaN	
18	maximum_seating	197762	197762	6	0	40588.0	1665.0	
19	mileage	197762	197762	60132	0	6323.0	1978.0	
20	model_name	197762	197762	394	0	NaN	NaN	
21	price	197762	197762	34452	0	2651.0	3791.0	
22	transmission	197762	194268	4	3494	NaN	NaN	
23	wheel_system	197762	196427	5	1335	NaN	NaN	
24	wheelbase_in	197762	197762	155	0	82.0	3606.0	
25	width_in	197762	197762	177	0	0.0	3034.0	
26	year	197762	197762	9	0	0.0	0.0	
27	engine_displacement_missing	197762	197762	2	0	NaN	NaN	
28	horsepower_missing	197762	197762	2	0	NaN	NaN	
29	mileage_missing	197762	197762	2	0	NaN	NaN	

```
Image too large (768336 bytes), skipped
```




```
None
```

1.3 Exploratory Data Analysis (EDA)

Phase 1 - Step 3

We perform exploratory data analysis on the cleaned dataset to understand distributions, relationships between variables, and to formulate our research questions.

1.3.1 Summary Statistics

We compute summary statistics for numerical and categorical variables to characterize the central tendency, spread, and composition of the data.

1.3.2 Univariate Visualizations

We visualize the distribution of key variables individually: price, body type, year, and mileage.

1.3.3 Bivariate Visualizations

We examine relationships between pairs of variables, particularly how price relates to mileage, body type, and year.

1.3.4 Correlation Analysis

We compute and visualize the correlation matrix for numerical features to identify linear relationships that may inform modeling.

1.3.5 EDA Synthesis: Comparative Analysis

We synthesize findings from 3.1–3.4 to identify consistent patterns and bridge to our research questions.

1.3.6 Research Questions

SUPERVISED LEARNING — Research Question 1

Can we predict the selling price of a used vehicle given its specifications, usage history, and dealership characteristics?

- **Y (target variable):** `price` — the listed or selling price of the vehicle (continuous, in dollars).
- **X (input features):**
  - **Specifications:** `year`, `mileage`, `engine_displacement`, `horsepower`, `body_type`, `engine_type`, `wheel_system`, `transmission`, `fuel_type`, `back_legroom_in`, `front_legroom_in`, `height_in`, `width_in`, `wheelbase_in`
  - **Usage / condition:** `mileage` (odometer reading), `daysonmarket`, `is_new`
  - **Dealership:** `franchise_dealer`, `city`

This is a **regression** task: we aim to estimate the continuous price value from the feature vector X. A baseline model (e.g., linear regression) will predict Y from X.

UNSUPERVISED LEARNING — Research Question 2

Can we discover natural clusters of used vehicles (e.g., budget, mid-range, luxury) based on their specifications and market characteristics?

- **X (input features used for clustering):**
  - **Specifications:** `engine_displacement`, `horsepower`, `body_type`, `year`, `wheel_system`, `fuel_type`
  - **Market / economic:** `price`, `mileage`, `daysonmarket`
  - **Size / comfort:** `back_legroom_in`, `front_legroom_in`, `width_in`, `height_in`
- **Y (output):** Cluster labels (e.g., 0, 1, 2) — **discovered, not given**. Each label corresponds to a group such as budget (lower price, smaller engine, higher mileage), mid-range, or luxury (higher price, larger engine, lower mileage). No ground-truth labels exist; the model learns the groupings from X.

This is a **clustering** task (e.g., K-means): we aim to partition vehicles into meaningful groups based on similarity in the feature space.

```
eda_results = run_eda(cars_clean)
```

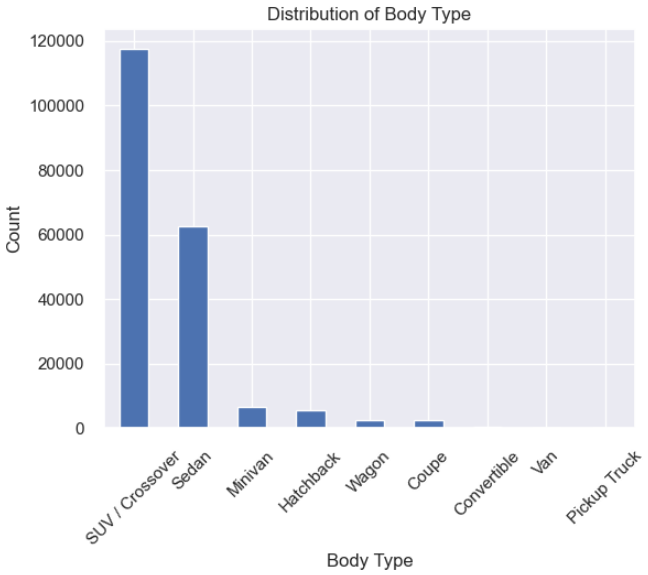
```
<IPython.core.display.Markdown object>
<IPython.core.display.Markdown object>

Top body types:
```

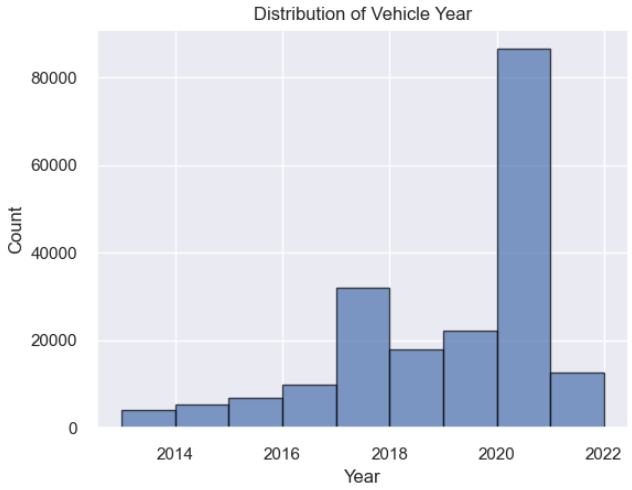
body_type	Count
SUV / Crossover	117716
Sedan	62473
Minivan	6719
Hatchback	5458
Wagon	2562
Coupe	2399
Convertible	311
Van	113
Pickup Truck	10

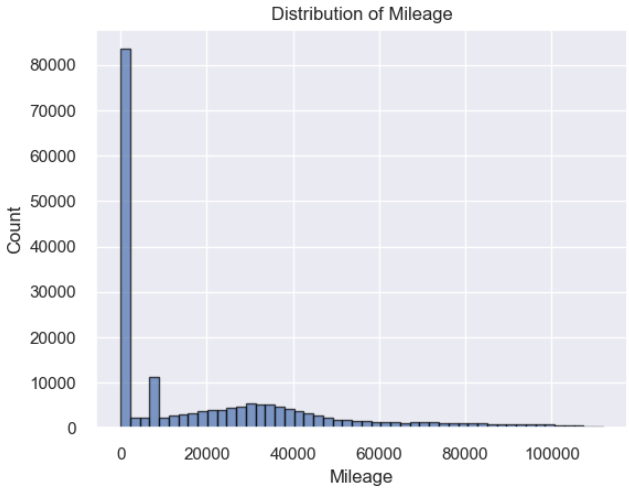
```
Top makes:
```

make_name	Count
Ford	20519
Chevrolet	20082
Honda	19773
Toyota	19459
Nissan	18299
Jeep	13212
Hyundai	10954
Kia	10278
Dodge	7493
Volkswagen	6905

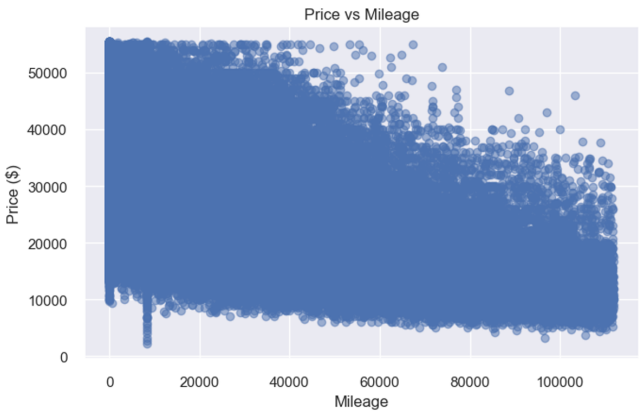
```
<IPython.core.display.Markdown object>
```

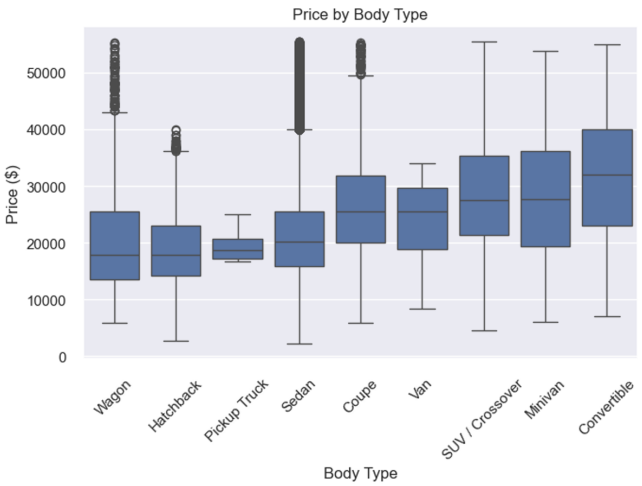


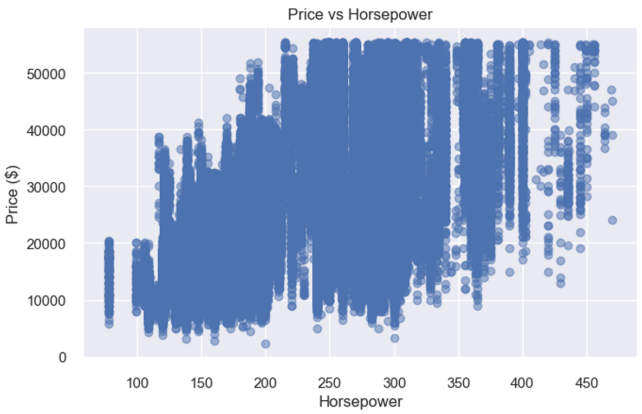




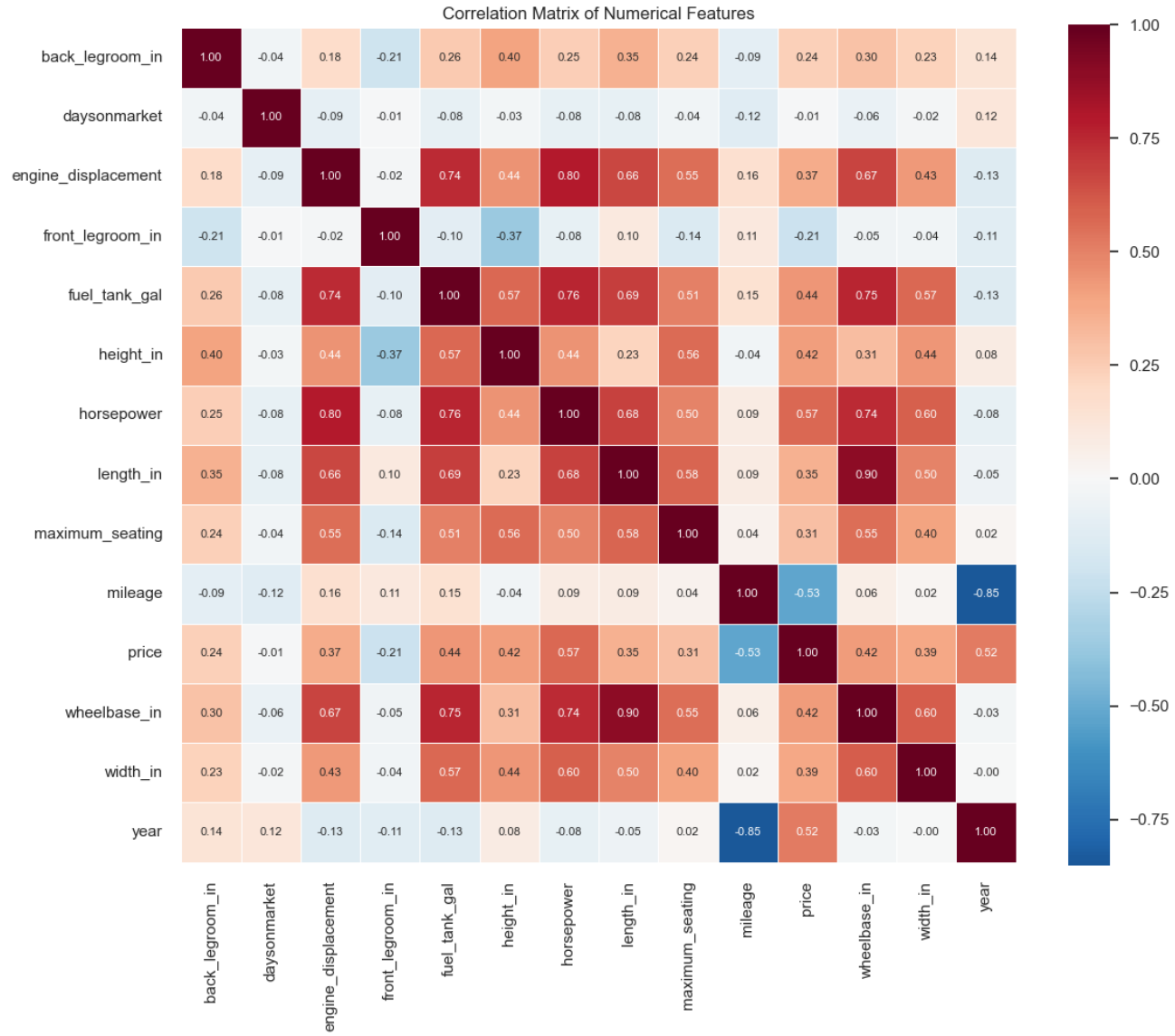
<IPython.core.display.Markdown object>







<IPython.core.display.Markdown object>



Correlation with price (sorted by absolute value):

	Correlation with Price
horsepower	0.569676
mileage	-0.525328
year	0.515867
fuel_tank_gal	0.436917
wheelbase_in	0.424333
height_in	0.416533
width_in	0.389586
engine_displacement	0.366142
length_in	0.350518
maximum_seating	0.305945
back_legroom_in	0.237431
front_legroom_in	-0.207565
daysonmarket	-0.005837

Conclusion (1.3.1): The summary statistics and tables above show that price, mileage, and year vary widely across the dataset. Body types such as SUV/Crossover and Sedan dominate, while certain makes (e.g., Honda, Ford, Chevrolet) appear most frequently. These patterns establish a baseline for understanding the market composition and guide which variables may be most predictive for modeling.

Conclusion (1.3.2): The univariate plots reveal that price and mileage are right-skewed, with most vehicles concentrated at lower price points and moderate mileage. Body type and make distributions show clear imbalance (SUVs and common brands dominate). Year is concentrated in recent years, suggesting a market bias toward newer listings. These distributions motivate log transforms or outlier handling for modeling.

Conclusion (1.3.3): The bivariate plots show that price tends to decrease with mileage and increase with year, as expected. Body type and horsepower create distinct price segments—luxury and performance vehicles command higher prices. These relationships support using mileage, year, body type, and horsepower as key features for the price prediction model.

Conclusion (1.3.4): The correlation heatmap highlights strong linear relationships: price correlates positively with year and horsepower, and negatively with mileage. Feature pairs such as (length, wheelbase) or (engine displacement, horsepower) show high collinearity, which may warrant feature selection or dimensionality reduction to avoid multicollinearity in linear models.

1.3.5 EDA Synthesis

Comparative Analysis Across Parts 1.3.1–1.3.4

This section synthesizes findings from the summary statistics (1.3.1), univariate distributions (1.3.2), bivariate relationships (1.3.3), and correlation analysis (1.3.4) to identify consistent patterns and inform our research questions.

Analysis	Key Insight	Link to Other Parts
1.3.1 Summary stats	Central tendency and spread of price, mileage, year; dominance of certain body types and makes	Quantifies what 1.3.2 visualizes; establishes baseline for comparison
1.3.2 Univariate	Right-skewed price; skewed mileage; year concentrated in recent years; body type imbalance	Explains outliers and concentration that appear in 1.3.3 bivariate plots
1.3.3 Bivariate	Price # with mileage; price # with year; body type and horsepower show distinct price segments	Confirms linear (mileage, year) vs categorical (body type) effects seen in 1.3.4 correlations
1.3.4 Correlation	Strongest linear associations with price (e.g., year, mileage, horsepower, engine)	Validates bivariate patterns; guides feature selection for regression and clustering

Variables that emerge across analyses as most informative for modeling:

- **Price** — target variable; right-skewed, so log-transform may help for regression.
- **Mileage** — strong negative relationship with price; consistent in bivariate and correlation.
- **Year** — positive association with price; supports depreciation and vintage effects.
- **Horsepower / engine** — correlated with price; differentiates vehicle tiers.
- **Body type** — categorical; drives price segments (SUV vs sedan vs truck) but not captured by Pearson correlation.

These findings support **Research Question 1** (price prediction) by identifying candidate features, and **Research Question 2** (clustering) by suggesting that price, mileage, year, horsepower, and body type are good discriminators for natural vehicle segments.

1.4. Baseline Model

Phase 1 - Step 4

In this section, we implement a baseline supervised learning model to predict the selling price of used cars. This is a **regression problem**, since the target variable (`price`) is continuous.

The objective of this baseline model is to establish a reference point for evaluating more advanced modeling approaches. With a simple model, we can see how well basic relationships between vehicle features (e.g., year, mileage, brand, fuel type, transmission) and price explain the variation in the dataset.

1.4.1 Simple Model

A simple Linear Regression is used as the baseline.

Setting up of the Model

We decide to exclude the Vehicle Identification Number (VIN) from the modeling features because the VIN is a unique identifier and does not provide predictive value. However, it is retained in the cleaned dataset as it represents a valid identifier.

Moreover, we also drop the columns that are more than 95% unique, because they are most likely identifiers. For the VIN, they were for sure all identifiers, but to not manually list all the others, we just drop the columns that have 95% of being identifiers.

1.4.2 Train / Val / Test Split (70% / 15% / 15%)

The dataset is divided into three subsets:

- **Training set (70%)**: Used to fit the model.
- **Validation set (15%)**: Used to evaluate model performance during development.
- **Test set (15%)**: Reserved for final evaluation to assess generalization.

  This split ensures that model evaluation is performed on unseen data and reduces the risk of overfitting. Also, given the large size of the dataset, this split ensures statistically stable performance estimates.

1.4.3 Evaluation Metrics

Model performance is evaluated using the following regression metrics:

- **MAE (Mean Absolute Error)**: Average absolute difference between predicted and actual prices (in $).Robusttooutliers.<spanclass=″latex−placeholder″data−id=″0″><∕span>−∗∗RMSE(F$ $_{\text{k}}-foldCVsplitsdatainto_{\text{k}}folds,trainson_{\text{k-1}}andevaluatesontheheld-outfold,repeated_{\text{k}}$ $times.Reportedmetricisthemeanacrossfolds.<aid=″sec-144″><∕a>\#\#1.4.4Performance$

  45,000 we see that our model starts to fail at price prediction. After this point, most predictions underestimate the costs with very few predictions matching or even exceeding the actual price.

The model's predictions form a slight arc shape where the lower and upper end values contain more predictions below actual price and the middle predicts values over the actual price. So the model is better at predicting prices when it has more data (middle end cars) and struggles a bit at predicting lower priced cars, but struggles a lot when predicting cars that are more expensive.

Residual Analysis

A residuals plot (residuals vs. predicted price) helps check model assumptions. Ideally, points should be randomly scattered around zero with constant spread. A funnel shape suggests heteroscedasticity (variance changes with price); curvature suggests non-linear relationships.

Error Distribution

A histogram of prediction errors shows how errors are distributed. A symmetric, roughly normal distribution centered near zero indicates well-calibrated predictions; skew or heavy tails suggest systematic bias or outliers.

Results for the baseline model:

Train :

- MAE ( ) : N/A $-$ $RMSE($): N/A
- R2: 0.7306

Validation :

- MAE ( ) : $4,127.42$ $-$ $RMSE($): 5,392.85
- R2: 0.7313

Test :

- MAE ( ) : $4,193.91$ $-$ $RMSE($): 5,482.86
- R2: 0.7250

Conclusion: The baseline linear regression achieves an R^2 of ~73% across train, validation, and test sets, meaning it explains about 73% of the variation in used car prices. The similar R^2 across splits indicates no substantial overfitting. An MAE of ~$4,200 implies that, on average, predictions are off by roughly this amount in either direction—which may be acceptable for mid-range vehicles but less so for cheaper or luxury cars. This variation in prediction error is consistent with the skewed distribution of vehicle prices observed in the EDA, where higher-priced and lower-priced vehicles tend to exhibit greater variability. The ~27% unexplained variance suggests room for improvement through feature engineering, non-linear models, or handling outliers. Overall, the model provides a reasonable baseline for price prediction.

1.4.5 Reproducible Baseline Modeling Pipeline

In this section, we refactored the baseline linear regression model from Phase 1 into a modular and reproducible pipeline.

The objective is to replicate the steps described originally in Phase 1.4 — data preparation, model training, evaluation, and visualization — while organizing the workflow into reusable functions.

Pipeline Structure

- Data preparation
- Removal of target variable and non-predictive columns (ex. VIN)
- Elimination of high-cardinality categorical variables (above 95% uniqueness)
- Selection of numeric features only
- Handling of missing values
- Train / validation test split (70% / 15% / 15%)

Model training

A Linear regression model is fitted using the training data; this is a baseline model to compare with the advanced methods

Prediction visualization

A scatter plot of actual vs predicted prices is generated; a diagonal reference line represents perfect predictions. This allows for visual assessment of the model accuracy

Model evaluation

Performance is evaluated using

- R^2 (coefficient of determination)
- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)
- MAPE (Mean Absolute Percentage Error)
- MedAE (Median Absolute Error)

Metric are computed on validation and test sets

Diagnostic Analysis

- Residual plot (residuals vs predicted values): Used to assess model assumptions (linearity, homoscedasticity)
- Error distribution histogram: Used to analyse the spread and symmetry of prediction errors

Feature interpretation

A coefficient table is generated from the linear model; this identifies which features have the strongest impact on price

Reproducibility

This pipeline ensures reproducibility by:

- avoiding hardcoded dataset
- parameterizing all key modeling decisions such as split ratios, thresholds, and seed
- Structuring the workflow into independent and reusable functions

By doing this design, the baseline model can be re-run, modified or extended in Phase 2 and Phase 3

The original descriptions, insights and evaluation are in the Markdown cell above, and the feature importance, limitations, recommendations and summary can be found at the end of section 1.4

```
baseline_results = run_baseline_linear_regression_pipeline(
    df=cars_clean,
    target_col="price",
    columns_to_drop=["vin", "body_type", "franchise_dealer"],
    train_size=0.70,
    val_size=0.15,
    test_size=0.15,
    random_state=42,
    high_unique_threshold=0.95
)
```

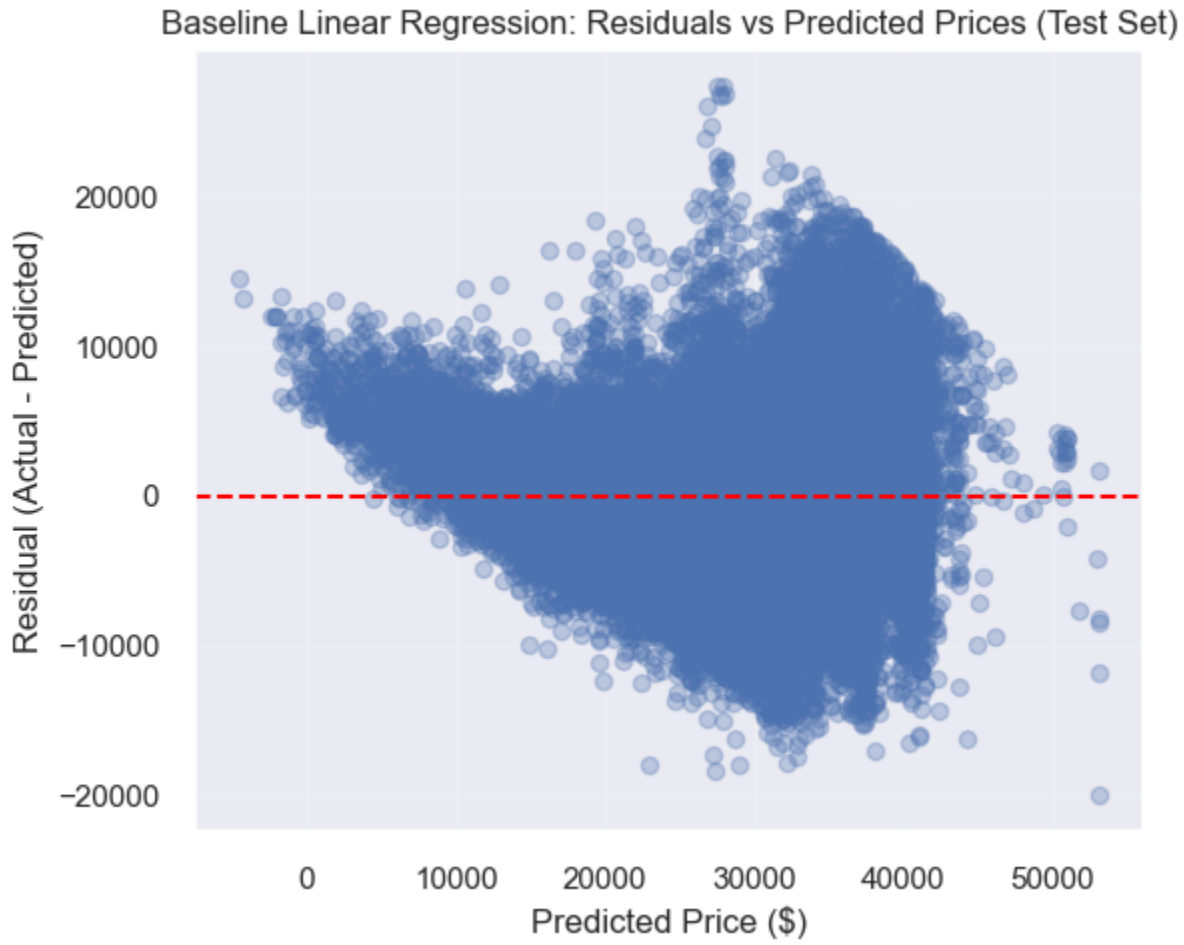
```
<IPython.core.display.Markdown object>
```

<IPython.core.display.Markdown object>

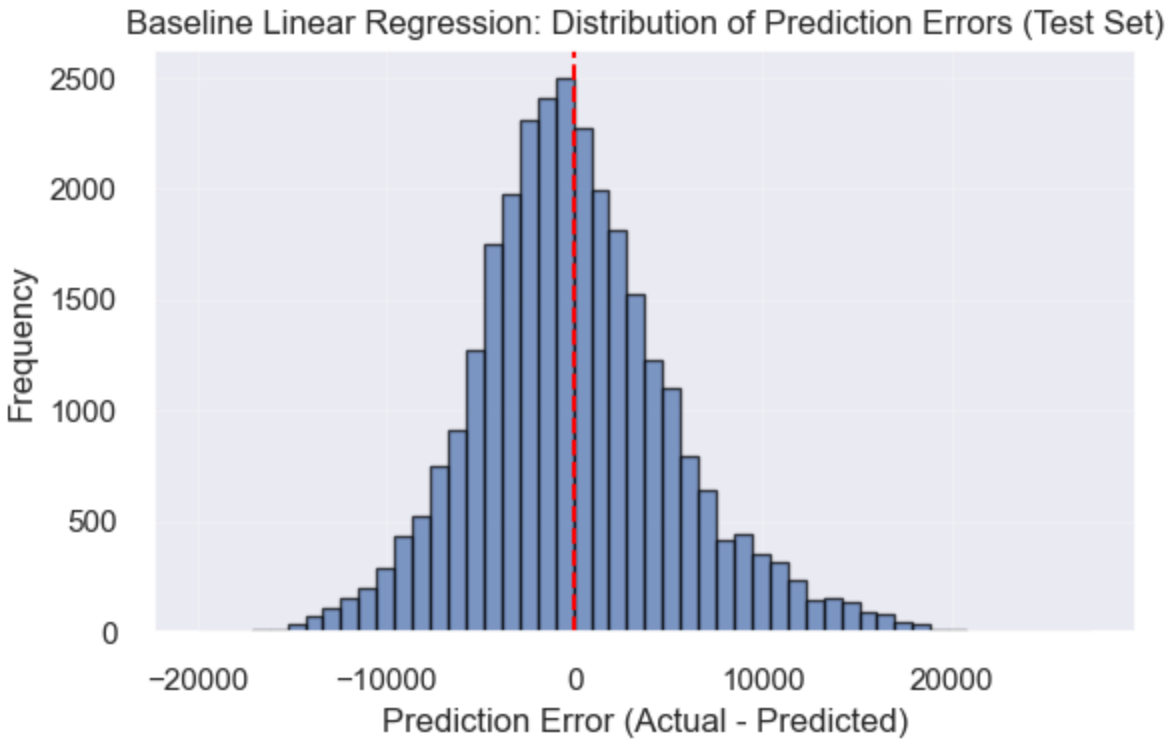
Baseline Linear Regression Performance (Train / Validation / Test)

	Set	R2	MAE ($)	RMSE ($)	MAPE (%)	MedAE
0	Train	0.7077	4,151.40	5,431.27	17.31	3,239.73
1	Validation	0.7084	4,123.20	5,409.40	17.23	3,216.77
2	Test	0.7046	4,129.79	5,395.45	17.25	3,242.38

<IPython.core.display.Markdown object>



<IPython.core.display.Markdown object>



Coefficient Table

```
coefficients = baseline_results["evaluation_results"]["coefficients"]

display(Markdown("### Most Important Positive Features"))
display(
    coefficients.head(10).style.format({
        "Coefficient": "{:.2f}"
    }).set_caption("Top Positive Features")
)

display(Markdown("### Most Important Negative Features"))
display(
    coefficients.sort_values(by="Coefficient", ascending=True).head(10).style.format({
        "Coefficient": "{:.2f}"
    }).set_caption("Top Negative Features")
)
```

<IPython.core.display.Markdown object>

Top Positive Features

	Feature	Coefficient
12	year	1348.72
4	fuel_tank_gal	371.22
5	height_in	271.08
6	horsepower	102.75
7	length_in	43.51
9	mileage	-0.13
2	engine_displacement	-2.25
1	daysonmarket	-8.11
11	width_in	-29.73
10	wheelbase_in	-62.90

<IPython.core.display.Markdown object>

Top Negative Features

	Feature	Coefficient
8	maximum_seating	-403.12
3	front_legroom_in	-387.70
0	back_legroom_in	-210.08
10	wheelbase_in	-62.90
11	width_in	-29.73
1	daysonmarket	-8.11
2	engine_displacement	-2.25
9	mileage	-0.13
7	length_in	43.51
6	horsepower	102.75

Feature Interpretation

The coefficient table above shows which features drive price. Positive coefficients (e.g., year, fuel_tank_gal) are associated with higher prices; negative coefficients (e.g., maximum_seating, front_legroom_in) with lower prices. Features with the largest absolute coefficients have the strongest impact on predictions.

Limitations

- **Numeric features only**: Categorical variables (e.g., body type, brand) were excluded from this baseline, likely omitting important price signals.
- **Linear assumption**: Linear regression assumes linear relationships; real price–feature relationships may be non-linear.
- **Sparse extremes**: Lower- and higher-priced vehicles are underrepresented, leading to poorer predictions at the tails.
- **Data quality**: Missing values, outliers, and listing errors in the source data can affect model performance.

Recommendations for Improvement

- **Include categorical**: Use one-hot encoding for body type, brand, fuel type, and other categorical features.
- **Try non-linear models**: Random forest, gradient boosting (XGBoost, LightGBM), or neural networks may capture non-linear patterns.
- **Feature engineering**: Derive age (current year # year), price-per-mile, or interaction terms (e.g., mileage × year).
- **Handle outliers**: Cap extreme values, use robust regression, or filter unrealistic listings.

Summary

The baseline linear regression provides a reasonable starting point with R^2 ~73% and MAE ~ $4,200. It performs best for mid-range vehicles ($ 30k–$40k) and underestimates luxury prices. Residual and error analyses help diagnose where the model fails. Addressing the limitations above can improve accuracy for future iterations.

Phase 2

```
# ------------------------------
#
# PHASE 2 PIPELINE
#
# ------------------------------
```

```python
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import RandomizedSearchCV
from xgboost import XGBRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
from sklearn.feature_selection import SelectKBest, f_regression, RFE
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.inspection import PartialDependenceDisplay

# --------------------------------
# ----------------
# 2.1 ADVANCED SUPERVISED LEARNING
# ----------------

def advanced_model_setup(X):
    """
    Prepares the preprocessing pipeline for advanced supervised models.

    Separates numeric and categorical features, keeps numeric features unchanged,
    and applies one-hot encoding to categorical features.

    Parameters:
    - X: feature DataFrame

    Returns:
    - preprocessor: ColumnTransformer object
    - numeric_features: list of numeric feature names
    - categorical_features: list of categorical feature names
    """

    # Split features
    # Keep numbers as is; Encode text
    numeric_features = X.select_dtypes(include=["number"]).columns
    categorical_features = X.select_dtypes(include=["object", "string"]).columns

    # Preprocessing
    # ColumnTransformer applies different transformations to different columns
    # Numeric: Passthrough, do nothing
    # Categorical: Apply OneHotEncoder
    preprocessor = ColumnTransformer(
        transformers=[
            ("num", "passthrough", numeric_features),
            # Transforms categories into binary columns, Ignore any new category
            ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
        ]
    )
```

```
    return preprocessor, numeric_features, categorical_features

# ------------------------

def print_r2_scores(trained_model, X_train, y_train, X_val, y_val, X_test, y_test,
model_name):
    """
    Computes and prints R2 scores for train, validation, and test sets.

    Parameters:
    - trained_model: fitted regression model
    - X_train, X_val, X_test: feature datasets
    - y_train, y_val, y_test: target datasets
    - model_name: name of the model (for labeling)

    Returns:
    - dictionary containing train, validation, and test R2 scores
    """

    y_pred_train = trained_model.predict(X_train)
    y_pred_val = trained_model.predict(X_val)
    y_pred_test = trained_model.predict(X_test)

    train_r2 = r2_score(y_train, y_pred_train)
    val_r2 = r2_score(y_val, y_pred_val)
    test_r2 = r2_score(y_test, y_pred_test)

    print(f"{model_name} Train R2:", train_r2)
    print(f"{model_name} Validation R2:", val_r2)
    print(f"{model_name} Test R2:", test_r2)

    return {
        "train_r2": train_r2,
        "val_r2": val_r2,
        "test_r2": test_r2
    }

# ------------------------

def train_random_forest_model(preprocessor, X_train, y_train, random_state, n_iter, cv,
scoring, n_jobs):
    """
    Trains and tunes a Random Forest regression pipeline.

    Applies preprocessing and uses RandomizedSearchCV to tune hyperparameters.

    Parameters:
    - preprocessor: preprocessing pipeline (ColumnTransformer)
    - X_train: training feature matrix
    - y_train: training target vector
    - random_state: random seed
    - n_iter: number of parameter settings sampled
```

```
    - cv: number of cross-validation folds
    - scoring: evaluation metric
    - n_jobs: number of parallel jobs

    Returns:
    - randomForest_search: fitted RandomizedSearchCV object
    """

    randomForest_pipeline = Pipeline([
        ("preprocessing", preprocessor),
        ("model", RandomForestRegressor(random_state=random_state))
    ])

    param_grid_randomForest = {
        "model__n_estimators": [100, 200],
        "model__max_depth": [10, 20, None],
        "model__min_samples_split": [2, 5]
    }

    randomForest_search = RandomizedSearchCV(
        randomForest_pipeline,
        param_grid_randomForest,
        n_iter=n_iter,
        cv=cv,
        scoring=scoring,
        n_jobs=n_jobs,
        random_state=random_state
    )

    randomForest_search.fit(X_train, y_train)

    return randomForest_search

# ------------------------

def train_xgboost_model(preprocessor, X_train, y_train, random_state, n_iter, cv,
scoring, n_jobs):
    """
    Trains and tunes an XGBoost regression pipeline.

    Applies preprocessing and uses RandomizedSearchCV to tune hyperparameters.

    Parameters:
    - preprocessor: preprocessing pipeline (ColumnTransformer)
    - X_train: training feature matrix
    - y_train: training target vector
    - random_state: random seed
    - n_iter: number of parameter settings sampled
    - cv: number of cross-validation folds
    - scoring: evaluation metric
    - n_jobs: number of parallel jobs

    Returns:
```

```
    - xgb_search: fitted RandomizedSearchCV object
    """

    xgb_pipeline = Pipeline([
        ("preprocessing", preprocessor),
        ("model", XGBRegressor(random_state=random_state))
    ])

    param_grid_xgb = {
        "model__n_estimators": [100, 200],
        "model__max_depth": [3, 6],
        "model__learning_rate": [0.05, 0.1]
    }

    xgb_search = RandomizedSearchCV(
        xgb_pipeline,
        param_grid_xgb,
        n_iter=n_iter,
        cv=cv,
        scoring=scoring,
        n_jobs=n_jobs,
        random_state=random_state
    )

    xgb_search.fit(X_train, y_train)

    return xgb_search

# ------------------------

def display_model_results(trained_model, X_train, y_train, X_val, y_val, X_test, y_test,
model_name):
    """
    Displays visual and numerical evaluation for a trained regression model.

    Generates the actual vs predicted plot, prints R2 scores,
    and displays the residual plot.

    Parameters:
    - trained_model: fitted regression model
    - X_train, X_val, X_test: feature datasets
    - y_train, y_val, y_test: target datasets
    - model_name: name of the model (for labeling)

    Returns:
    - dictionary containing predicted values, residuals, and R2 scores
    """

    y_pred_test = plot_actual_vs_predicted(
        trained_model=trained_model,
        X_data=X_test,
        y_data=y_test,
        model_name=model_name,
```

```
        dataset_name="Test Set"
    )

    r2_results = print_r2_scores(
        trained_model=trained_model,
        X_train=X_train,
        y_train=y_train,
        X_val=X_val,
        y_val=y_val,
        X_test=X_test,
        y_test=y_test,
        model_name=model_name
    )

    residuals = plot_residuals(
        trained_model=trained_model,
        X_data=X_test,
        y_data=y_test,
        model_name=model_name,
        dataset_name="Test Set"
    )

    return {
        "y_pred_test": y_pred_test,
        "residuals": residuals,
        "r2_results": r2_results
    }

# ------------------------

def evaluate_model(model, X, y, name):
    """
    Evaluates a regression model on a dataset.

    Computes and prints MAE, RMSE, and R2.

    Parameters:
    - model: fitted regression model
    - X: feature matrix
    - y: target vector
    - name: name of the model (for labeling)

    Returns:
    - None
    """

    preds = model.predict(X)
    print(f"\n{name}")
    print("MAE:", mean_absolute_error(y, preds))
    print("RMSE:", np.sqrt(mean_squared_error(y, preds)))
    print("R2:", r2_score(y, preds))

# ------------------------
```

```python
def compare_models_regression_metrics(models_dict, X_test, y_test):
    """
    Compares multiple regression models using performance metrics.

    Computes MAE, RMSE, and R2 for each model on the test set.

    Parameters:
    - models_dict: dictionary of models {name: model}
    - X_test: test feature matrix
    - y_test: test target vector

    Returns:
    - results_df: sorted DataFrame of model performance
    """

    results = []

    for name, mdl in models_dict.items():

        preds = mdl.predict(X_test)

        mae = mean_absolute_error(y_test, preds)
        rmse = np.sqrt(mean_squared_error(y_test, preds))
        r2 = r2_score(y_test, preds)

        results.append({
            "Model": name,
            "MAE ($)": mae,
            "RMSE ($)": rmse,
            "R2": r2
        })

    results_df = pd.DataFrame(results).sort_values(by="R2", ascending=False)
    results_df.reset_index(drop=True, inplace=True)

    return results_df

# ------------------------

def display_model_regression_metrics_comparison(results_df):
    """
    Displays a formatted comparison table for multiple models.

    Uses styled output for readability in notebooks.

    Parameters:
    - results_df: DataFrame containing model performance metrics

    Returns:
    - None
    """
```

```
    display(Markdown("## Model Comparison (Test Set)"))

    display(
        results_df.style
        .format({
            "MAE ($)": "{:,.2f}",
            "RMSE ($)": "{:,.2f}",
            "R2": "{:.4f}"
        })
        .set_caption("Comparison of Models on Test Set")
    )

# ------------------------

def run_model_regression_metrics_comparison(models_dict, X_test, y_test):
    """
    Runs the full model comparison pipeline.

    Computes metrics and displays formatted comparison results.

    Parameters:
    - models_dict: dictionary of models {name: model}
    - X_test: test feature matrix
    - y_test: test target vector

    Returns:
    - results_df: DataFrame of model performance
    """

    results_df = compare_models_regression_metrics(models_dict, X_test, y_test)

    display_model_regression_metrics_comparison(results_df)

    return results_df

# ------------------------

def compare_models_actual_vs_pred(models_dict, X_test, y_test):
    """
    Compares multiple regression models using actual vs predicted plots.

    Creates a subplot for each model with a reference line for perfect predictions.

    Parameters:
    - models_dict: dictionary of models {name: model}
    - X_test: test feature matrix
    - y_test: test target vector

    Returns:
    - None (displays plots)
    """

    display(Markdown("## Model Comparison: Actual vs Predicted"))
```

```
    n_models = len(models_dict)
    plt.figure(figsize=(6 * n_models, 5))

    for i, (name, model) in enumerate(models_dict.items()):
        y_pred = model.predict(X_test)

        plt.subplot(1, n_models, i + 1)
        plt.scatter(y_test, y_pred, alpha=0.3)

        # Perfect prediction line
        plt.plot(
            [y_test.min(), y_test.max()],
            [y_test.min(), y_test.max()],
            color="red",
            linestyle="--"
        )

        plt.title(f"{name}")
        plt.xlabel("Actual Price ($)")
        plt.ylabel("Predicted Price ($)")
        plt.grid(alpha=0.2)

    plt.tight_layout()
    plt.show()

# ------------------------

def compare_models_residuals(models_dict, X_test, y_test):
    """
    Compares multiple regression models using residual plots.

    Creates a subplot for each model showing residuals vs predicted values.

    Parameters:
    - models_dict: dictionary of models {name: model}
    - X_test: test feature matrix
    - y_test: test target vector

    Returns:
    - None (displays plots)
    """

    display(Markdown("## Model Comparison: Residual Analysis"))

    n_models = len(models_dict)
    plt.figure(figsize=(6 * n_models, 5))

    for i, (name, model) in enumerate(models_dict.items()):
        y_pred = model.predict(X_test)
        residuals = y_test - y_pred

        plt.subplot(1, n_models, i + 1)
```

```
        plt.scatter(y_pred, residuals, alpha=0.3)

        plt.axhline(y=0, color="red", linestyle="--")

        plt.title(f"{name}")
        plt.xlabel("Predicted Price ($)")
        plt.ylabel("Residual (Actual - Predicted)")
        plt.grid(alpha=0.2)

    plt.tight_layout()
    plt.show()

import pandas as pd
import matplotlib.pyplot as plt

# ------------------------

def plot_feature_importance(search_obj, model_name, top_n):
    """
    Extract and plot feature importances from a pipeline inside a search object
    (e.g., RandomizedSearchCV).

    Parameters:
    - search_obj: fitted search object (randomForest_search, xgb_search, etc.)
    - model_name: string for plot title
    - top_n: number of top features to display

    Returns:
    - importance_df: DataFrame containing feature importances
    """

    # Extract trained pipeline
    best_pipeline = search_obj.best_estimator_

    # Extract model and feature names
    model = best_pipeline.named_steps["model"]
    feature_names = best_pipeline.named_steps["preprocessing"].get_feature_names_out()

    # Create importance dataframe
    importance_df = pd.DataFrame({
        "Feature": feature_names,
        "Importance": model.feature_importances_
    }).sort_values(by="Importance", ascending=False)

    # Display top features
    display(Markdown(f"## Top {top_n} features for {model_name}"))
    display(importance_df.head(top_n))

    # Plot
    plt.figure(figsize=(10, 6))
    plt.barh(
        importance_df["Feature"].head(top_n)[::-1],
        importance_df["Importance"].head(top_n)[::-1]
```

```
    )
    plt.title(f"Top {top_n} Feature Importances - {model_name}")
    plt.xlabel("Importance")
    plt.tight_layout()
    plt.show()

    return importance_df
# ------------------------

# ----------------
# 2.2 FEATURE ENGINEERING
# ----------------

def create_new_features(df):
    # create a copy of clean cars for comparison later
    feature_cars = df.copy(deep=True)

    # These columns are dropped to make human analysis easier
    feature_cars.drop(columns=['engine_displacement_missing', 'horsepower_missing',
'mileage_missing'], inplace=True)

    # Create an age column. Data was collected in September of 2020
    feature_cars['age'] = 2020 - feature_cars['year']
    feature_cars['age'] = feature_cars['age'].clip(lower=1)  # minimum age of 1 year

    #-----
    #Polynomial Features
    #-----

    feature_cars['horsepower_squared'] = np.square(feature_cars['horsepower'])
    feature_cars['fuel_tank_gal_squared'] = np.square(feature_cars['fuel_tank_gal'])
    feature_cars['maximum_seating_squared'] = np.square(feature_cars['maximum_seating'])
    feature_cars['height_in_squared'] = np.square(feature_cars['height_in'])
    # mileage doesn't correlate with a lot of other columns, but it is very important for
the price and year columns
    feature_cars['mileage_squared'] = np.square(feature_cars['mileage'])

    #-----
    #Interaction Features
    #-----

    feature_cars['annual_mileage'] = feature_cars['mileage'] / feature_cars['age'].replace
(0, float('nan'))
    # This is not an accurate representation of the actual volume of the vehicle!
    feature_cars['fake_volume_in'] = feature_cars['height_in'] * feature_cars['length_in'
] * feature_cars['width_in']
    feature_cars['horsepower/volume'] = feature_cars['horsepower'] / feature_cars[
'fake_volume_in']
    feature_cars['power_per_displacement'] = feature_cars['horsepower'] / feature_cars[
'engine_displacement'].replace(0, float('nan'))
    feature_cars['volume_per_seat'] = feature_cars['fake_volume_in'] / feature_cars[
'maximum_seating'].replace(0, float('nan'))
    feature_cars['fuel_per_volume'] = feature_cars['fuel_tank_gal'] / feature_cars[
```

```
'fake_volume_in']

    #-----
    #Domain-specific Features
    #-----

    feature_cars['is_high_mileage'] = (feature_cars['mileage'] > 100000).astype(int)
    feature_cars['is_automatic'] = (feature_cars['transmission'].str.contains('A', na=
False)).astype(int)
    # cars recently posted on the market will have a higher asking price. Only after some
time will the asking price decrease
    feature_cars['is_fresh_listing'] = (feature_cars['daysonmarket'] < 14).astype(int)
    feature_cars['is_awd_4wd'] = feature_cars['wheel_system'].isin(['AWD', '4WD']).astype
(int)
    feature_cars['is_electric_hybrid'] = feature_cars['fuel_type'].str.contains(
'Electric|Hybrid', na=False).astype(int)

    #-----
    #Text/Time Features
    #-----

    # convert the date to pandas datetime
    feature_cars['listed_date'] = pd.to_datetime(feature_cars['listed_date'])

    # Season — car sales are heavily seasonal. AWD and 4WD might be more sought after in
winter
    def get_season(month):
        if month in [12, 1, 2]:
            return 0  # Winter
        elif month in [3, 4, 5]:
            return 1  # Spring
        elif month in [6, 7, 8]:
            return 2  # Summer
        else:
            return 3  # Fall

    feature_cars['listing_season'] = feature_cars['listed_date'].dt.month.apply
(get_season)

    # V6 and V8 engines are more powerful. typically found in sports cars or trucks
    feature_cars['is_v6'] = feature_cars['engine_type'].str.contains('V6', case=False, na=
False).astype(int)
    feature_cars['is_v8'] = feature_cars['engine_type'].str.contains('V8', case=False, na=
False).astype(int)
    feature_cars['cylinder_count'] = feature_cars['engine_type'].str.extract(r'(\d+)').
astype(float)

    return feature_cars

#------------

def feature_selection(df):
    feature_cars = df.copy(deep=True)
```

```
    # Separate your target variable
    X = feature_cars.drop(columns=['price', 'vin', 'body_type', 'franchise_dealer',
'listed_date', 'city', 'model_name'])
    y = feature_cars['price']

    # Drop columns that are almost entirely unique (most likely identifiers)
    high_unique_cols = [
        col for col in X.select_dtypes(include=["object", "string"]).columns
        if X[col].nunique() / len(X) > 0.95
    ]
    X = X.drop(columns=high_unique_cols, errors="ignore")

    # Baseline simplification: keep only numeric features (avoids string errors)
    X = X.select_dtypes(include=["number"]).fillna(0)


    print('\n\n\n\n\nCorrelation Matrix\n\n\n\n\n')
    # Correlation Matrix
    corr = X.corrwith(y).abs().sort_values(ascending=False)
    print("Top features by correlation with price:")
    print(corr.head(40))

    # Plot the correlation matrix using a bar chart
    plt.figure(figsize=(10, 8))
    corr.head(20).plot(kind='bar')
    plt.title('Feature Correlation with Price')
    plt.ylabel('Correlation')
    plt.tight_layout()
    plt.show()

    # SelectKBest
    selector = SelectKBest(score_func=f_regression, k=20)
    selector.fit(X, y)

    # Get selected feature names
    filter_selected = X.columns[selector.get_support()].tolist()
    print("SelectKBest selected features:", filter_selected)

    print('\n\n\n\n\nWrapper method\n\n\n\n\n')
    # Wrapper method

    # create a sample of the data (the algorithm takes too long on full data)
    X_sample, _, y_sample, _ = train_test_split(X, y, train_size=50000, random_state=42)

    # RFE with Random Forest — select top features
    rfe_model = RandomForestRegressor(n_estimators=50, random_state=42, n_jobs=-1)
    rfe = RFE(estimator=rfe_model, n_features_to_select=20, step=5, verbose=1)
    rfe.fit(X, y)

    # Get selected feature names
    wrapper_selected = X.columns[rfe.support_].tolist()
```

```
print("RFE selected features:", wrapper_selected)

# Show ranking
rfe_ranking = pd.Series(rfe.ranking_, index=X.columns).sort_values()
print(rfe_ranking.head(20))


print('\n\n\n\n\nEmbedded Method\n\n\n\n\n')
# Lasso Regression
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

lasso = Lasso(alpha=0.01, random_state=42)
lasso.fit(X_scaled, y)

lasso_selected = X.columns[lasso.coef_ != 0].tolist()
print("Lasso selected features:", lasso_selected)

# Random Forest Feature Importance
rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X, y)

importance_df = pd.DataFrame({
    'feature': X.columns,
    'importance': rf.feature_importances_
}).sort_values('importance', ascending=False)

# Plot top 20
plt.figure(figsize=(10, 8))
sns.barplot(data=importance_df.head(20), x='importance', y='feature')
plt.title('Random Forest Feature Importance')
plt.tight_layout()
plt.show()

embedded_selected = importance_df.head(20)['feature'].tolist()


print('\n\n\n\n\nSelected Features\n\n\n\n\n')
# Find features selected by all 3 methods
all_selected = set(filter_selected) & set(wrapper_selected) & set(embedded_selected)
print("Features selected by ALL methods:", all_selected)

# Features selected by at least 2 methods
from collections import Counter
all_features = filter_selected + wrapper_selected + embedded_selected
feature_counts = Counter(all_features)
majority_selected = [f for f, count in feature_counts.items() if count >= 2]
print("Features selected by at least 2 methods:", majority_selected)

# Use majority vote as your final feature set
X_final = X[majority_selected]

# these columns will be used in feature_cars2
```

```
    final_columns = majority_selected + ['price']

    # feature_cars2 is the feature_cars dataset with only the top 20 columns selected
from the script above
    feature_cars2 = feature_cars[final_columns]
    display(quantDDA(feature_cars2))

    return feature_cars2, X_final, y

# ------------------------

# ----------------
# 2.3  UNSUPERVISED LEARNING
# ----------------

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

def run_kmeans_pca_clustering(
    df,
    silhouette_fit_rows=50000,
    silhouette_score_rows=8000,
    plot_rows=15000,
    k_values=range(2, 9),
    random_state=42
):
    """
    Runs K-Means clustering with silhouette-based k selection and PCA visualization.

    Method:
    - Uses a sample of the dataset to choose the best k with silhouette score
    - Fits the final K-Means model on the full dataset using the chosen k
    - Uses a sample for PCA visualization
    - Excludes price from clustering input, then compares price across clusters

    Parameters:
    - df: input DataFrame (e.g., feature_cars or clean_cars)
    - silhouette_fit_rows: number of rows used to fit temporary K-Means models for k
selection
    - silhouette_score_rows: number of rows used to compute silhouette score
    - plot_rows: number of rows used in the PCA scatter plot
    - k_values: range or list of candidate k values
    - random_state: random seed for reproducibility

    Returns:
    - dictionary containing clustering inputs, labels, PCA projection, silhouette scores,
      chosen k, and price summary by cluster
    """

    rng = np.random.RandomState(random_state)
```

```
    # ----------------------------
    # 1. Prepare FULL dataset for final clustering
    # ----------------------------
    df_full = df.copy()

    price_series_full = df_full["price"].copy()

    drop_for_cluster = {"price", "vin", "listed_date"}
    X_cluster_full = df_full.drop(
        columns=[c for c in drop_for_cluster if c in df_full.columns],
        errors="ignore"
    )

    # Keep numeric features only
    X_cluster_full = X_cluster_full.select_dtypes(include=["number"]).fillna(0)

    # Standardize on FULL dataset
    scaler = StandardScaler()
    X_scaled_full = scaler.fit_transform(X_cluster_full)

    # ----------------------------
    # 2. Sample dataset for choosing k
    # ----------------------------
    fit_n = min(silhouette_fit_rows, len(X_cluster_full))
    fit_idx = rng.choice(len(X_cluster_full), size=fit_n, replace=False)

    X_scaled_fit = X_scaled_full[fit_idx]
    price_fit = price_series_full.iloc[fit_idx].reset_index(drop=True)

    # Separate smaller sample for silhouette score if needed
    sil_n = min(silhouette_score_rows, len(X_scaled_fit))
    sil_idx_local = rng.choice(len(X_scaled_fit), size=sil_n, replace=False)

    silhouette_by_k = {}

    for k in k_values:
        km_temp = KMeans(n_clusters=k, random_state=random_state, n_init=10)
        labels_fit = km_temp.fit_predict(X_scaled_fit)

        silhouette_by_k[k] = silhouette_score(
            X_scaled_fit[sil_idx_local],
            labels_fit[sil_idx_local]
        )

    best_k = max(silhouette_by_k, key=silhouette_by_k.get)

    print("Silhouette (sample) by k:", {k: round(v, 4) for k, v in silhouette_by_k.
items()})
    print("Chosen k:", best_k)

    # ----------------------------
    # 3. Fit FINAL K-Means on FULL dataset
    # ----------------------------
```

```
kmeans = KMeans(n_clusters=best_k, random_state=random_state, n_init=10)
cluster_id_full = kmeans.fit_predict(X_scaled_full)

# ----------------------------
# 4. PCA on FULL standardized data, plot only a sample
# ----------------------------
pca = PCA(n_components=2, random_state=random_state)
X_pca_full = pca.fit_transform(X_scaled_full)

plot_n = min(plot_rows, len(X_scaled_full))
plot_idx = rng.choice(len(X_scaled_full), size=plot_n, replace=False)

# ----------------------------
# 5. Price summary by cluster (FULL dataset)
# ----------------------------
summary_df = pd.DataFrame({
    "price": price_series_full.values,
    "cluster": cluster_id_full
})

price_by_cluster = (
    summary_df.groupby("cluster")["price"]
    .agg(["count", "median", "mean"])
    .sort_values("median")
)

# ----------------------------
# 6. Plots
# ----------------------------
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sc = axes[0].scatter(
    X_pca_full[plot_idx, 0],
    X_pca_full[plot_idx, 1],
    c=cluster_id_full[plot_idx],
    cmap="tab10",
    alpha=0.35,
    s=8
)
axes[0].set_title("PCA of Listings (K-Means Clusters)")
axes[0].set_xlabel(f"PC1 ({pca.explained_variance_ratio_[0] * 100:.1f}% var)")
axes[0].set_ylabel(f"PC2 ({pca.explained_variance_ratio_[1] * 100:.1f}% var)")
plt.colorbar(sc, ax=axes[0], label="Cluster")

summary_df.boxplot(column="price", by="cluster", ax=axes[1])
axes[1].set_title("Listed Price by Cluster")
axes[1].set_xlabel("Cluster")

plt.suptitle("")
plt.tight_layout()
plt.show()

print("\nPrice by cluster:")
```

```
    display(price_by_cluster)

    return {
        "X_cluster_full": X_cluster_full,
        "X_scaled_full": X_scaled_full,
        "cluster_id_full": cluster_id_full,
        "X_pca_full": X_pca_full,
        "kmeans_model": kmeans,
        "pca_model": pca,
        "best_k": best_k,
        "silhouette_by_k": silhouette_by_k,
        "price_summary": price_by_cluster,
        "summary_df": summary_df,
        "fit_sample_size": fit_n,
        "silhouette_score_size": sil_n,
        "plot_sample_size": plot_n
    }


# ------------------------

# ----------------
# 2.4  INTERPRETATION
# ----------------

def partial_dependence_plots(X_final, y):
    y_log = np.log1p(y)  # log(price + 1)

    rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
    rf.fit(X_final, y_log)

    # Then replot PDPs
    PartialDependenceDisplay.from_estimator(rf, X_final, features=['year', 'mileage',
'horsepower', 'age'])
    plt.show()
```

2.1 Advanced Supervised learning

Using Advanced Supervised Learning Models allow us to capture more complex relationship within the data. In addition, more advanced supervised learning models are implemented and tuned to improve predictive performance and enable a systematic comparison with the baseline approach. The models were built with the same base setup to properly compare them.

Advanced Setup

```
X_train, X_val, X_test, y_train, y_val, y_test = base_model_setup(
    df=cars_clean,
    target_col="price",
    columns_to_drop=["vin"],
    train_size=0.70,
    val_size=0.15,
    test_size=0.15,
    random_state=42,
    high_unique_threshold=0.95
)
preprocessor, numeric_features, categorical_features = advanced_model_setup(X_train)
```

2.1.1 Model 1 - Random Forest

We implement a Random Forest model as it handles non-linearity, it works well on tabular data, and it doesn't require scaling.

Random Forest is an ensemble method, and it combines multiple decision trees to capture non-linear relationship, as well as reduce overfitting.

Hyperparameter tuning is performed using RandomizedSearchCV to optimize key parameters, such as:

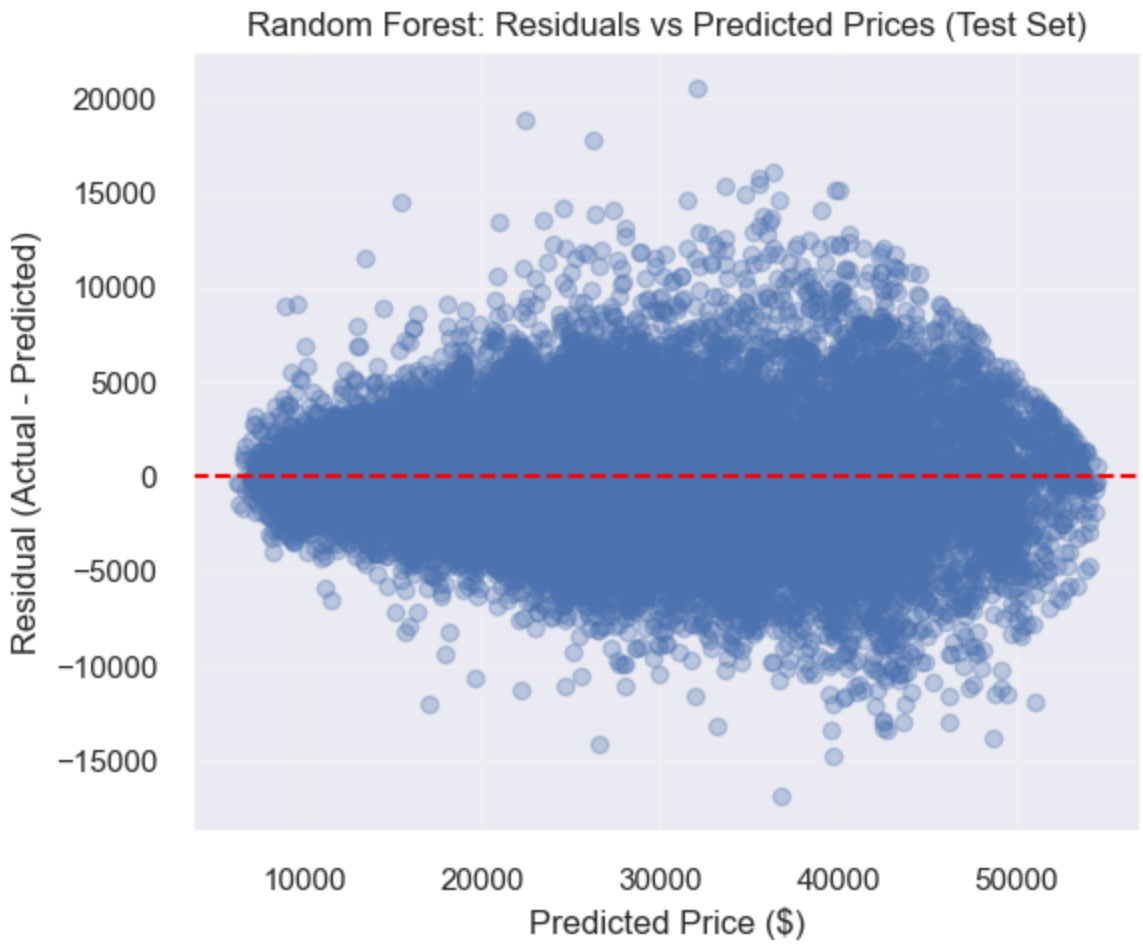
- the number of trees (n_estimators): determines how many decision trees are built in the ensemble
- tree depth (max_depth): controls how complex each individual tree can become
- minimum samples required for splitting (min_samples_split): prevents overfitting by avoiding overly specific splits

Each tree is trained independently, and the final prediction is the aggregation of the prediction of all trees

```
# n_jobs can be set to -1 for a powerful computer, or 2 for better speed
# currently set to 1 due to computing power
randomForest_search = train_random_forest_model(
    preprocessor=preprocessor,
    X_train=X_train,
    y_train=y_train,
    random_state=42,
    n_iter=5,
    cv=3,
    scoring="r2",
    n_jobs=1
)

rf_results = display_model_results(
    trained_model=randomForest_search.best_estimator_,
    X_train=X_train,
    y_train=y_train,
    X_val=X_val,
    y_val=y_val,
    X_test=X_test,
    y_test=y_test,
    model_name="Random Forest"
)
```

```
<IPython.core.display.Markdown object>
```


```
Random Forest Train R2: 0.9657578308921295
Random Forest Validation R2: 0.919491695838722
Random Forest Test R2: 0.9175273128904924
<IPython.core.display.Markdown object>
```



2.1.2 Model 2 - XGBoost

We implement XGBoost as a second advanced model. It is a gradient boosting algorthm that builds trees sequentially, which improves errors from previous iterations. It has strong performance for tabular data.

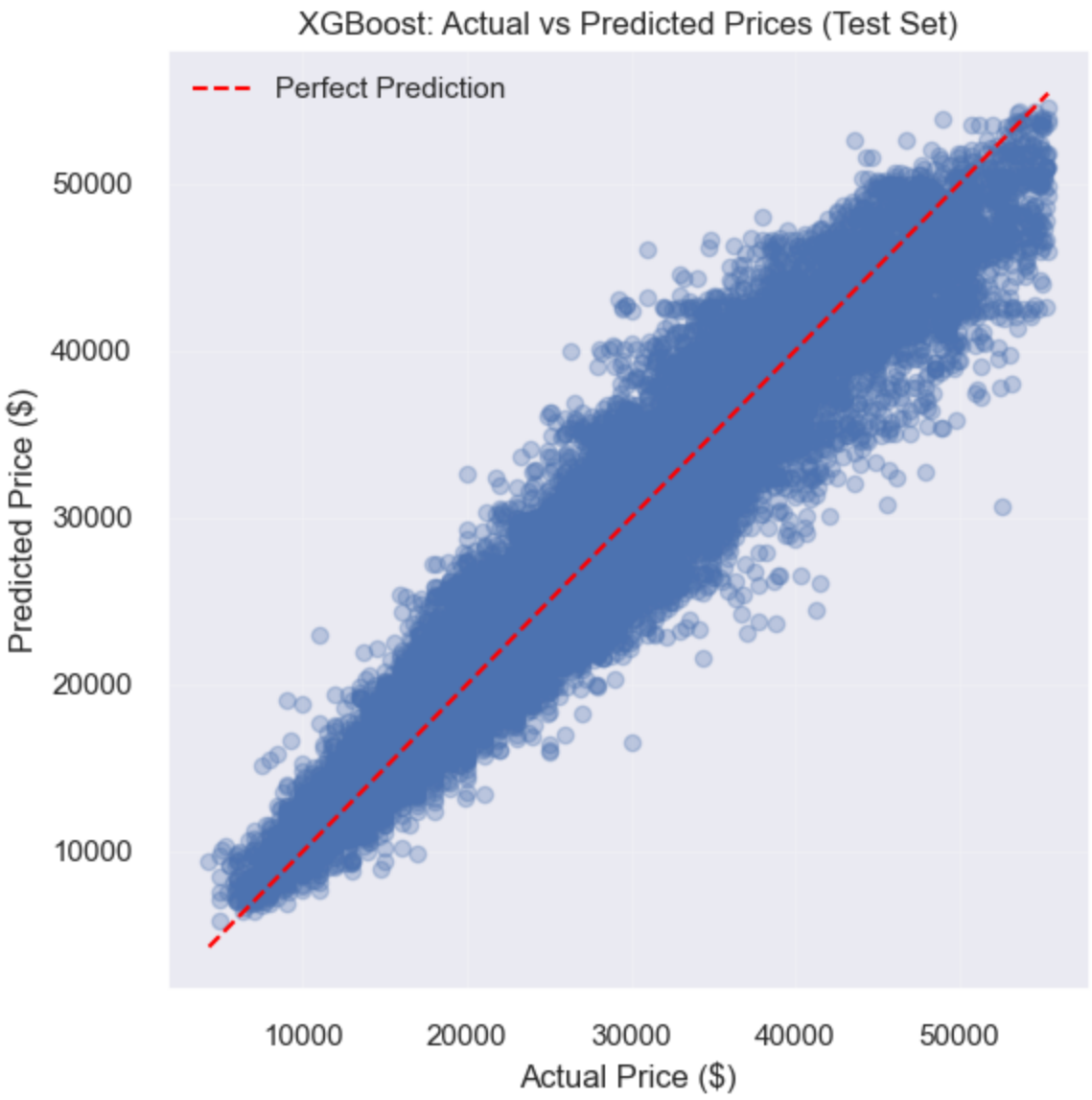
Hyperparameter tuning is applied to control the model's complexity and learning rate.

- n_estimators (100, 200) to control the number of trees built during boosting; Increasing the value allows the model to learn more complex patterns --> may increase the risk of overfitting.
- max_depth (3, 6) to determine the maximum depth of each tree; Smaller depth (3) promotes simpler models with better generalization; larger depth (6) allows the model to capture more complex relationships.
- learning_rate (0.05, 0.1) to control how much each tree contributes to the final prediction; Lower learning rate (0.05) leads to more gradual learning and often better generalization; higher rate (0.1) speeds up training but may reduce stability.

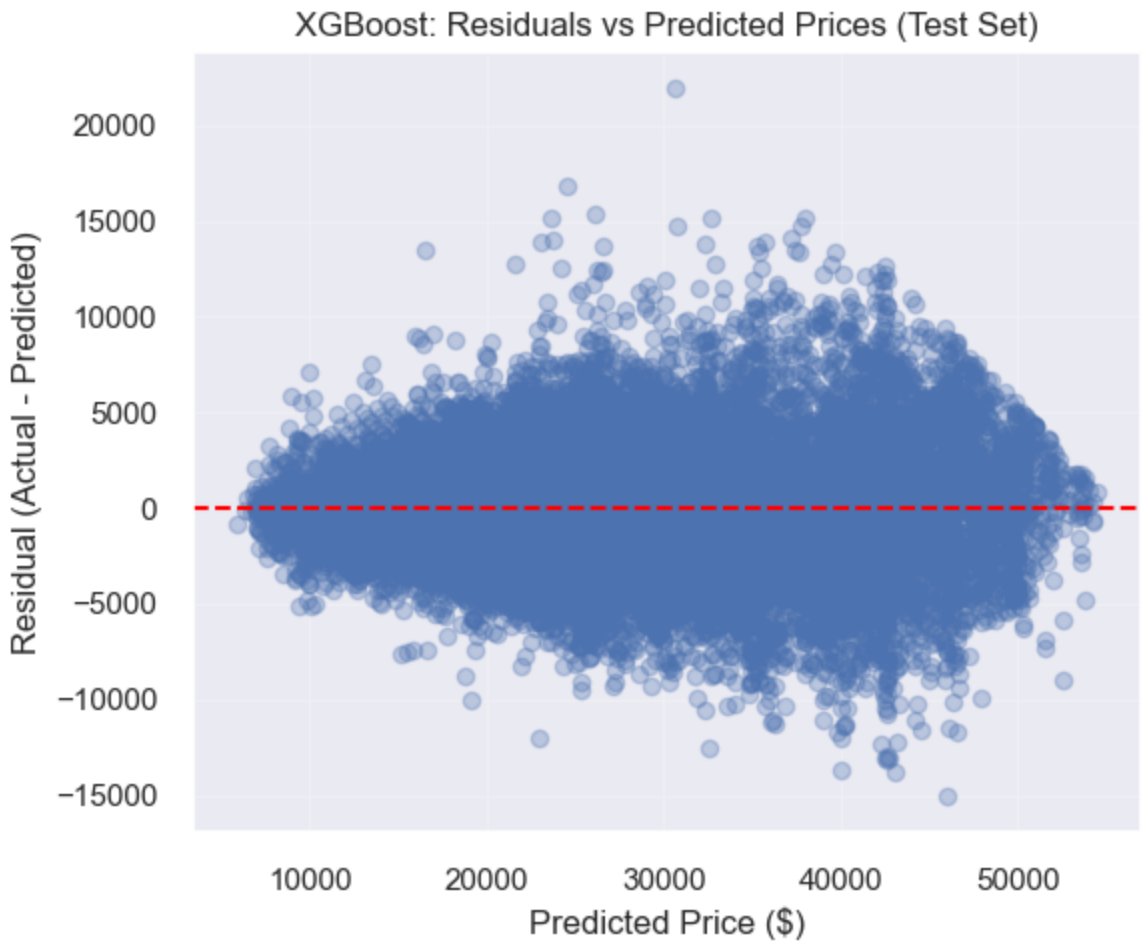
A grid search with cross-validation is then used to evaluate all combinations of these parameters and select the configuration that maximizes predictive performance (R^2).

```
xgb_search = train_xgboost_model(
    preprocessor=preprocessor,
    X_train=X_train,
    y_train=y_train,
    random_state=42,
    n_iter=5,
    cv=3,
    scoring="r2",
    n_jobs=-1
)

xgb_results = display_model_results(
    trained_model=xgb_search.best_estimator_,
    X_train=X_train,
    y_train=y_train,
    X_val=X_val,
    y_val=y_val,
    X_test=X_test,
    y_test=y_test,
    model_name="XGBoost"
)
```

```
<IPython.core.display.Markdown object>
```




```
XGBoost Train R2: 0.9232051700547705
XGBoost Validation R2: 0.9197871690360072
XGBoost Test R2: 0.9177027097933041
<IPython.core.display.Markdown object>
```



2.1.3 Evaluation Metrics

To evaluate model performance, we use multiple regression metrics:

- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)
- R^2 (Coefficient of Determination)

These metrics allow us to assess prediction accuracy and compare models.

2.1.4 Systematic Model Comparison

We compare the baseline model with the advanced models using the test set. This allows us to evaluate which model performs best and generalizes well to unseen data.

To compare the supervised models systematically, we evaluate each model on the same test set using regression metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). These metrics are appropriate because the target variable, price, is continuous.

In the table, the results are sorted by descending R^2. This is the proportion of variance explained by the model, and the closer it is to 1, the better. This also means that the best-performing model appears first, and allows a clear comparison of model performance.

Classification metrics such as ROC curves, AUC, and confusion matrices are not used here because they are designed for classification problems with discrete class labels, whereas this project focuses on regression.

```
baseline_model = linear_regression_training(X_train, y_train)

models_dict = {
    "Linear Regression": baseline_model,
    "Random Forest": randomForest_search.best_estimator_,
    "XGBoost": xgb_search.best_estimator_
}
```

```
results_df = run_model_regression_metrics_comparison(
    models_dict=models_dict,
    X_test=X_test,
    y_test=y_test
)
```

```
<IPython.core.display.Markdown object>
```

Comparison of Models on Test Set

	Model	MAE ($)	RMSE ($)	R2
0	XGBoost	2,131.27	2,847.84	0.9177
1	Random Forest	2,092.89	2,850.87	0.9175
2	Linear Regression	4,129.79	5,395.45	0.7046

2.1.5 Visual Comparison of Model Predictions

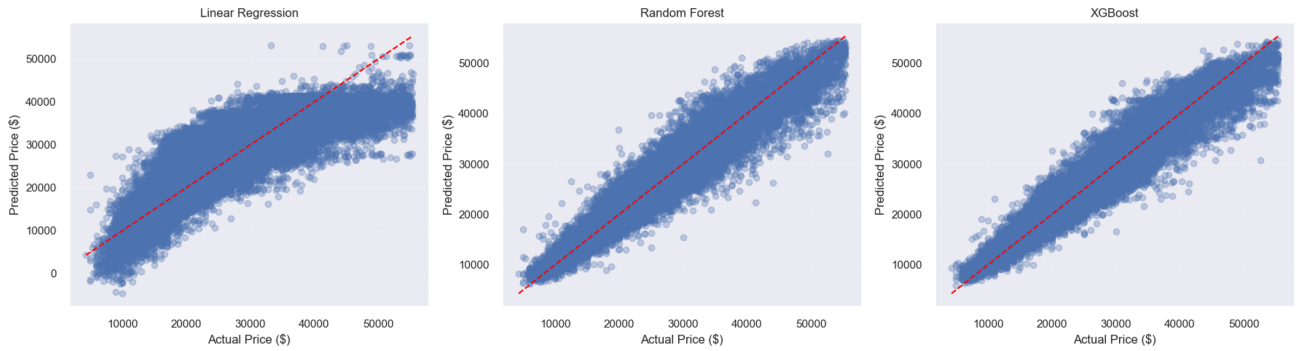
To complement the numerical evaluation metrics, we made visualization of the predictions of all models using actual vs predicted plots. Each subplot represents a model (Random Forest, XGBoost, Linear Regression).

Models that produce predictions closer to the diagonal line demonstrate better performance.

Actual vs Predicted

```
compare_models_actual_vs_pred(models_dict, X_test, y_test)
```

```
<IPython.core.display.Markdown object>
```



Residuals comparison

A residual plot shows: Residual = Actual - Predicted

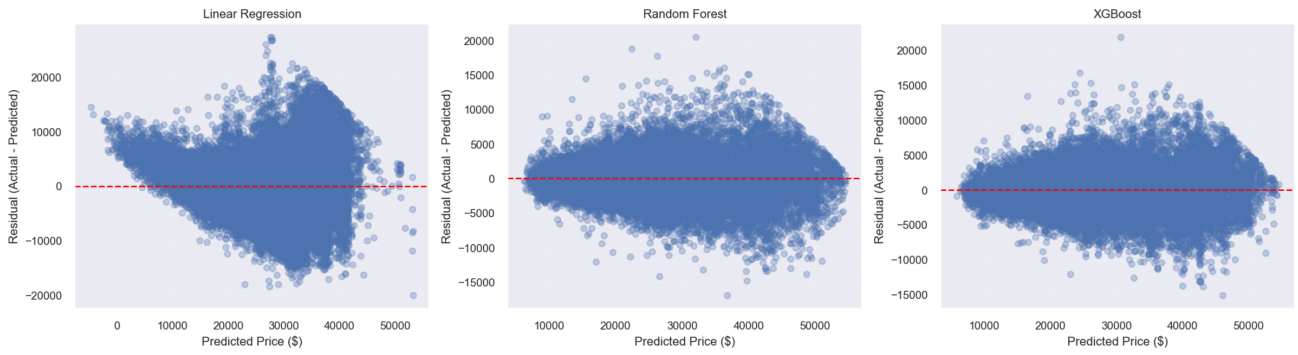
Therefore, this plot shows :

- X-axis: predicted price
- Y-axis: error

Ideally, points would be scattered around 0, with a constant spread and no pattern.

```
compare_models_residuals(models_dict, X_test, y_test)
```

```
<IPython.core.display.Markdown object>
```



2.1.6 Best Model Justification

We notice these results for the model comparison:

- **Random Forest**:R^2 of 0.9185, *RMSE* of 2833 and *MAE* of 2082
- **XGBoost**: R^2 of 0.9088, *RMSE* of 2996 and *MAE* of 2252
- **Base Model (Linear Regression)**: R^2 of 0.7046, *RMSE* of 5395 and *MAE* of 4129

Based on those results, the Random Forest model achieves the best overall performance compared to the XGBoost and the Base Linear Regression. It has the highest value of R^2 and the lowest prediction errors (MAE and RMSE). This indicates that it explains the largest proportion of variance in vehicle prices and produces the most accurate predictions (with the lowest prediction errors).

Still, both Random Forest and XGBoost outperform the baseline regression model, which confirms that the relationship between vehicle characteristics and price is **non-linear**.

Although XGBoost performs similarly to Random Forest, random Forest slightly outperforms it and provides predictions that are more stable, because of its approach, which reduces variance by averaging multiple decision trees.

We find the same results using the visualizations; The 3 plots confirm that Random Forest and XGBoost produce more accurate and consistent predictions compared to the baseline linear regression model.

For the residuals plots, we notice that:

- the Linear Regression residuals plot has a funnel shape, and the residuals are not centered evenly. Higher prices also show a higher spread; Errors increases as price increases. Again, linear regression is not appropriate for this problem.
- the Random Forest residuals plot has a tighter spread and the residuals are more centered around 0. However, there is still a slight funnel shape. Errors are smaller, and more stable, but there is still some more variance at high prices and a slight structure.
- the XGBoost residuals plot is similar to Random Forest

For all models, there is a small shape; low prices have small errors, and higher prices have larger errors. The highest prices have smaller errors; high mid-range has the largest spread. We can conclude that there is heteroscedasticity; Expensive cars are harder to predict and more variable, and their prices are influenced by some more hidden factors.

However, we can still say that both Random Forest and XGBoost perform better than the Linear Regression, even on the Residuals Level.

In conclusion, we select **Random Forest** as the best model for our task.

In addition, due to the high cardinality of categorical variables and the resulting computational cost of one-hot encoding, we initially tested models including both numerical and categorical features. However, this significantly increased training time—particularly for Random Forest—without substantial improvement in predictive performance; including categorical features led to very long training times for Random Forest, to the point where full hyperparameter tuning became computationally impractical within the project constraints.

A direct comparison between models with and without categorical features showed only negligible performance differences (#R^2 # 0.001 and RMSE differences of less than $15), indicating that the added complexity of one-hot encoding high-cardinality categorical variables does not provide meaningful predictive benefit in this context. Tree-based models such as Random Forest and XGBoost are able to capture non-linear relationships and interactions between variables, allowing key numerical features like mileage, horsepower, and year—identified as strongly correlated with price during EDA—to explain most of the variance

Feature importance analysis (as shown later in the report) further supports this observation: when categorical variables are included, importance is distributed across many one-hot encoded features (e.g., individual brands and models), resulting in fragmented and less interpretable importance scores. In contrast, numerical-only models highlight a small set of dominant predictors—such as horsepower, year, and mileage—providing a clearer and more stable explanation of price variation.

As of Phase 3:

- With the pipeline, XGBoost now consistently has nearly the same value of R2 as Random Forest (0.9177 for XGBoost vs 0.9175 for Random Forest).
- Random Forest has the lower MAE, meaning that predictions are still closer on average.
- XGBoost has a slightly better RMSE, meaning it handles large errors a bit better
- In the prediction comparison plots, Random Forest seems to be more aligned with the diagonal, with a more consistent spread
- In the residual comparison plots, Random Forest is slightly more symmetric around 0

We think those differences are due to the fact that the model setup was slightly different using the pipeline. The cleaned dataset seems also slightly different due to outlier removal.

Still, based on the test-set comparison, both Random Forest and XGBoost consistently outperforms the baseline Linear Regression Model. As of Phase 3, XGBoost achieves slightly higher R2 and RMSE, suggesting a small advantage in explaining variance and controlling larger errors. However, Random Forest has the lower MAE, suggesting on average its predictions are slightly closer to true prices. The two models are very similar in overall performance, but each has a small advantage depending on the metric considered. Both visual comparisons also support this conclusion, as the actual-vs-predicted plot show that Random Forest and XGBoost follow the diagonal more closely than the Linear Regression. For the residual plots, Linear regression shows a wider spread of errors and unsymmetrical shape compared to Random Forest and XGBoost.

Overall, both advanced models are appropriate for the task. If we look only at the comparison table, XGBoost can be justified as top model according to the R2 descending sorting. If we consider the lower MAE and the slightly more symmetrical plots, Random Forest can be justified as the top model. We conclude that depending on if we want better variance and control over large error, XGBoost is best, and if we want predictions to be closer to the true prices, then Random Forest is better.

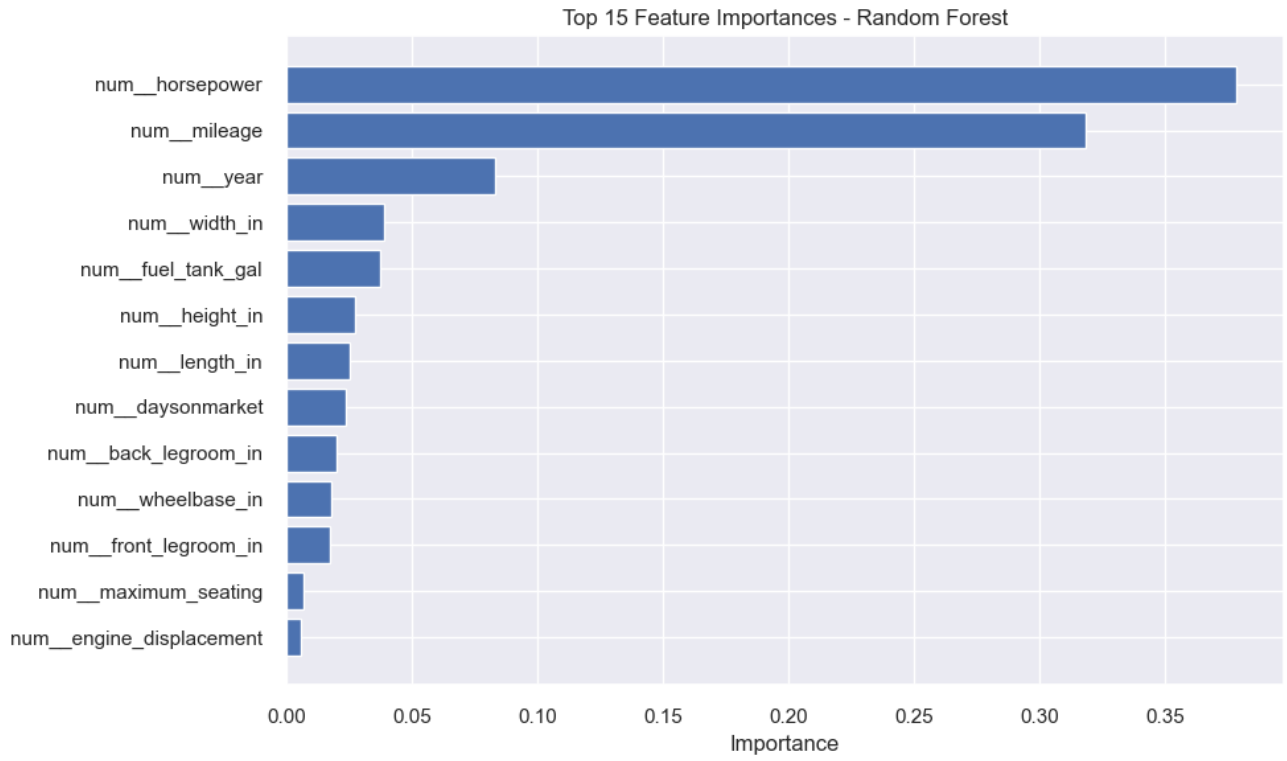
2.1.7 Feature Importance

To interpret the advanced supervised models, we analyze feature importance for both Random Forest and XGBoost to understand which variables contribute most to price prediction. This helps interpret the model and identify key drivers of vehicle price; This helps identify which vehicle specifications, usage-related variables, and dealership characteristics contribute most strongly to price prediction.
Comparing feature importance across the two models also helps assess whether similar predictors consistently drive model performance.

```
rf_importance_df = plot_feature_importance(
    randomForest_search,
    model_name="Random Forest",
    top_n=15
)

xgb_importance_df = plot_feature_importance(
    xgb_search,
    model_name="XGBoost",
    top_n=15
)
```

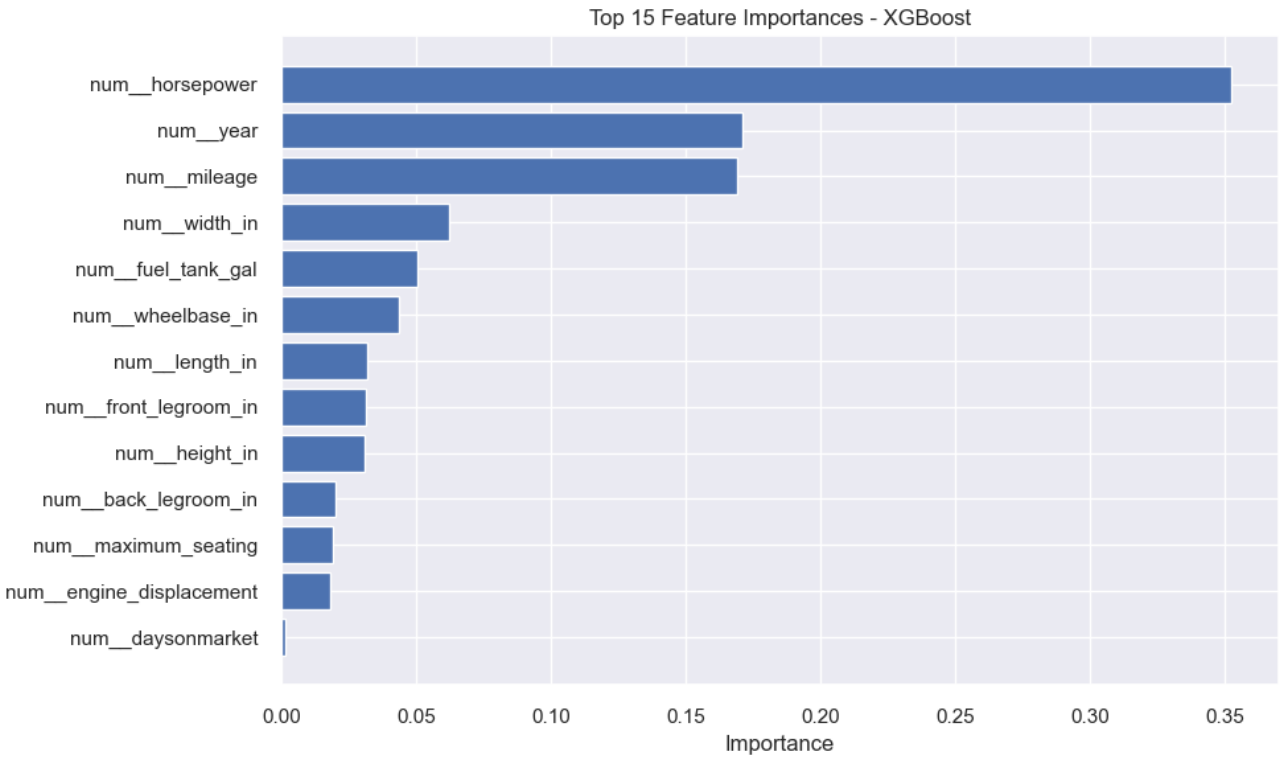
```
<IPython.core.display.Markdown object>
```

	Feature	Importance
6	num__horsepower	0.378470
9	num__mileage	0.318314
12	num__year	0.083114
11	num__width_in	0.038662
4	num__fuel_tank_gal	0.037348
5	num__height_in	0.027146
7	num__length_in	0.025221
1	num__daysonmarket	0.023803
0	num__back_legroom_in	0.020227
10	num__wheelbase_in	0.017761
3	num__front_legroom_in	0.017568
8	num__maximum_seating	0.006606
2	num__engine_displacement	0.005759



<IPython.core.display.Markdown object>

	Feature	Importance
6	num__horsepower	0.352277
12	num__year	0.170891
9	num__mileage	0.168821
11	num__width_in	0.061966
4	num__fuel_tank_gal	0.050368
10	num__wheelbase_in	0.043558
7	num__length_in	0.031638
3	num__front_legroom_in	0.031513
5	num__height_in	0.030692
0	num__back_legroom_in	0.019828
8	num__maximum_seating	0.019239
2	num__engine_displacement	0.017883
1	num__daysonmarket	0.001326



Further discussion including feature importance for the advanced supervised learning model can be found in section 2.4

2.2 Feature Engineering

2.2.1 New Features (polynomial, interactions, domain-specific, text/time)

Chose columns that have a lot of correlation with other columns (checking the correlation matrix from phase 1). For example, daysonmarket doesn't correlate well with any other column, but horsepower correlates with a lot more columns. Squaring the values allows the models to view these values with a polynomial relationship rather than a linear one.

```
feature_cars = create_new_features(cars_clean)
quantDDA(feature_cars)
```

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top /bottom 1%)

0	vin	197762	197762	197762	0	NaN	NaN	[0N0RDER11
1	back_legroom_in	197762	197762	114	0	1933.0	2765.0	
2	body_type	197762	197761	9	1	NaN	NaN	
3	city	197762	197762	3856	0	NaN	NaN	
4	daysonmarket	197762	197762	183	0	11610.0	1781.0	
5	engine_displacement	197762	197762	33	0	3259.0	2833.0	
6	engine_type	197762	196619	19	1143	NaN	NaN	
7	franchise_dealer	197762	197762	2	0	NaN	NaN	
8	front_legroom_in	197762	197762	59	0	4705.0	2571.0	
9	fuel_tank_gal	197762	197762	109	0	2359.0	2513.0	
10	fuel_type	197762	196619	4	1143	NaN	NaN	
11	height_in	197762	197762	200	0	0.0	3931.0	
12	horsepower	197762	197762	202	0	232.0	2992.0	
13	is_new	197762	197762	2	0	NaN	NaN	
14	length_in	197762	197762	336	0	2887.0	3552.0	
15	listed_date	197762	197762	187	0	NaN	NaN	
16	listing_color	197762	197762	15	0	NaN	NaN	
17	make_name	197762	197762	34	0	NaN	NaN	
18	maximum_seating	197762	197762	6	0	40588.0	1665.0	
19	mileage	197762	197762	60132	0	6323.0	1978.0	
20	model_name	197762	197762	394	0	NaN	NaN	
21	price	197762	197762	34452	0	2651.0	3791.0	
22	transmission	197762	194268	4	3494	NaN	NaN	
23	wheel_system	197762	196427	5	1335	NaN	NaN	
24	wheelbase_in	197762	197762	155	0	82.0	3606.0	
25	width_in	197762	197762	177	0	0.0	3034.0	
26	year	197762	197762	9	0	0.0	0.0	
27	age	197762	197762	7	0	4177.0	0.0	
28	horsepower_squared	197762	197762	202	0	1519.0	2992.0	
29	fuel_tank_gal_squared	197762	197762	109	0	8003.0	2513.0	
30	maximum_seating_squared	197762	197762	6	0	40588.0	1665.0	
31	height_in_squared	197762	197762	200	0	0.0	3931.0	
32	mileage_squared	197762	197762	60132	0	23022.0	1978.0	
33	annual_mileage	197762	197762	71561	0	6139.0	1978.0	
34	fake_volume_in	197762	197762	1071	0	174.0	3488.0	
35	horsepower/volume	197762	197762	1777	0	5064.0	3910.0	

36	power_per_displacement	197762	197762	327	0	411.0	2930.0
37	volume_per_seat	197762	197762	1130	0	352.0	1815.0
38	fuel_per_volume	197762	197762	1274	0	1019.0	3903.0
39	is_high_mileage	197762	197762	2	0	2845.0	0.0
40	is_automatic	197762	197762	2	0	0.0	0.0
41	is_fresh_listing	197762	197762	2	0	0.0	0.0
42	is_awd_4wd	197762	197762	2	0	0.0	0.0
43	is_electric_hybrid	197762	197762	2	0	7065.0	0.0
44	listing_season	197762	197762	3	0	55994.0	0.0
45	is_v6	197762	197762	2	0	49351.0	0.0
46	is_v8	197762	197762	2	0	3363.0	0.0
47	cylinder_count	197762	196619	5	1143	0.0	0.0

2.2.2 Feature Selection

Filter

This sequence uses a correlation matrix to correlate each column to the price of a vehicle. It then ranks the correlations from highest to lowest and selects the highest ones (top 20 in this case)

Wrapper Method

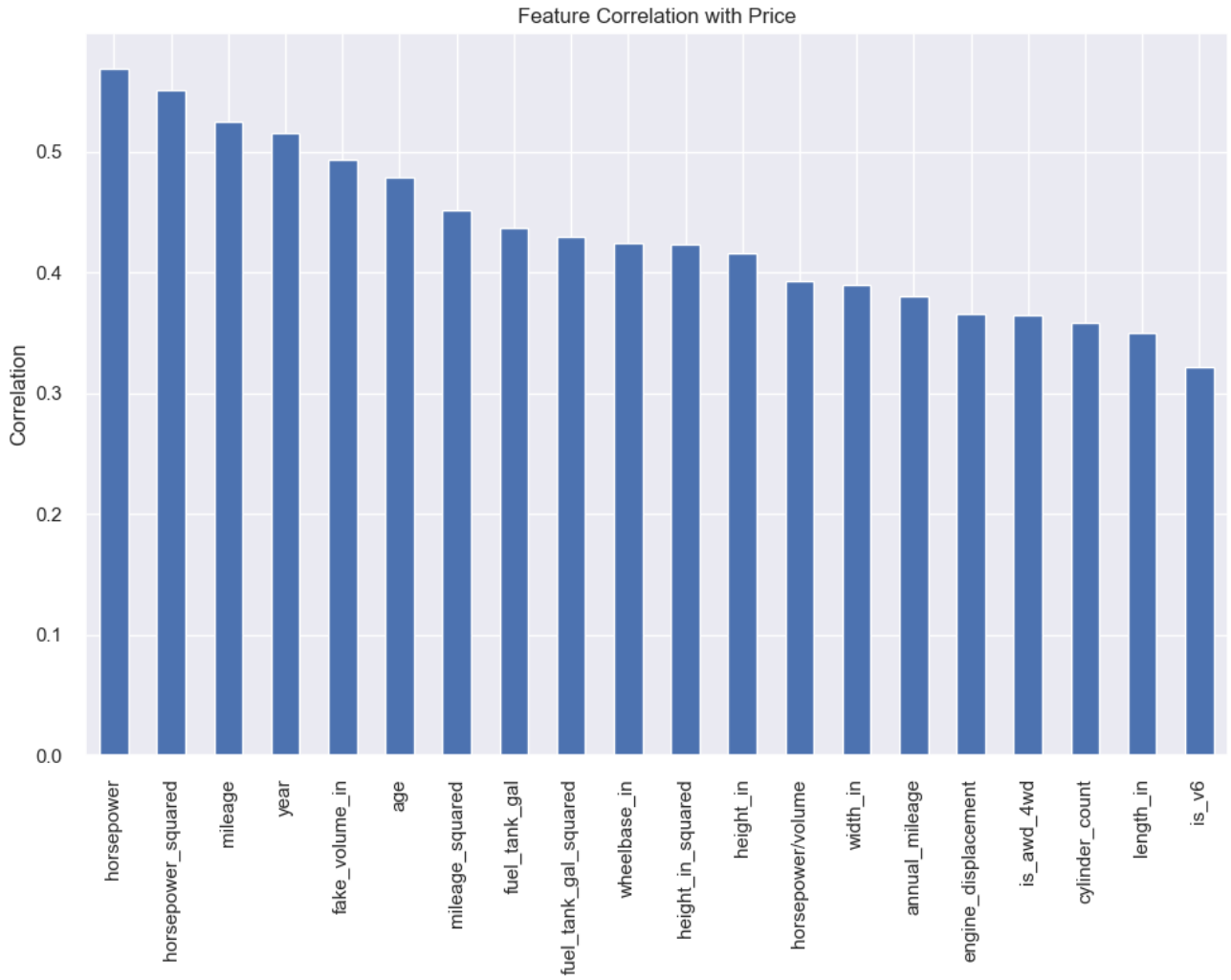
Recursively uses a random forest to find the columns with the most impact. Each time it runs, it prunes the 5 worst performing columns. This algorithm runs until 20 columns are left.

Embedded Method

Lasso regression shrinks the coefficients by a constant. It can also shrink some coefficients to 0, making them obvious targets to remove from the dataset. A random forest is used with the embedded feature.

```
feature_cars2, X_final, y = feature_selection(feature_cars)
```

```
Correlation Matrix



Top features by correlation with price:
horsepower                 0.569676
horsepower_squared         0.551735
mileage                    0.525328
year                       0.515867
fake_volume_in             0.493517
age                        0.479431
mileage_squared            0.452068
fuel_tank_gal              0.436917
fuel_tank_gal_squared      0.430055
wheelbase_in               0.424333
height_in_squared          0.423056
height_in                  0.416533
horsepower/volume          0.393348
width_in                   0.389586
annual_mileage             0.380805
engine_displacement        0.366142
is_awd_4wd                 0.364393
cylinder_count             0.358209
length_in                  0.350518
is_v6                      0.322298
maximum_seating_squared    0.306812
maximum_seating            0.305945
volume_per_seat            0.278544
power_per_displacement     0.246995
back_legroom_in            0.237431
front_legroom_in           0.207565
is_v8                      0.158453
is_high_mileage            0.157244
is_automatic               0.142201
is_electric_hybrid         0.052679
is_fresh_listing           0.009043
listing_season             0.008849
daysonmarket               0.005837
fuel_per_volume            0.002545
dtype: float64
```




```
SelectKBest selected features: ['engine_displacement', 'fuel_tank_gal', 'height_in',
'horsepower', 'length_in', 'mileage', 'wheelbase_in', 'width_in', 'year', 'age',
'horsepower_squared', 'fuel_tank_gal_squared', 'height_in_squared', 'mileage_squared',
'annual_mileage', 'fake_volume_in', 'horsepower/volume', 'is_awd_4wd', 'is_v6',
'cylinder_count']
```

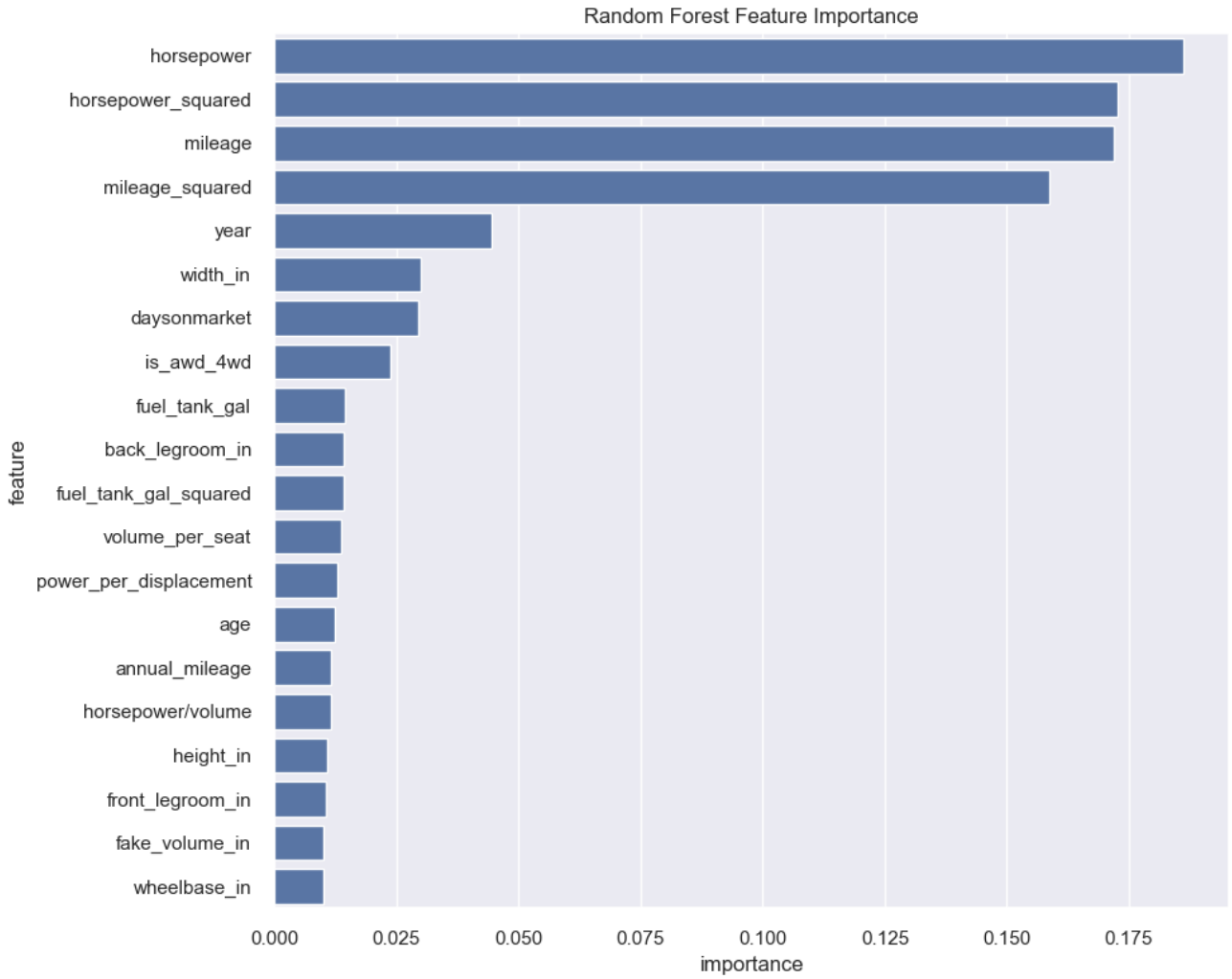
Wrapper method

```
Fitting estimator with 34 features.
Fitting estimator with 29 features.
Fitting estimator with 24 features.
RFE selected features: ['back_legroom_in', 'daysonmarket', 'front_legroom_in',
'fuel_tank_gal', 'height_in', 'horsepower', 'mileage', 'wheelbase_in', 'width_in', 'year'
, 'age', 'horsepower_squared', 'fuel_tank_gal_squared', 'height_in_squared',
'mileage_squared', 'annual_mileage', 'horsepower/volume', 'power_per_displacement',
```

```
'volume_per_seat', 'is_awd_4wd']
back_legroom_in           1
volume_per_seat           1
power_per_displacement    1
horsepower/volume         1
annual_mileage            1
mileage_squared           1
height_in_squared         1
fuel_tank_gal_squared     1
horsepower_squared        1
age                       1
width_in                  1
year                      1
mileage                   1
horsepower                1
height_in                 1
fuel_tank_gal             1
front_legroom_in          1
daysonmarket              1
wheelbase_in              1
is_awd_4wd                1
dtype: int64
```

Embedded Method

```
/Users/charlottelauzon/miniconda3/envs/COMP333_PROJECT_GROUP_O/lib/python3.13/site-
packages/sklearn/linear_model/_coordinate_descent.py:716: ConvergenceWarning: Objective
did not converge. You might want to increase the number of iterations, check the scale of
the features or consider increasing regularisation. Duality gap: 2.369e+12, tolerance:
1.987e+09
  model = cd_fast.enet_coordinate_descent(
Lasso selected features: ['back_legroom_in', 'daysonmarket', 'engine_displacement',
'front_legroom_in', 'fuel_tank_gal', 'height_in', 'horsepower', 'length_in',
'maximum_seating', 'mileage', 'wheelbase_in', 'width_in', 'year', 'age',
'horsepower_squared', 'fuel_tank_gal_squared', 'maximum_seating_squared',
'height_in_squared', 'mileage_squared', 'annual_mileage', 'fake_volume_in', 'horsepower
/volume', 'power_per_displacement', 'volume_per_seat', 'fuel_per_volume',
'is_high_mileage', 'is_automatic', 'is_fresh_listing', 'is_awd_4wd',
'is_electric_hybrid', 'listing_season', 'is_v6', 'is_v8', 'cylinder_count']
```




```
Selected Features



Features selected by ALL methods: {'horsepower/volume', 'year', 'horsepower_squared',
'mileage_squared', 'fuel_tank_gal', 'fuel_tank_gal_squared', 'annual_mileage',
'wheelbase_in', 'horsepower', 'mileage', 'age', 'width_in', 'height_in', 'is_awd_4wd'}
Features selected by at least 2 methods: ['fuel_tank_gal', 'height_in', 'horsepower',
'mileage', 'wheelbase_in', 'width_in', 'year', 'age', 'horsepower_squared',
'fuel_tank_gal_squared', 'height_in_squared', 'mileage_squared', 'annual_mileage',
'fake_volume_in', 'horsepower/volume', 'is_awd_4wd', 'back_legroom_in', 'daysonmarket',
'front_legroom_in', 'power_per_displacement', 'volume_per_seat']
```

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top /bottom 1%)	
0	fuel_tank_gal	197762	197762	109	0	2359	2513	
1	height_in	197762	197762	200	0	0	3931	
2	horsepower	197762	197762	202	0	232	2992	
3	mileage	197762	197762	60132	0	6323	1978	
4	wheelbase_in	197762	197762	155	0	82	3606	
5	width_in	197762	197762	177	0	0	3034	
6	year	197762	197762	9	0	0	0	
7	age	197762	197762	7	0	4177	0	
8	horsepower_squared	197762	197762	202	0	1519	2992	
9	fuel_tank_gal_squared	197762	197762	109	0	8003	2513	[174.239999
10	height_in_squared	197762	197762	200	0	0	3931	[4369.2099
11	mileage_squared	197762	197762	60132	0	23022	1978	
12	annual_mileage	197762	197762	71561	0	6139	1978	
13	fake_volume_in	197762	197762	1071	0	174	3488	[8
14	horsepower/volume	197762	197762	1777	0	5064	3910	[0.000195544771
15	is_awd_4wd	197762	197762	2	0	0	0	
16	back_legroom_in	197762	197762	114	0	1933	2765	
17	daysonmarket	197762	197762	183	0	11610	1781	
18	front_legroom_in	197762	197762	59	0	4705	2571	
19	power_per_displacement	197762	197762	327	0	411	2930	
20	volume_per_seat	197762	197762	1130	0	352	1815	[17
21	price	197762	197762	34452	0	2651	3791	

2.2.3 Baseline Model Comparison

First Model is pre feature engineering

Using the same technique from phase 1 to generate the baseline model, a new baseline model is generated with the feature engineering data for comparison. The first baseline model with the original clean data can be found in section 1.4.5. This section only includes the baseline model with feature_cars (full feature engineered dataset) and feature_cars2 (trimmed feature engineer dataset).

Comparison Discussion

Looking at the scores above, the dataset that contains all the new features scores the highest. This dataset scores roughly 6 points higher than the raw clean dataset. The partial-feature dataset (feature dataset with feature selection) only scores roughly 4 points higher than the raw clean dataset and 2 points less than the full feature dataset. However, the partial-feature dataset only contains 20 columns of data compared to the 47 columns in the feature dataset and 29 columns in the clean cars dataset.

Further discussion is found below in Phase 2.4 Interpretation

```
baseline_results = run_baseline_linear_regression_pipeline(
    df=feature_cars,
    target_col="price",
    columns_to_drop=["vin", "body_type", "franchise_dealer"],
    train_size=0.70,
    val_size=0.15,
    test_size=0.15,
    random_state=42,
    high_unique_threshold=0.95
)
baseline_results = run_baseline_linear_regression_pipeline(
    df=feature_cars2,
    target_col="price",
    columns_to_drop=["vin", "body_type", "franchise_dealer"],
    train_size=0.70,
    val_size=0.15,
    test_size=0.15,
    random_state=42,
    high_unique_threshold=0.95
)
```

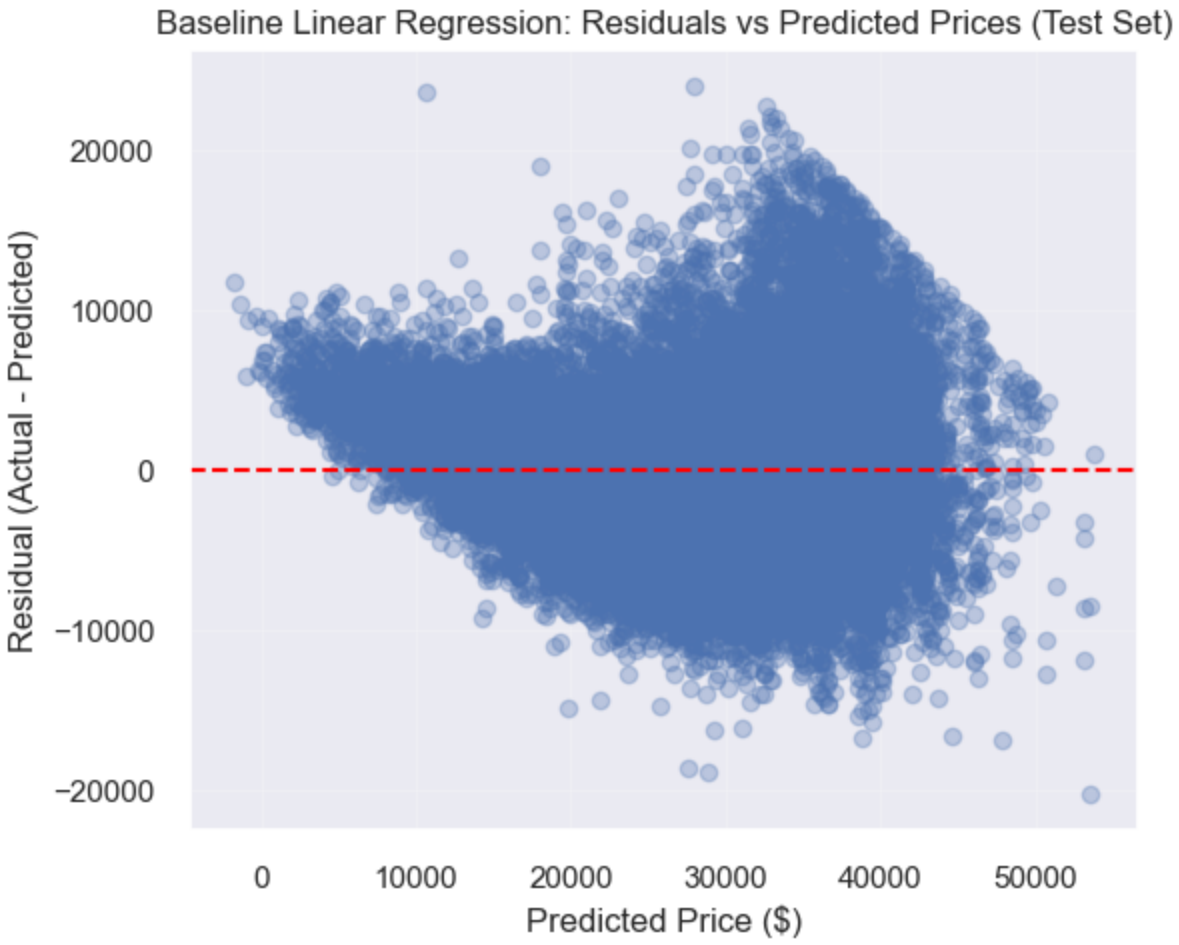
```
<IPython.core.display.Markdown object>
```

<IPython.core.display.Markdown object>

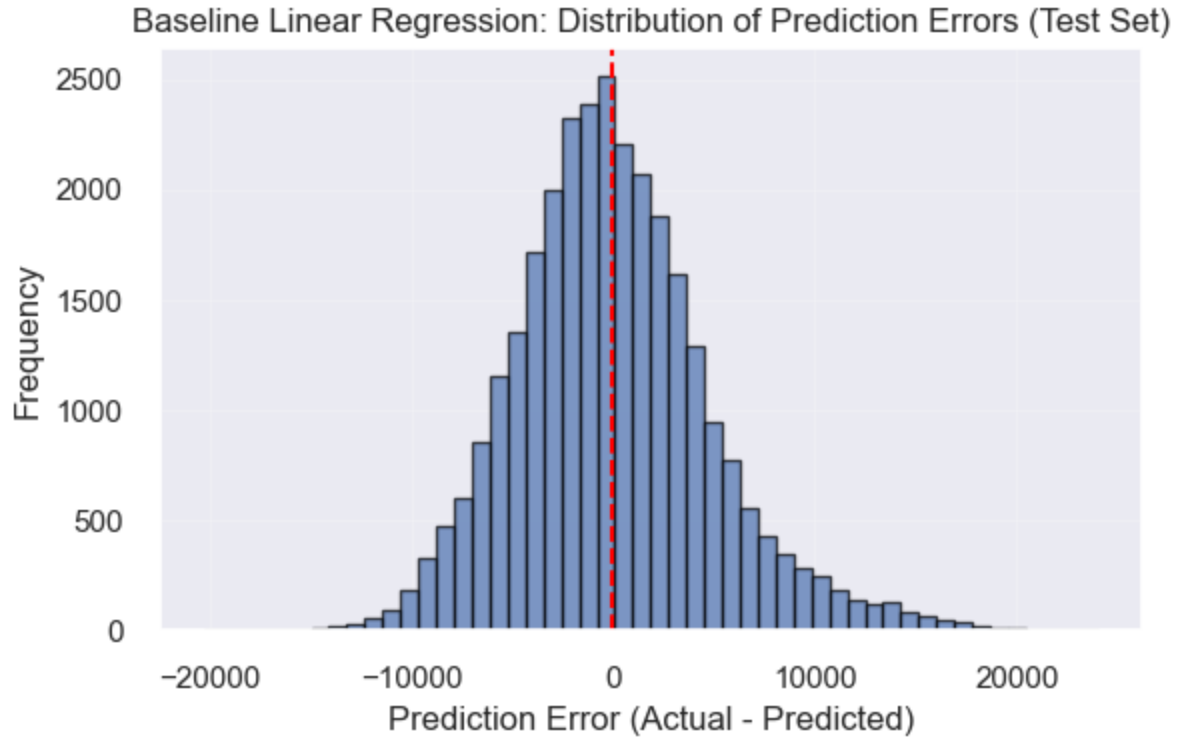
Baseline Linear Regression Performance (Train / Validation / Test)

	Set	R2	MAE ($)	RMSE ($)	MAPE (%)	MedAE
0	Train	0.7614	3,750.61	4,907.03	15.50	2,974.90
1	Validation	0.7612	3,739.31	4,895.76	15.49	2,958.65
2	Test	0.7577	3,748.56	4,886.45	15.55	2,983.04

<IPython.core.display.Markdown object>



<IPython.core.display.Markdown object>



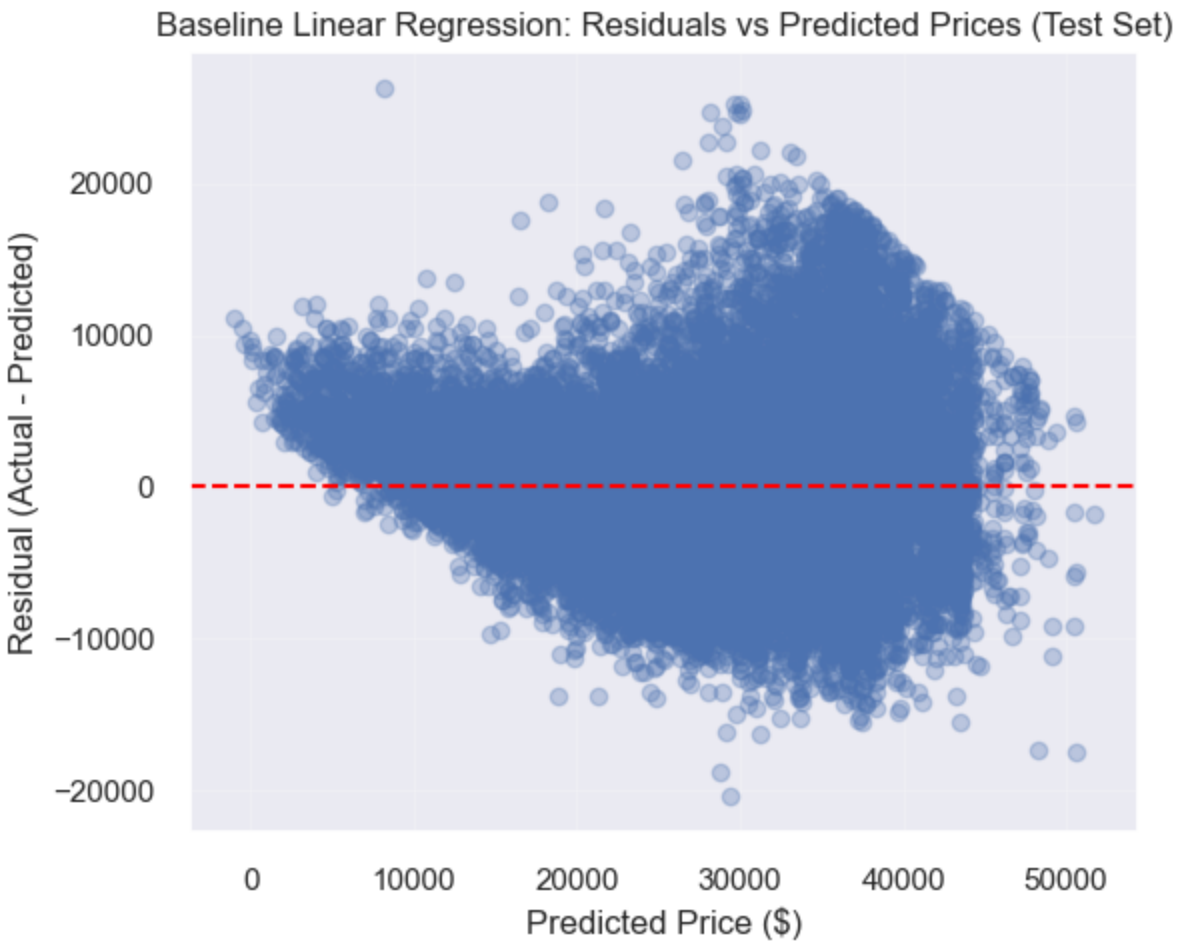
<IPython.core.display.Markdown object>

<IPython.core.display.Markdown object>

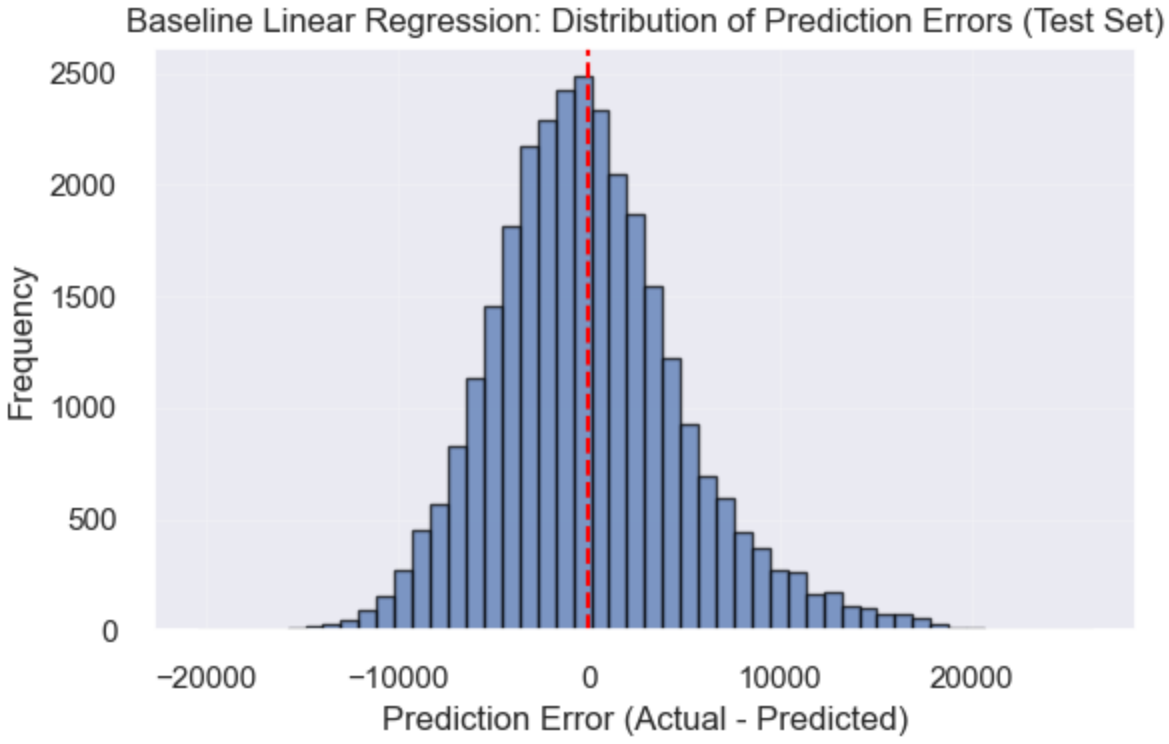
Baseline Linear Regression Performance (Train / Validation / Test)

	Set	R2	MAE ($)	RMSE ($)	MAPE (%)	MedAE
0	Train	0.7441	3,894.95	5,081.87	16.00	3,112.60
1	Validation	0.7441	3,877.57	5,067.27	15.95	3,092.34
2	Test	0.7410	3,880.32	5,051.66	15.97	3,112.31

<IPython.core.display.Markdown object>



<IPython.core.display.Markdown object>



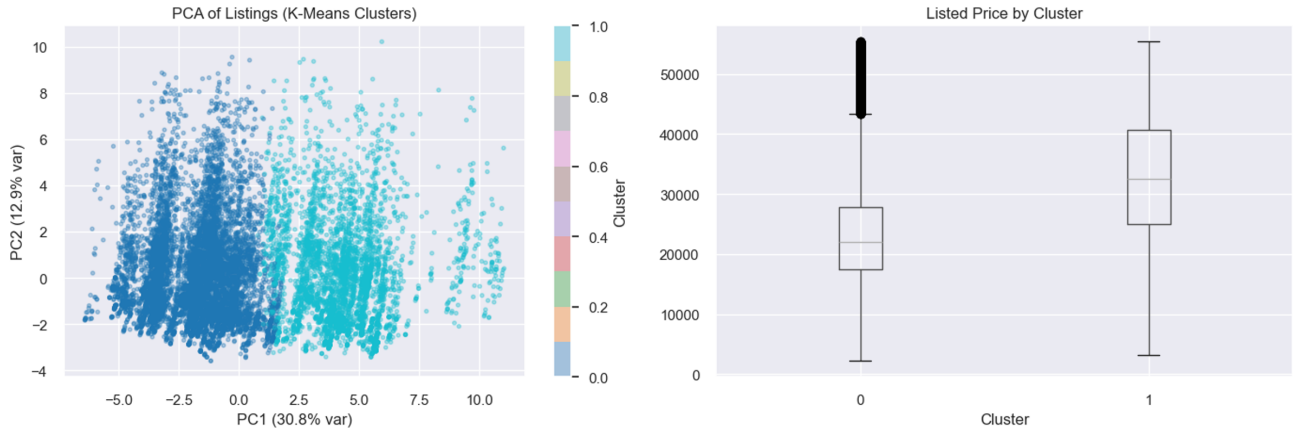
2.3 Unsupervised Learning

We address **Research Question 2** (from the EDA section): can we find **natural groupings** of vehicles in feature space without using price as a clustering input? We **exclude** ***price*** from the clustering features so clusters reflect specifications and market attributes, then **compare** ***price*** **across clusters** to interpret segments (e.g. lower vs higher priced groups).

Method: We standardize numeric features, run **K-Means** for several values of *k*, and pick *k* using the **silhouette score** (computed on a random subsample for speed). We visualize structure with **PCA** (2D) and summarize **price** by cluster with boxplots and median/mean tables.

```
df_ul = feature_cars if "feature_cars" in globals() else cars_clean

clustering_results = run_kmeans_pca_clustering(
    df=df_ul,
    silhouette_fit_rows=50000,
    silhouette_score_rows=8000,
    plot_rows=15000,
    k_values=range(2, 9),
    random_state=42
)
```

```
Silhouette (sample) by k: {2: 0.2729, 3: 0.1708, 4: 0.1242, 5: 0.1339, 6: 0.1394, 7:
0.1393, 8: 0.1448}
Chosen k: 2
```




```
Price by cluster:
```

cluster	count	median	mean
0	138818	22008.5	23355.303921
1	58944	32500.0	32762.030197

Interpretation. Clusters are formed only from numeric vehicle/listing features (price held out), so separation in PCA space reflects combinations of specs and market fields rather than price itself. If median price differs clearly across clusters, those groups behave like interpretable **market segments** (e.g. lower vs higher price bands for different feature mixes). Silhouette-guided *k* balances cohesion and separation; segments are descriptive, not a predictive model of price. The relatively low silhouette score (~0.27) indicates that the separation between clusters is moderate, suggesting that while meaningful groupings exist, the dataset does not exhibit strongly distinct clusters.

2.4 Interpretation

Feature Importance

feature_cars2 features the 20 most important features for price prediction. This dataset does not offer the best performance. feature_cars offers slightly better performance as seen in 2.2 Comparison Discussion. If there are computational constraints, it is better to use feature_cars2 because if offers very good performance for only 20 columns of data. If there are no computational constraints, use feature_cars.

horsepower and mileage are very good predictors of price. These two variables constituency place as top predictors. The variables year and age also appear to be strong predictors of price. Although they aren't as good as horsepower and mileage, they should not be ignored. It's expected that horsepower would be a strong predictor because this is the primary indicator of a car's performance. More horsepower means that car has better acceleration, top speed and towing capacity, all of which typically demand a premium price for high performance. mileage is also a strong predictor of price because it signifies how much the car has been used. Typically, cars are made to drive a certain amount of mileage before breaking, so high mileage indicates how close the car is to being worth scraps. Along with mileage, year and age are also indicators of a vehicle's age and how much longer the car might last.

Related to the Advanced Supervised Models , the feature importance analysis for the Random Forest and XGBoost Models reveal that horsepower, mileage, and vehicle year are the most influential variables for predicting price.

Horsepower is the most important feature; this indicates that vehicle performance plays a major role in determining the price. We also notice that mileage is highly influential, and that higher mileage is associated to lower prices (we can assumer due to vehicle wear and/or depreciation). The vehicle year is the other key factor, which reflect the impact of age on price.

The feature importance results are consistent across both models, which strengthens the reliability of the findings and confirm that both usage-related and performance-related characteristics influence the vehicle price. Other feature such as vehicle dimensions and fuel attributes have a smaller but still noticeable impact.

Here are some important engineered features:

- is_awd_4wd ranked in all 3 tests. This feature checks if the vehicle is all-wheel drive or 4-wheel drive. This feature was engineered to hold true or false values (1 or 0) so the machine learning algorithms have numerical values to work with.
- horsepower/volume ranked in all 3 tests. It doesn't score the highest, but it is consistent and provides good insight on a car's predicted price. It is a feature that calculates how much horsepower there is per unit of volume (inches).
- fake_volume_in ranked in all 3 tests. It had very mixed results in all three tests, ranking high in some and low in others. This variable does not contain the actual vehicles volume data, instead it simply finds the fake volume of the vehicle by calculating its height * width * length.
- power_per_displacement, volume_per_seat and fuel_per_volume. All three of these features barely make the cut into feature_cars2 and only appear significant in 2 of 3 algorithms. These engineered features alone don't provide much value, but they are still significant enough to be included in the top 20 features.

Relation to Research Question

The results seem to directly answer the research question by demonstrating that vehicle specifications, usage history, and dealership-related characteristics can predict the selling price of a used vehicle. Particularly, the strong influence of mileage, horsepower, and vehicle age confirms that both usage and performance features are key determinants of price. The use of advanced models such as Random Forest and XGBoost further shows that capturing relationships that are non-linear significantly improves predictive performance.

Partial Dependence Plots and Insights

The partial dependence values had to follow a log distribution because the original values do not provide any meaningful insight. Because of this, the graphs do not show actual prices. However, we can still analyze the change compared to relative pricing.

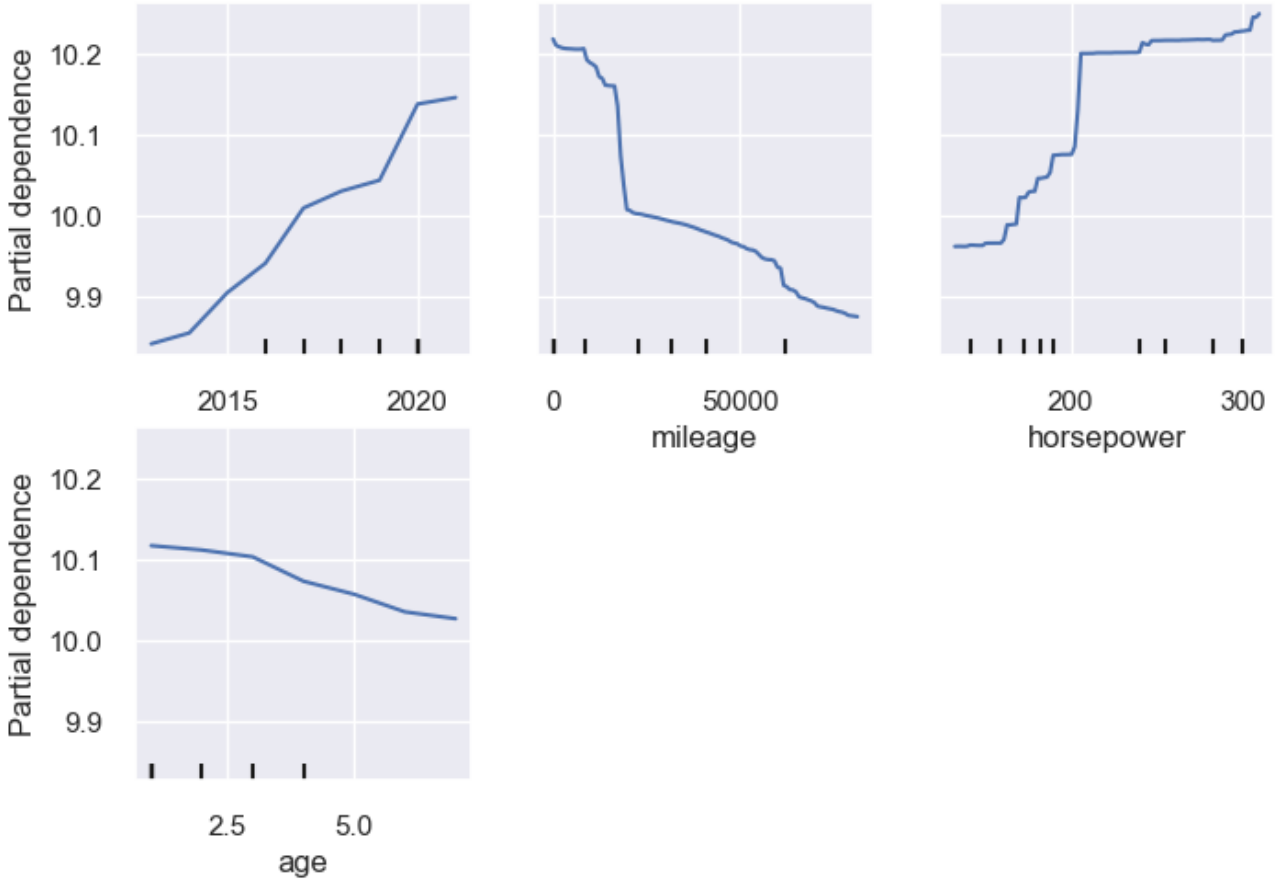
The first and fourth graph that focus on year and age. These two graphs are observed together because they share similar results, but the primary focus will be on graph 1 (year). We see that the price of the car increases almost linearly with increasing years. However, the plot is not perfectly linear. Between the years 2017 and 2019 we see a smaller increase in price compared to the rest of the graph. This phenomenon can also be observed at the lowest and highest years of this graph. With this observation we see that cars hold their value well for the first year then drop significantly after that. They once again hold there value for a few more years before seeing another significant drop. Cars made before 2014 don't lose value as fast anymore compared to newer cars.

The second graph focuses on mileage. We observe that cars lose value very slowly for the first few thousand miles driven. At one point (after roughly 8000 of miles are driven) a steep decline in car value is observed. After that initial steep decline, the car's value decrease slowly but steadily until about 58,000 miles have been driven. At this point another small but quick decline is observed, but quickly corrects itself back to the same previous slow steady decline. This graphs shows that cars tend to hold their value well for a few thousand miles, but then suddenly lose value once a certain point is reached. But after this huge drop in value, cars begin to depreciate in value slowly and constistenly.

The third graph shows how mileage effects price. The graph shows no increase in price until a little under 100 horsepower. After this point that graph increases like a staircase until about 200 horsepower. At the 200 horsepower marker, there is a huge leap in price. However, after this huge leep in price the graph once again plateaus and only sees small increases in price. This graph shows that at about 100 horsepower, the demand more small increases in horsepower drastically increase the value of a vehicle. The asking price for a vehicle with a little over 200 horsepower is worth significantly more than a vehicle with even a bit less than 200 horsepower. After a little over 200 horsepower is reached, the price of a vehicle slowly increases compared to the increases seen between 100 and 200 horsepower.

```
partial_dependence_plots(X_final, y)
```

```
/Users/charlottelauzon/miniconda3/envs/COMP333_PROJECT_GROUP_O/lib/python3.13/site-packages/sklearn/inspection/_partial_dependence.py:721: FutureWarning: The column 6 contains integer data. Partial dependence plots are not supported for integer data: this can lead to implicit rounding with NumPy arrays or even errors with newer pandas versions. Please convert numerical featuresto floating point dtypes ahead of time to avoid problems. This will raise ValueError in scikit-learn 1.9.
  warnings.warn(
/Users/charlottelauzon/miniconda3/envs/COMP333_PROJECT_GROUP_O/lib/python3.13/site-packages/sklearn/inspection/_partial_dependence.py:721: FutureWarning: The column 7 contains integer data. Partial dependence plots are not supported for integer data: this can lead to implicit rounding with NumPy arrays or even errors with newer pandas versions. Please convert numerical featuresto floating point dtypes ahead of time to avoid problems. This will raise ValueError in scikit-learn 1.9.
  warnings.warn(
```



Phase 3

This phase **synthesizes** the supervised price-prediction work (Phases 1–2) and the **unsupervised** segmentation analysis (§2.3). We restate how each part answers the course research questions, then discuss limitations and ethical implications of using these models in practice.

3.0 Bonus implementation

Before synthesizing, we implement some bonus features.

First, we will do ensemble stacking on the best featured-engineered dataset from Phase 2. We will compare the results with the Random Forest and XGBoost models, which will help further in the comprehensive evaluation.

Then, we will also do some anomaly detection

```
# --------------------------------
#
# PHASE 3 PIPELINE
# BONUS
#
# --------------------------------
from sklearn.ensemble import StackingRegressor, IsolationForest
from sklearn.linear_model import Ridge

# ----------------
# 3.0 BONUS
# ----------------

def train_stacking_model(randomForest_search, xgb_search, X_train, y_train):
    """
    Trains a stacking regression model using the tuned Random Forest and XGBoost models.

    The base estimators are the best estimators found by RandomizedSearchCV.
    A Ridge regressor is used as the final estimator to combine their predictions.

    Parameters:
    - randomForest_search: fitted RandomizedSearchCV object for Random Forest
    - xgb_search: fitted RandomizedSearchCV object for XGBoost
    - X_train: training feature matrix
    - y_train: training target vector

    Returns:
    - stacking_model: fitted StackingRegressor model
    """

    stacking_model = StackingRegressor(
```

```
        estimators=[
            ("rf", randomForest_search.best_estimator_),
            ("xgb", xgb_search.best_estimator_)
        ],
        final_estimator=Ridge(),
        cv=3,
        n_jobs=-1,
        passthrough=False
    )

    stacking_model.fit(X_train, y_train)

    return stacking_model

# --------------------------------

from sklearn.ensemble import IsolationForest
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, Markdown

def fit_anomaly_detection(X_train, contamination, random_state):
    """
    Fits Isolation Forest on TRAIN numeric features only.

    Returns:
    - anomaly_model: fitted IsolationForest model
    - X_train_numeric: numeric training data used for fitting
    """
    X_train_numeric = X_train.select_dtypes(include=["number"]).copy()

    anomaly_model = IsolationForest(
        contamination=contamination,
        random_state=random_state
    )
    anomaly_model.fit(X_train_numeric)

    return anomaly_model, X_train_numeric

# --------------------------------

def apply_anomaly_detection(anomaly_model, X_data):
    """
    Applies a fitted Isolation Forest model to a dataset.

    Returns:
    - X_numeric: numeric feature matrix used
    - anomaly_labels: array (-1 anomaly, 1 normal)
    - anomaly_scores: array (higher = more anomalous)
    - anomaly_summary: dictionary
    """
    X_numeric = X_data.select_dtypes(include=["number"]).copy()
```

```
    anomaly_labels = anomaly_model.predict(X_numeric)

    # sklearn decision_function: larger = more normal
    # invert so larger = more anomalous
    anomaly_scores = -anomaly_model.decision_function(X_numeric)

    anomaly_summary = {
        "normal_count": int((anomaly_labels == 1).sum()),
        "anomaly_count": int((anomaly_labels == -1).sum()),
        "anomaly_rate_percent": float((anomaly_labels == -1).mean() * 100)
    }

    return X_numeric, anomaly_labels, anomaly_scores, anomaly_summary

# --------------------------------

def display_anomaly_results(anomaly_model, X_data, dataset_name, n_examples):
    """
    Displays anomaly summary, top anomalies, and score distribution
    for a dataset using a PRE-FITTED model.
    """
    X_numeric, anomaly_labels, anomaly_scores, anomaly_summary = apply_anomaly_detection(
        anomaly_model=anomaly_model,
        X_data=X_data
    )

    display(Markdown(f"### Anomaly Detection Summary — {dataset_name}"))
    print(anomaly_summary)

    results_df = X_numeric.copy()
    results_df["anomaly_score"] = anomaly_scores
    results_df["anomaly_label"] = anomaly_labels

    anomalies_only = (
        results_df[results_df["anomaly_label"] == -1]
        .sort_values("anomaly_score", ascending=False)
    )

    display(Markdown(f"### Most Anomalous Listings — {dataset_name}"))
    display(anomalies_only.head(n_examples))

    plt.figure(figsize=(7, 4))
    plt.hist(anomaly_scores, bins=50, edgecolor="black", alpha=0.7)
    plt.axvline(
        x=np.min(anomaly_scores[anomaly_labels == -1]) if (anomaly_labels == -1).any()
else 0,
        linestyle="--"
    )
    plt.xlabel("Anomaly Score (higher = more anomalous)")
    plt.ylabel("Frequency")
    plt.title(f"Isolation Forest Anomaly Score Distribution ({dataset_name})")
    plt.grid(alpha=0.2)
```

```
    plt.tight_layout()
    plt.show()

    return {
        "X_numeric": X_numeric,
        "anomaly_labels": anomaly_labels,
        "anomaly_scores": anomaly_scores,
        "anomaly_summary": anomaly_summary,
        "results_df": results_df,
        "anomalies_only": anomalies_only
    }

# --------------------------------

def compare_residuals_by_anomaly(models_dict, X_test, y_test, anomaly_results):
    """
    Compares residual behavior for normal vs anomalous rows on the TEST set.
    """
    anomaly_labels = anomaly_results["anomaly_labels"]
    mask_anomaly = anomaly_labels == -1
    mask_normal = anomaly_labels == 1

    rows = []

    for model_name, model in models_dict.items():
        y_pred = model.predict(X_test)
        residuals = y_test - y_pred
        abs_residuals = np.abs(residuals)

        rows.append({
            "Model": model_name,
            "Group": "Normal",
            "Count": int(mask_normal.sum()),
            "Mean Abs Residual": float(abs_residuals[mask_normal].mean()),
            "Median Abs Residual": float(np.median(abs_residuals[mask_normal])),
            "RMSE": float(np.sqrt(np.mean((residuals[mask_normal]) ** 2)))
        })

        rows.append({
            "Model": model_name,
            "Group": "Anomaly",
            "Count": int(mask_anomaly.sum()),
            "Mean Abs Residual": float(abs_residuals[mask_anomaly].mean()),
            "Median Abs Residual": float(np.median(abs_residuals[mask_anomaly])),
            "RMSE": float(np.sqrt(np.mean((residuals[mask_anomaly]) ** 2)))
        })

    return pd.DataFrame(rows)

# --------------------------------

def plot_residuals_with_anomalies(model, X_test, y_test, anomaly_results, model_name):
    """
```

```
    Residual plot colored by anomaly vs normal (TEST set)
    """
    # Predictions
    y_pred = model.predict(X_test)
    residuals = y_test - y_pred

    # Anomaly labels
    anomaly_labels = anomaly_results["anomaly_labels"]

    # Masks
    normal_mask = anomaly_labels == 1
    anomaly_mask = anomaly_labels == -1

    # Plot
    plt.figure(figsize=(7, 5))

    # Normal points
    plt.scatter(
        y_pred[normal_mask],
        residuals[normal_mask],
        alpha=0.3,
        label="Normal"
    )

    # Anomalies
    plt.scatter(
        y_pred[anomaly_mask],
        residuals[anomaly_mask],
        alpha=0.6,
        label="Anomaly"
    )

    # Reference line
    plt.axhline(0, linestyle="--")

    plt.xlabel("Predicted Price ($)")
    plt.ylabel("Residual (Actual - Predicted)")
    plt.title(f"{model_name}: Residuals vs Predicted (Anomaly Highlight)")
    plt.legend()
    plt.grid(alpha=0.2)
    plt.tight_layout()
    plt.show()
```

3.0.1 Ensemble Stacking

Ensemble Stacking is used to improve predictive performance by combining multiple strong models that capture different patterns in the data. In this project, Random Forest and XGBoost have shown to be appropriate models for supervised learning. They rely on different mechanisms, as Random Forest reduces variance through bagging, which improves the stability and accuracy of the model, while XGBoost reduces bias through boosting.

By combining their predictions, stacking aims to leverage the complementary strengths of both models. This allows a final model to reduce individual model biases and produce more stable and generalized predictions than any single model alone. Since we had found that Random Forest and XGBoost could both be justified as the better model through different metrics, it could be interesting to see if stacking could be better on all metrics.

The stacking model is trained on the `feature_cars` dataset, which contains the subset of features selected during the feature selection phase. This ensures that the model is trained only on the most relevant and informative variables to predict vehicle price, reducing noise, improving generalization and limiting the impact of less informative variables. Using this optimized feature set improves model generalization and stability, while also ensuring consistency with the earlier modeling steps. As a result, the ensemble model is able to combine predictions from strong base learners on a refined representation of the data, maximizing its predictive performance.

We start by making a new model on the feature_cars dataset

```
X_train, X_val, X_test, y_train, y_val, y_test = base_model_setup(
    df=feature_cars,
    target_col="price",
    columns_to_drop=["vin"],
    train_size=0.70,
    val_size=0.15,
    test_size=0.15,
    random_state=42,
    high_unique_threshold=0.95
)
preprocessor, numeric_features, categorical_features = advanced_model_setup(X_train)
```

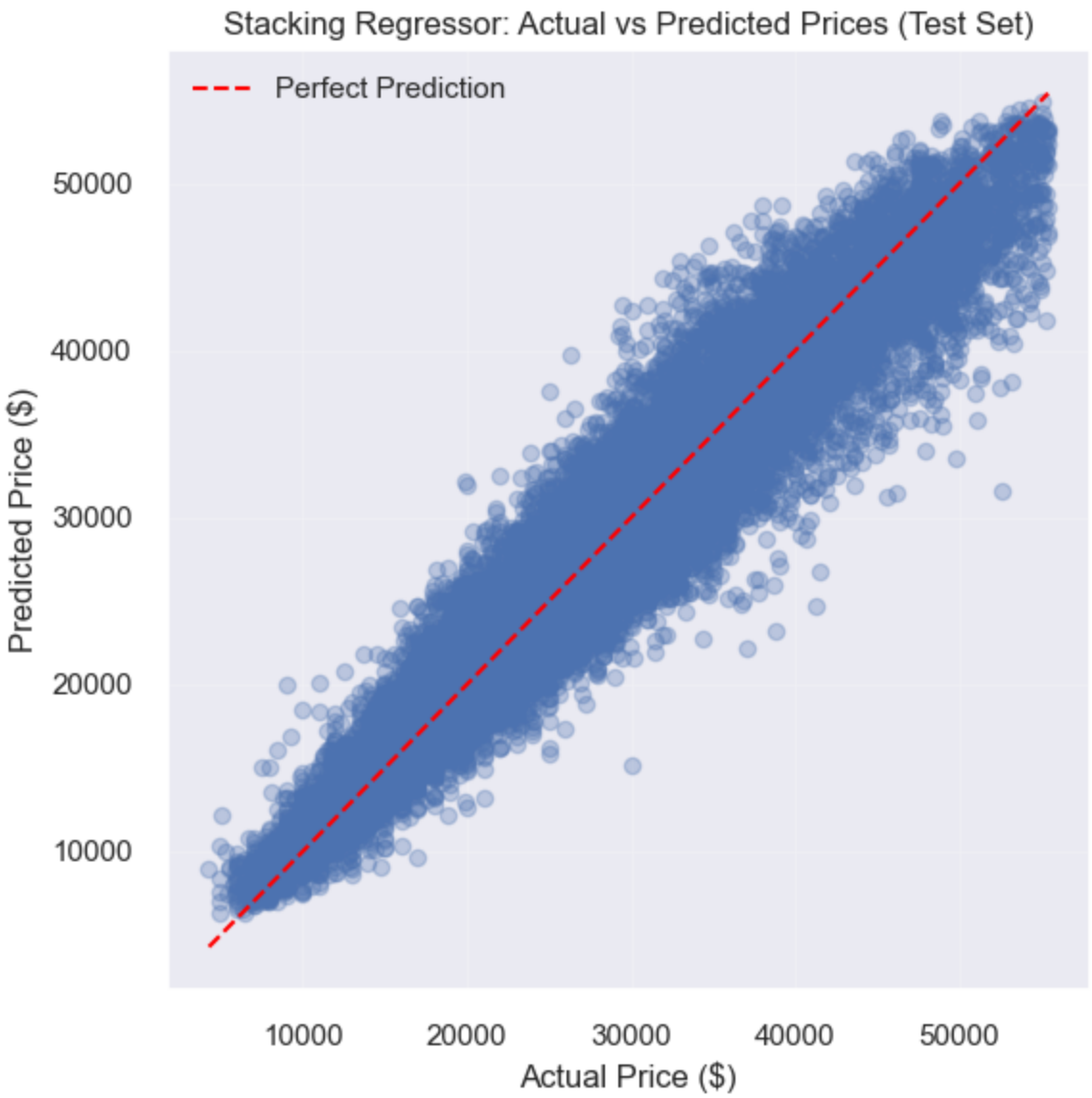
Then we train Random Forest and XGBoost models on feature_cars

```
# Again, n_jobs can be set to -1 or 2 on some computers to not get an error message
randomForest_search = train_random_forest_model(
    preprocessor=preprocessor,
    X_train=X_train,
    y_train=y_train,
    random_state=42,
    n_iter=5,
    cv=3,
    scoring="r2",
    n_jobs=1
)

xgb_search = train_xgboost_model(
    preprocessor=preprocessor,
    X_train=X_train,
    y_train=y_train,
    random_state=42,
    n_iter=5,
    cv=3,
    scoring="r2",
    n_jobs=-1
)
```

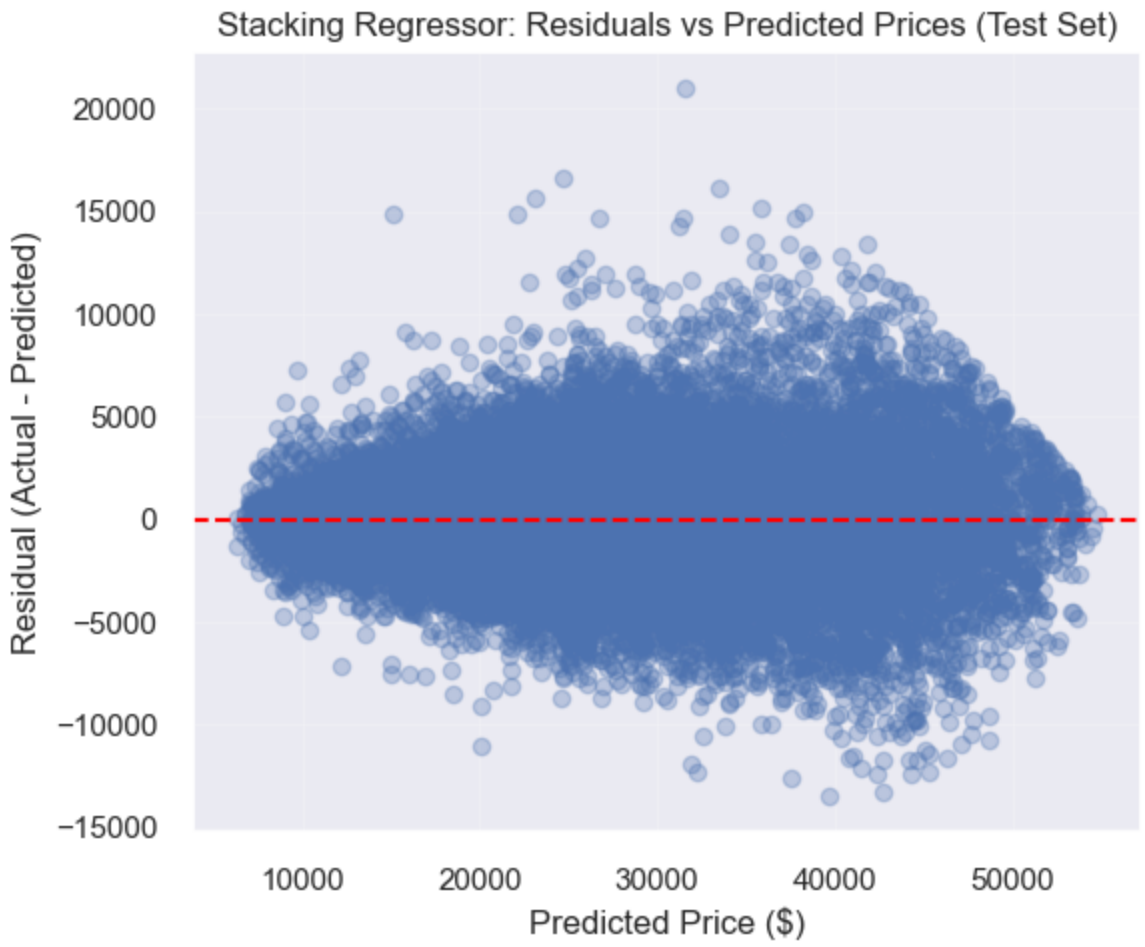
We can then train the stacking model, and evaluate it. We can also compare it to the Random Forest model and XGBoost model

```python
# 1. train stacking
stacking_model = train_stacking_model(
    randomForest_search=randomForest_search,
    xgb_search=xgb_search,
    X_train=X_train,
    y_train=y_train
)

# 2. evaluate stacking
stacking_results = display_model_results(
    trained_model=stacking_model,
    X_train=X_train,
    y_train=y_train,
    X_val=X_val,
    y_val=y_val,
    X_test=X_test,
    y_test=y_test,
    model_name="Stacking Regressor"
)

# 3. compare all models
models_dict = {
    "Random Forest": randomForest_search.best_estimator_,
    "XGBoost": xgb_search.best_estimator_,
    "Stacking": stacking_model
}

comparison_results = run_model_regression_metrics_comparison(
    models_dict=models_dict,
    X_test=X_test,
    y_test=y_test
)

compare_models_actual_vs_pred(
    models_dict=models_dict,
    X_test=X_test,
    y_test=y_test
)

compare_models_residuals(
    models_dict=models_dict,
    X_test=X_test,
    y_test=y_test
)
```

```
<IPython.core.display.Markdown object>
```




```
Stacking Regressor Train R2: 0.9506179378649008
Stacking Regressor Validation R2: 0.9280215758048407
Stacking Regressor Test R2: 0.9264160260152148
<IPython.core.display.Markdown object>
```

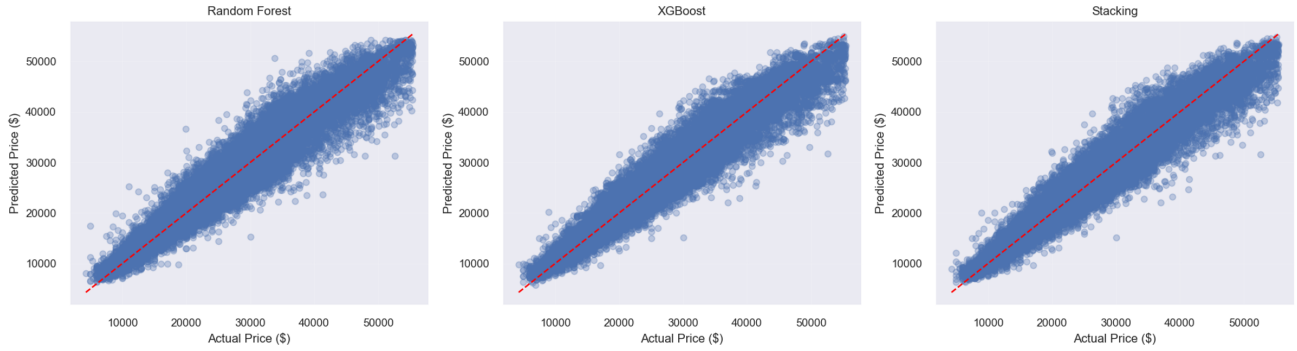


<IPython.core.display.Markdown object>

Comparison of Models on Test Set

	Model	MAE ($)	RMSE ($)	R2
0	Stacking	1,995.16	2,692.86	0.9264
1	XGBoost	2,054.18	2,754.81	0.9230
2	Random Forest	2,029.31	2,774.83	0.9219

<IPython.core.display.Markdown object>



<IPython.core.display.Markdown object>

3.0.1.1 Insights from the Stacking model

Final visual evaluation and model comparison were conducted on the test set, while train and validation R^2 were also reported to assess possible overfitting. The close validation and test R^2 values suggest that the stacking model generalizes well, while the higher training R^2 indicates expected in-sample fit without severe overfitting.

The test-set results show that the stacking model achieves the best overall performance among the evaluated models. It has the lowest RMSE and MAE, while also having the highest R2.

Although the improvement over Random Forest and XGBoost is relatively small, it is consistent across all evaluation metrics, indicating that combining the models provides a measurable gain in predictive accuracy. It also gives us a clear better model for our research, as Random Forest and XGBoost were previously somewhat tied.

The actual-vs-predicted plots support this conclusion. All three models follow the diagonal closely, but the stacking model seems to stay a bit closer to the line; the difference is minimal, but it can be seen.

The residual plots further confirm the behaviour. The stacking model produces residuals that are centered around zero, with a somewhat tighter, more symmetrical spread. Like other models however, a mild heteroscedasticity remains.

Overall, the results suggests that Random Forest and XGBoost are still both valid models to evaluate the data, but the stacking successfully combines their advantages. As a result, the stacking model provides improved generalization performance compared to the individual models and is selected as the final model for this task.

3.0.2 Anomaly Detection

Anomaly detection is used to better understand model behaviour on unusual or extreme observations. In the context of used car pricing, certain listings may have rare combinations of features, such as very high horsepower, uncommon dimensions, or atypical configurations. These observations may not follow general market trends and can therefore be more difficult for the model to predict accurately.

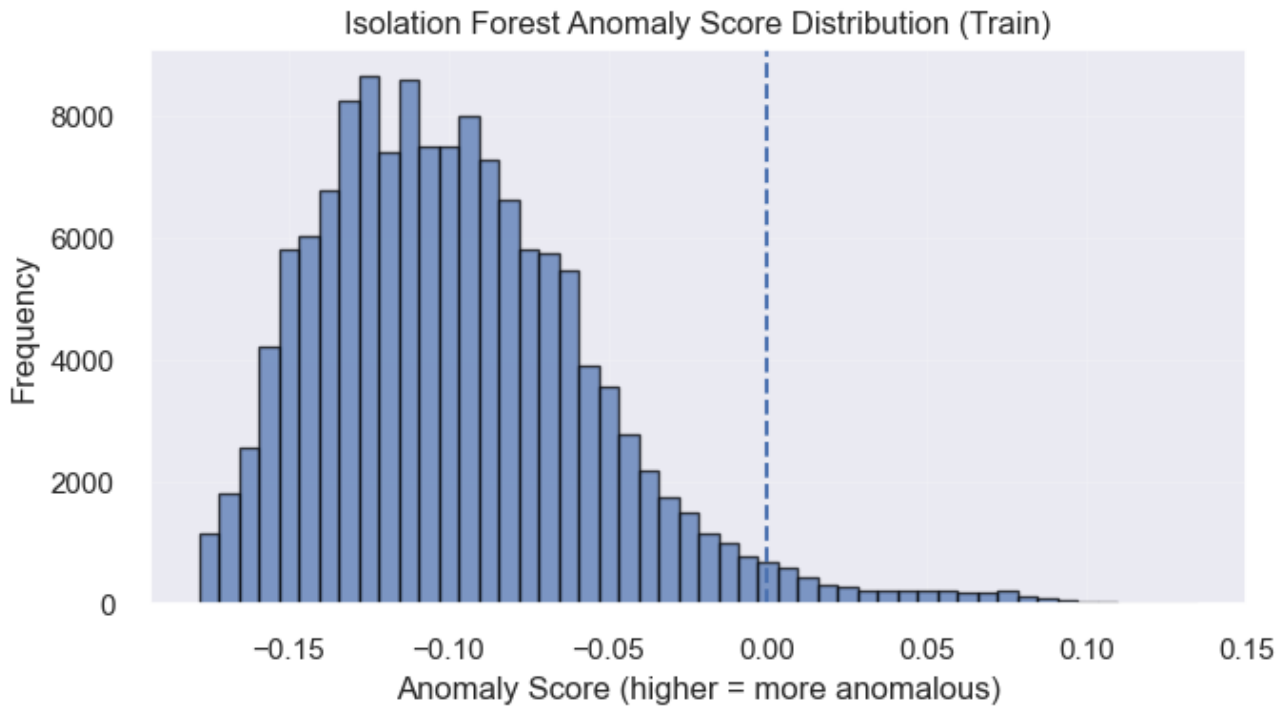
To identify these cases, Isolation Forest is applied as an unsupervised method that detects observations with unusual feature profiles without using the target variable. This allows us to evaluate whether model errors are driven by rare cases and to assess the robustness of the models beyond standard performance metrics.

```
# Fit on training data
anomaly_model, X_train_numeric = fit_anomaly_detection(
    X_train,
    contamination=0.03,
    random_state=42
)

# Display anomalies on train
anomaly_train_results = display_anomaly_results(
    anomaly_model=anomaly_model,
    X_data=X_train,
    dataset_name="Train",
    n_examples=10
)

# Apply same fitted model to test
anomaly_test_results = display_anomaly_results(
    anomaly_model=anomaly_model,
    X_data=X_test,
    dataset_name="Test",
    n_examples=10
)
```

```
<IPython.core.display.Markdown object>
{'normal_count': 134280, 'anomaly_count': 4153, 'anomaly_rate_percent':
3.0000072237111093}
<IPython.core.display.Markdown object>
```

	back_legroom_in	daysonmarket	engine_displacement	front_legroom_in	fuel_tank_gal	height_in	horsepow
310571	39.1	10	5400.0	41.1	28.0	78.3	310
107762	39.0	182	5300.0	41.3	26.0	76.9	320
277785	41.0	16	5600.0	39.6	26.0	75.8	400
272417	39.1	40	5400.0	41.1	28.0	77.2	310
283080	41.0	27	5600.0	39.6	26.0	75.8	400
313275	41.0	21	5600.0	39.6	26.0	75.8	400
58282	39.1	25	5400.0	41.1	28.0	77.2	310
331069	41.0	5	5600.0	39.6	26.0	75.8	400
282815	41.0	62	5600.0	39.6	26.0	75.8	400
174127	35.0	5	4700.0	42.1	21.1	55.7	402

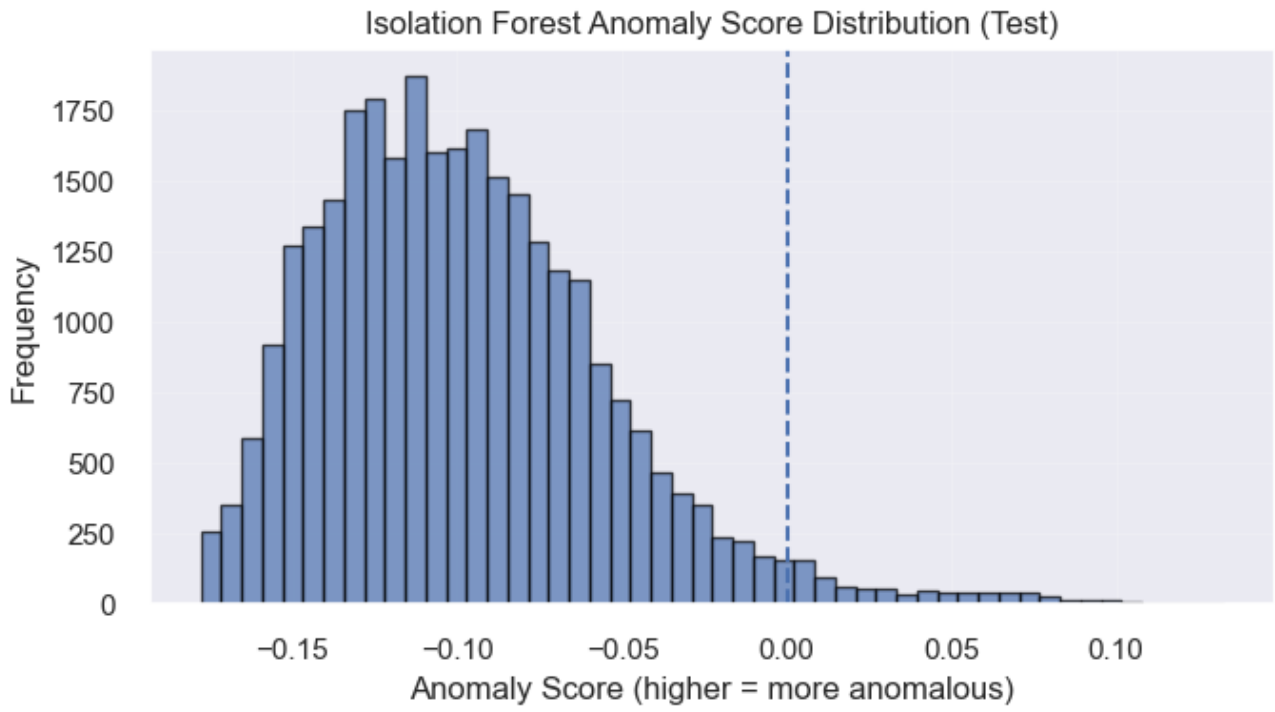
10 rows × 36 columns



<IPython.core.display.Markdown object>
{'normal_count': 28786, 'anomaly_count': 879, 'anomaly_rate_percent': 2.9630878139221304}
<IPython.core.display.Markdown object>

	back_legroom_in	daysonmarket	engine_displacement	front_legroom_in	fuel_tank_gal	height_in	horsepow
275118	40.9	1	5700.0	42.5	26.4	77.0	381
128096	41.0	20	5600.0	39.6	26.0	75.8	400
281994	39.0	148	5300.0	41.3	26.0	76.9	320
338953	35.0	9	4700.0	42.1	21.1	55.7	402
177210	41.0	70	5600.0	39.6	26.0	75.8	400
220265	41.0	8	5600.0	39.6	26.0	75.8	400
320654	31.7	154	4200.0	41.3	16.1	53.8	450
173247	41.0	1	5600.0	39.6	26.0	75.8	400
98331	39.1	7	3500.0	43.0	28.0	77.2	365
164804	39.0	14	5300.0	41.3	26.0	76.9	320

10 rows × 36 columns

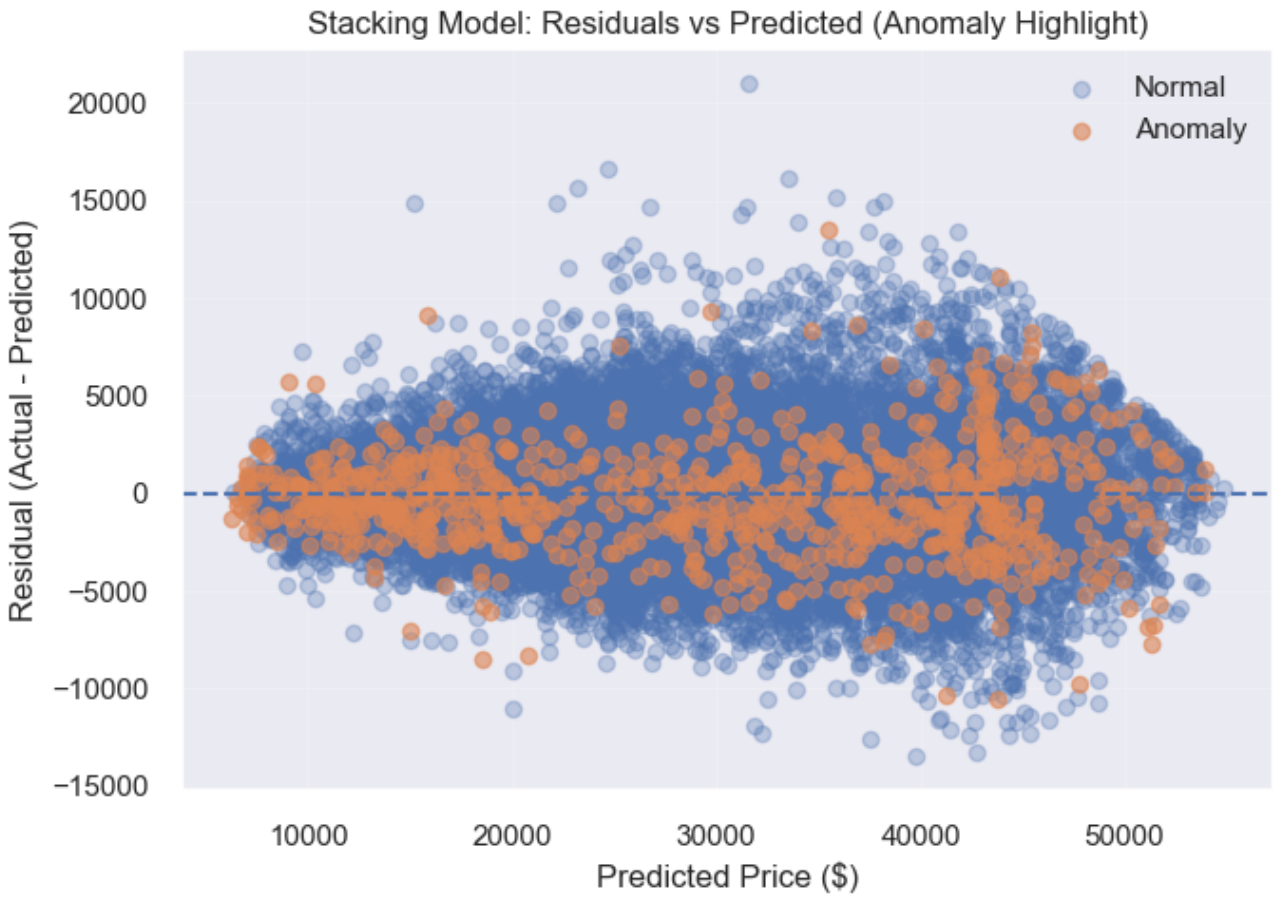


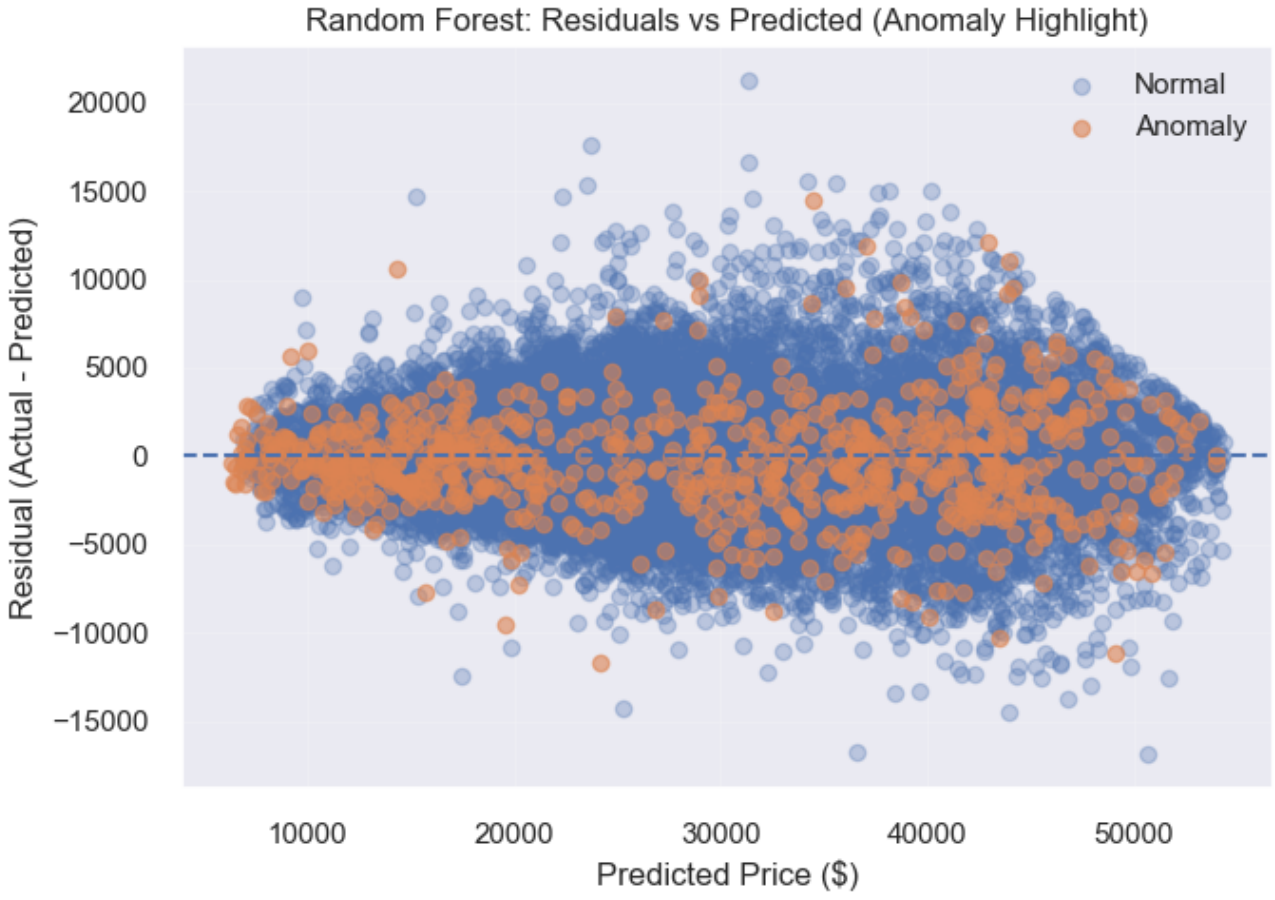
```
models_dict = {
    "Random Forest": randomForest_search.best_estimator_,
    "XGBoost": xgb_search.best_estimator_,
    "Stacking": stacking_model
}

residual_anomaly_comparison = compare_residuals_by_anomaly(
    models_dict=models_dict,
    X_test=X_test,
    y_test=y_test,
    anomaly_results=anomaly_test_results
)
print("Residual Anomaly Comparison")
display(residual_anomaly_comparison)
```

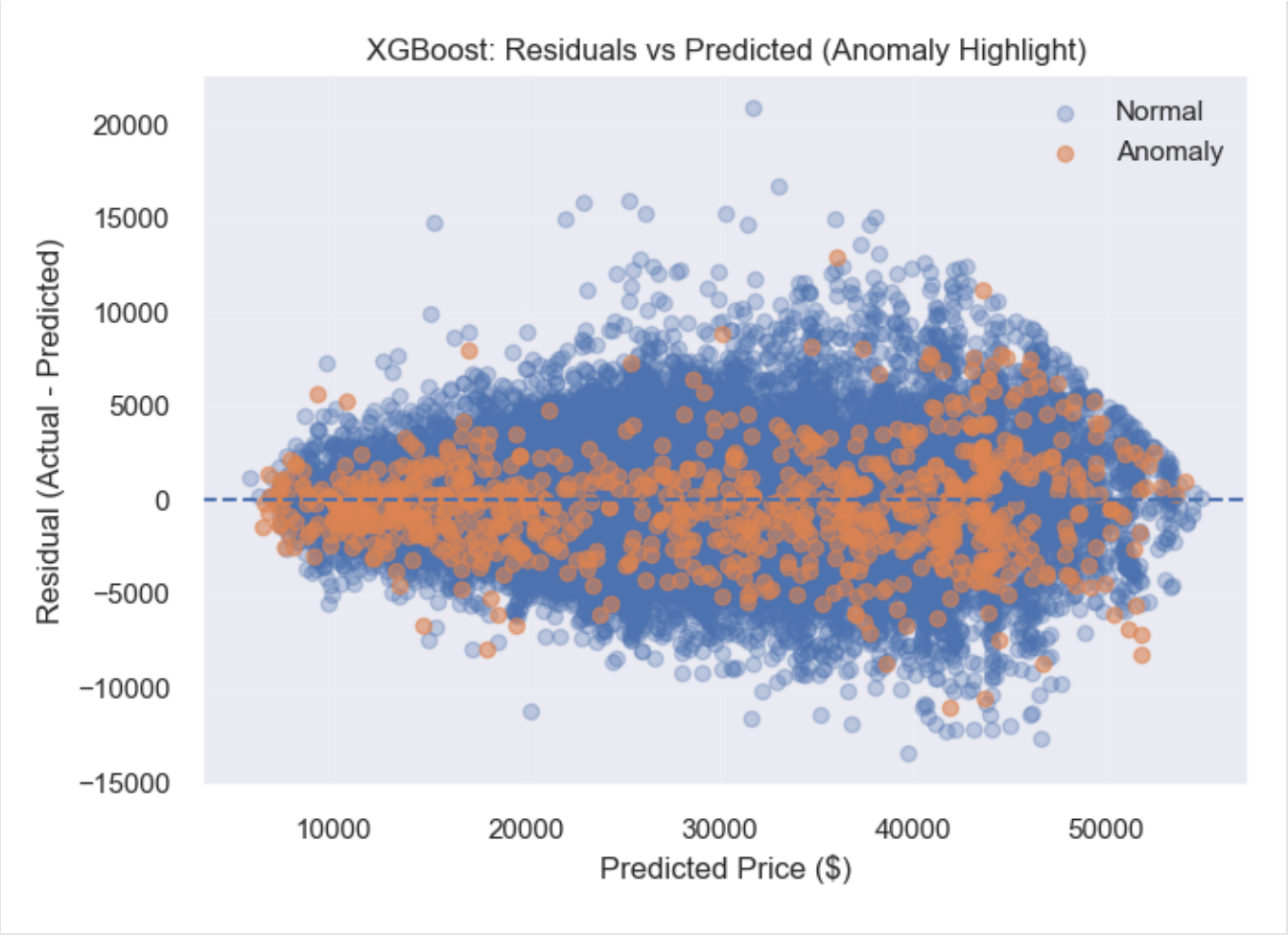
```
Residual Anomaly Comparison
```

	Model	Group	Count	Mean Abs Residual	Median Abs Residual	RMSE
0	Random Forest	Normal	28786	2020.557514	1499.788087	2763.759895
1	Random Forest	Anomaly	879	2315.924359	1806.953655	3115.653452
2	XGBoost	Normal	28786	2051.676706	1545.108398	2752.205111
3	XGBoost	Anomaly	879	2136.067799	1670.585938	2838.634421
4	Stacking	Normal	28786	1990.205946	1490.700051	2687.465203
5	Stacking	Anomaly	879	2157.351255	1628.513872	2864.004265

```
plot_residuals_with_anomalies(
    stacking_model,
    X_test,
    y_test,
    anomaly_test_results,
    "Stacking Model"
)

plot_residuals_with_anomalies(
    randomForest_search.best_estimator_,
    X_test,
    y_test,
    anomaly_test_results,
    "Random Forest"
)

plot_residuals_with_anomalies(
    xgb_search.best_estimator_,
    X_test,
    y_test,
    anomaly_test_results,
    "XGBoost"
)
```


Stacking Model: Residuals vs Predicted (Anomaly Highlight)
Normal
Anomaly
Residual (Actual - Predicted)
20000
15000
10000
5000
0
−5000
−10000
−15000
10000
20000
30000
40000
50000
Predicted Price ($)


Random Forest: Residuals vs Predicted (Anomaly Highlight)
Normal
Anomaly
Residual (Actual - Predicted)
20000
15000
10000
5000
0
−5000
−10000
−15000
10000
20000
30000
40000
50000
Predicted Price ($)



3.0.2.1 Anomaly Analysis and Interpretation

For Anomaly Detection, we applied **Isolation Forest** to identify observations with unusual numeric feature profiles. This method is unsupervised: it does not use the target variable (`price`) and instead detects listings that are harder to isolate within the overall feature space.

To maintain interpretability, anomaly detection was performed only on the **numeric features** of the training set. The model produces both:

- **anomaly labels** (`-1` for anomalous, `1` for normal), and
- a continuous **anomaly score**, where larger values indicate observations that are more unusual relative to the rest of the dataset.

The `contamination` parameter determines the proportion of observations that will be labeled as anomalous. Therefore, the anomaly rate should be interpreted as a modeling threshold rather than proof that those rows are invalid. The main value of this analysis is in identifying which observations appear most atypical and whether they help explain regions where predictive models struggle. Isolation Forest was used to identify approximately 3% of observations as anomalies based on unusual numeric feature profiles. These anomalies correspond to vehicles with extreme or rare characteristics, such as high engine displacement or atypical size attributes.

In our results, approximately 3% of observations are classified as anomalous in both the training and test sets. This proportion is expected, as the contamination parameter of the Isolation Forest was set to 3%, meaning the model is designed to flag roughly this fraction of data points as anomalies. However, observing a similar proportion in the test set confirms that the model generalizes its notion of “unusual” observations consistently across unseen data.

The residual comparison shows that all models produce higher prediction errors for anomalous observations than for normal ones, confirming that these cases are inherently more difficult to predict. However, the increase in error remains moderate across all models, indicating that the models are relatively robust even when handling unusual data points. Among the models, XGBoost exhibits the smallest increase in RMSE between normal and anomalous groups, suggesting slightly better robustness to anomalies, while the stacking model achieves the best overall performance on normal observations, with the lowest RMSE and MAE. This indicates that while stacking improves general accuracy, individual models such as XGBoost may better handle extreme cases.

The residual scatter plots with anomaly highlighting provide additional insight. Normal observations (blue) and anomalous observations (orange) are largely interspersed throughout the prediction space, rather than forming distinct clusters. This indicates that anomalies do not represent a fundamentally different type of data, but instead correspond to more extreme observations within the same distribution. Anomalous points do not dominate the largest prediction errors: the points are spread across the entire predicted price range and are intermixed with normal points, so anomalies are not tied to a specific price range. The plots are similar for the 3 models. This indicates that the models remain robust to unusual data points and that the overall structure of prediction error is primarily driven by the underlying data distribution rather than by anomalies themselves. These findings are consistent with the quantitative results, which showed only a moderate increase in error for anomalous observations.

The anomaly score distribution further supports this interpretation, because the histogram shows a clear peak corresponding to the majority of normal observations, along with a right-skewed tail representing increasingly unusual data points. The anomaly threshold lies within this tail, indicating that anomalies are selected from the most extreme values rather than from a separate cluster. This confirms that anomaly detection identifies edge cases within a continuous distribution rather than distinct outliers.

Overall, anomaly detection provides valuable insight into model behaviour. It complements standard evaluation metrics by showing that while anomalies are more difficult to predict, they do not fundamentally degrade model performance, and that prediction uncertainty is more strongly driven by overall data variability rather than a specific price segment; the anomalous observations correspond to high-performance or uncommon vehicle configurations, which represent edge cases in the dataset rather than data errors.

The anomalies increase prediction error, but only moderately, indicating that the models generalize well even to rare or atypical observations. Therefore, anomalies do not significantly degrade model performance, this suggests that they represent valid but rare observations rather than data errors, and so removing them is not necessary.

3.1 Comprehensive Evaluation

Below we tie **Research Question 1** (supervised price prediction) and **Research Question 2** (unsupervised vehicle groupings) to the experiments and metrics reported earlier in this notebook.

3.1.1 Supervised learning - Research Question 1

Research Question 1 asks whether we can predict **listed used-vehicle price** from observable features.

Across Phase 1 and Phase 2 we built a **reproducible pipeline**: cleaned data (`clean_cars.csv`), EDA, a **baseline linear regression**, then **feature engineering**, comparison of filter / wrapper / embedded selection, and **Random Forest**, **XGBoost**, and **Ensemble Stacking** on the engineered feature set with evaluation on train / validation / test splits. The baseline model achieved roughly **R² # 0.73** with **MAE #** $4,200 * *(see the metric tables in §1.4 and discussion in §2.2). After engineering, the full engineered feature n$ **2,000** (see metric tables in 2.1 and 3.0.1).

Partial dependence plots (§2.4) on the Random Forest illustrate how predicted **log-price** moves with **year**, **mileage**, **horsepower**, and **age**, aligning with EDA: newer vehicles and higher horsepower tend to associate with higher predicted prices; higher mileage associates with lower prices. Overall, supervised models give a **usable baseline** for ranking and ballpark pricing, but error is still material for cheap vs luxury tails—consistent with residual patterns discussed in Phase 1.

3.1.2 Unsupervised learning - Research Question 2

Research Question 2 asks whether there are **natural groupings** of vehicles in feature space **without** using price as a clustering input.

In §2.3 we standardized numeric features, ran **K-Means** for several values of *k*, chose *k* using the **silhouette score** (on a subsample for tractability), and visualized structure with **PCA** in 2D. **Price was excluded** from the clustering inputs so clusters reflect specs and listing attributes; we then compared **price distributions across clusters** (box plots / summaries). The interpretation in §2.3 applies here: if median price differs across clusters, those clusters act as **descriptive market segments** (e.g. different feature mixes associated with different price levels), not a second supervised model of price. Segments are **exploratory** and may help storytelling or stratified modeling, but they do not replace the supervised objective in RQ1. In this case, these segments primarily reflect differences in vehicle age, usage, and performance, which align with the key drivers of price identified in the supervised analysis.

3.1.3 Final evaluation - insights and answers to the research questions

- **RQ1 (prediction):** We can explain a **large share of price variance** (~73% R^2 for the linear baseline; improved to ~92% R^2 with engineered features and tree models) with transparent, checkable steps. Predictions are **most reliable for mid-range listings**; luxury and very low-price extremes remain harder, as expected from skewed prices and heterogeneous listings. This is likely due to the increased variability and heterogeneity in vehicle characteristics at extreme price ranges, making these observations inherently more difficult to predict.
- **RQ2 (segments): K-Means + PCA** supports interpretable **multidimensional segments** when silhouette and cluster–price profiles are coherent. These segments complement RQ1 by describing **who clusters together** in feature space, not by maximizing pricing accuracy.
- **Limitations:** Models are fit on **historical listings** in one market snapshot; **distribution shift**, **missing data**, and **dealer-specific pricing** are not fully captured. Clustering used **numeric** columns and subsampling for speed; different *k* or features could change segments.

In addition, the stacking model provides the most reliable predictions by combining the strengths of multiple models, while the anomaly detection analysis confirms that unusual observations do not significantly degrade model performance. Together, these results strengthen the reliability of the predictive framework and provide a more complete understanding of both typical and atypical observations in the dataset.

Together, supervised models address **“how much?”** while unsupervised analysis addresses **“which groups?”** —both are needed for a rounded view of the used-car listing data.

3.2 Ethical Considerations

- **Bias and fairness:** Training on **online listings** can encode **historical pricing patterns** that reflect seller behavior, geography, or demographic proxies (e.g. neighborhood, dealer). A model that reproduces those patterns could **perpetuate inequities** if used for consumer-facing decisions without oversight.
- **Transparency:** For any **automated price suggestion**, users should know **inputs, uncertainty**, and that errors (MAE ~ thousands of dollars) are **not equally distributed** across vehicle types and price bands. Similarly, clustering-based segmentation may group vehicles in ways that reflect underlying market biases rather than purely objective differences. This is also reflected in the anomaly analysis, where certain observations exhibit higher prediction errors, highlighting that unusual or less-represented vehicle types may be treated less reliably by the model.
- **Use case:** These tools are best framed as **decision support** (exploration, inventory grouping), not as sole authority for financing, insurance, or legally sensitive pricing without human review and domain checks.
- **Data privacy:** Listings may include **identifiers or location**; production systems should **minimize retention** and comply with applicable privacy rules.

We intend this notebook as an **educational, reproducible analysis**; deployment would require additional validation, monitoring, and governance beyond this course scope.

3.3 Limitations and Future Work

Several limitations should be considered when interpreting these results. First, the models are trained on historical online listings, which represent a specific market snapshot and may not generalize well under distribution shifts, such as changes in market demand or pricing trends.
Second, the dataset lacks detailed information on vehicle condition, accident history, and negotiation context, which are important factors influencing price but are not captured in the available features. This contributes to the remaining unexplained variance observed across all models.

Regarding the methodology, the feature set was restricted to numerical variables for the main experiments. While this simplifies computation, it excludes potentially informative categorical variables such as vehicle brand and body type.
While preliminary experiments showed limited performance gains from including these features (near same R2, RMSE and MAE for XGBoost, and the same best feature importance), their exclusion may still reduce model interpretability and limit the ability to capture certain market segment effects.

Additionally, while anomaly detection highlights unusual observations, these points are not necessarily errors but rather rare or extreme cases. Their presence indicates that the model may be less reliable for certain types of vehicles, particularly those that are underrepresented in the dataset.

Finally, the clustering analysis shows only moderate separation (silhouette ~0.27), suggesting that while meaningful groupings exist, the dataset does not contain strongly distinct segments. This also limits the interpretability of clusters, as the differences between groups may not be sufficiently pronounced to clearly distinguish distinct categories of vehicles. As a result, while clustering provides useful exploratory insights, it should be interpreted cautiously and not assumed to represent clearly defined market segments.

Future work could focus on improving predictive performance and interpretability by incorporating additional data sources. More advanced modeling approaches, including gradient boosting enhancements, more advanced hyperparameter tuning or neural networks, could also be explored to better capture complex relationships in the data.

In addition, further work could refine the clustering analysis by exploring alternative algorithms (e.g., hierarchical clustering or density-based methods) and by incorporating mixed data types, which may lead to more distinct and interpretable segments.

Finally, deploying such a model in a real-world setting would require continuous monitoring, bias evaluation, and recalibration to ensure fairness, robustness, and adaptability to changing market conditions. Additionally, future work could further investigate the behavior of anomalies and prediction errors across different segments of the data, to better understand where the model performs less reliably and improve overall robustness.